

AeroPorter

Complete English Defense Handbook

Мобильный робот-ассистент · пособие к защите на английском

☐ AeroPorter — Complete English Defense Handbook

Мобильный робот-ассистент для аэропорта · Полное пособие к защите на английском

Project — AeroPorter

Team — Ilya Karyakin (Grade 9), Stanislav Paramonov (Grade 9), Artur Sabirzanov (Grade 11)

Purpose — Concentrate of vocabulary, phrases, jury Q&A, and defense scripts that actually help at an engineering defense

Language — English (with Russian support where useful)

Honesty rule — Prefer accurate claims over impressive ones. Demo ≠ certified airport deployment.

How to use this handbook

1. Week -2: Learn Section 1 (vocabulary) + Section 7 (AeroPorter-specific terms).
2. Week -1: Memorize Section 2 phrases + Section 8 slide script.
3. 3-4 days before: Drill Section 3-5 questions aloud; practice Short answers first.
4. 1-2 days before: Run 2-3 Mock Interviews from Section 9.
5. Morning of defense: Skim Section 10 mistakes + Section 11.3 (buy time) + elevator pitch.

Golden rule: If you do not know — say what you *do* know, what you tested, and what remains future work. Never invent a LiDAR brand name, Nav2 usage, or “already working in a real terminal” if that is not proven.

One-sentence project thesis (memorize)

AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype with a custom ROS 2 LiDAR mapping and A→B navigation stack and a multilingual passenger kiosk; teleop, mapping, and demo escort/ticket flows exist in software, while real-terminal deployment and industrial safety remain explicitly out of claimed results.

Technical snapshot (truth table)

Topic — What is true in the repo

Compute — Raspberry Pi 5 + Arduino Mega 2560

Drive — 4× mecanum wheels, serial teleop bridge, encoder odometry (FR encoder untrusted)

Sensing — LiDAR via `cspc_lidar` (ROS 2), USB camera (YuNet face), optional Tapo C200 prototype

UI — 10.1" 1280×800 kiosk, languages RU / EN / ZH / TT

Navigation — Custom ROS 2 Jazzy stack: occupancy grid, A*, pure pursuit, CSM + ICP — not Nav2

Safety — Arduino cmd timeout ~1.2 s; drive watchdog ~0.25 s; kiosk heartbeat lease ~6 s → stop

Airport claim — Navigation program written; real airport operation is not claimed

Table of contents

1. [Technical Vocabulary](#1-technical-vocabulary) — 900–1200 defense-ready terms
2. [Most Useful Phrases](#2-most-useful-phrases) — 300+ presentation / Q&A phrases
3. [Jury Questions](#3-jury-questions) — 220+ questions with intent
4. [Ready Answers](#4-ready-answers) — short / full / alternative
5. [Tricky Questions](#5-tricky-questions) — depth checks + strong answers
6. [Follow-up Questions](#6-follow-up-questions) — chains after the main question

7. [AeroPorter Vocabulary](#7-aeroporter-vocabulary) — project-only glossary
 8. [Fully Ready Defense](#8-fully-ready-defense) — intro, slides, closing
 9. [Mock Interviews](#9-mock-interviews) — 12 full rehearsals
 10. [Most Common Mistakes](#10-most-common-mistakes)
 11. [Useful Add-ons](#11-useful-add-ons) — linkers, hedges, comparisons, buy-time phrases
 12. [Bonus](#bonus) — pronunciation, pitches, emergency recovery
-

Legend

Marker — Meaning

Short — ~20–30 seconds

Full — ~1–2 minutes

Alternative — Another angle if the first answer is cut short

↓ — Follow-up the jury may ask next

□ / □ — Common mistake → preferred form

Study the concentrate, not every English word in existence. Speak clearly, admit limits, defend design choices with reasons.

1. Technical Vocabulary

Only defense-useful engineering terms. Format: EN — RU | gloss

1.1 Mechanics (236 terms)

1. chassis — шасси | structural base that carries motors, batteries, and sensors
2. frame — рама | rigid structure supporting the robot body
3. mecanum wheel — механум-колесо | wheel with rollers at 45° enabling omnidirectional motion
4. roller — ролик | small angled cylinder on a mecanum wheel
5. axle — ось | shaft that a wheel rotates around
6. bearing — подшипник | reduces friction between rotating and fixed parts
7. gearbox — редуктор | gear assembly that reduces speed and increases torque
8. torque — крутящий момент | rotational force produced by a motor
9. traction — сцепление | grip between wheel and floor
10. payload — полезная нагрузка | mass the robot can usefully carry
11. ground clearance — клиренс | distance between floor and lowest robot point
12. wheelbase — колёсная база | distance between front and rear wheel centers
13. track width — колея | distance between left and right wheels
14. center of mass — центр масс | point where mass is concentrated; affects tipping
15. vibration — вибрация | oscillation that can disturb sensors and fasteners
16. fastener — крепеж | bolt, screw, or nut that holds parts together
17. mount — крепление | bracket or plate used to fix a component
18. enclosure — корпус | protective housing around electronics
19. bumper — бампер | edge protection that absorbs light collisions
20. caster — поворотное колесо | passive swivel wheel (not used on AeroPorter drive)
21. differential drive — дифференциальный привод | two-wheel drive layout (contrast with mecanum)
22. omnidirectional — всенаправленный | able to move in any planar direction
23. strafe — движение боком | sideways translation without rotating
24. yaw — рыскание | rotation about the vertical axis
25. pitch — тангаж | rotation about the lateral axis
26. roll — крен | rotation about the longitudinal axis
27. degree of freedom — степень свободы | independent axis of motion
28. actuator — исполнительный механизм | device that produces motion
29. linkage — шарнирная связь | mechanical connection transferring motion
30. belt drive — ременная передача | power transmission via belt
31. chain drive — цепная передача | power transmission via chain
32. coupling — муфта | connects two shafts
33. backlash — люфт | lost motion between mating parts
34. stiffness — жесткость | resistance to elastic deformation
35. damping — демпфирование | energy dissipation that reduces oscillation
36. friction — трение | resistance to relative motion
37. wear — износ | material loss from use over time
38. tolerance — допуск | allowed dimensional deviation
39. alignment — соосность / выравнивание | correct relative position of parts
40. rigidity — жесткость конструкции | ability to keep shape under load
41. prototype — прототип | early working version for testing ideas
42. fabrication — изготовление | process of making physical parts
43. 3D printing — 3D-печать | additive manufacturing of plastic parts
44. laser cutting — лазерная резка | cutting sheet materials with a laser
45. assembly — сборка | putting parts together into a system

46. disassembly — разборка | taking the system apart for repair
47. maintenance — обслуживание | scheduled care to keep the system working
48. calibration fixture — калибровочное приспособление | jig that helps set known reference positions
49. load cell — тензодатчик | sensor measuring force (future option)
50. shock absorber — амортизатор | device reducing impact forces
51. heat sink — радиатор | metal part that dissipates heat
52. cable management — укладка кабелей | routing wires to avoid snags and noise
53. strain relief — разгрузка натяжения | protects a cable from being pulled out
54. footprint — габарит в плане | floor area occupied by the robot (~0.82×0.56 m)
55. turning radius — радиус поворота | space needed to change heading
56. static friction — статическое трение | friction before motion starts
57. kinetic friction — кинетическое трение | friction during motion
58. slip — проскальзывание | wheel rotation without matching floor motion
59. odometry error — ошибка одометрии | pose drift from imperfect wheel sensing
60. mechanical advantage — механическое преимущество | force amplification through mechanisms
61. moment of inertia — момент инерции | resistance to angular acceleration
62. counterweight — противовес | mass used to balance the robot
63. bracket — кронштейн | angled support for mounting
64. spacer — проставка | part that sets fixed distance between pieces
65. washer — шайба | disk under a nut/bolt head
66. thread — резьба | helical ridge on a screw
67. nut — гайка | internally threaded fastener
68. bolt — болт | externally threaded fastener
69. set screw — стопорный винт | screw that locks a hub on a shaft
70. hub — ступица | central part connecting wheel to shaft
71. sprocket — звездочка | toothed wheel for chain
72. pulley — шкив | wheel guiding a belt
73. idler — натяжной ролик | pulley that tensions a belt/chain
74. shaft collar — стопорное кольцо | clamp that holds axial position on a shaft
75. keyway — шпоночный паз | slot for a key transmitting torque
76. shear — срез | failure mode from parallel force
77. fatigue — усталость материала | failure after repeated load cycles
78. yield strength — предел текучести | stress where permanent deformation begins
79. ultimate strength — предел прочности | maximum stress before fracture
80. factor of safety — коэффициент запаса | design margin above expected load
81. CAD — САПР | computer-aided design of parts
82. BOM — спецификация | bill of materials — parts list
83. kinematic chain — кинематическая цепь | series of linked moving bodies
84. workspace — рабочая зона | reachable area for a mechanism
85. singularity — сингулярность | configuration where control becomes ill-conditioned
86. compliance — податливость | intentional elasticity in a structure
87. hard stop — механический упор | physical limit preventing over-travel
88. limit switch — концевой выключатель | switch detecting end of travel
89. cable chain — кабель-канал / кабельная цепь | guided flexible cable carrier
90. grommet — втулка / проходка | protects wires through a hole
91. thermal expansion — тепловое расширение | size change with temperature
92. corrosion — коррозия | chemical degradation of metal
93. lubrication — смазка | reducing friction with lubricant
94. seal — уплотнение | keeps dust/moisture out

95. IP rating — степень защиты IP | ingress protection against dust/water
96. service hatch — сервисный люк | opening for maintenance access
97. modular design — модульная конструкция | system built from replaceable blocks
98. rapid prototyping — быстрое прототипирование | fast build-test iteration
99. fit — посадка | how tightly mating parts assemble
100. clearance fit — посадка с зазором | intentional gap between parts
101. interference fit — посадка с натягом | parts pressed together tightly
102. jig — кондуктор | tool that guides manufacturing/assembly
103. fixture — приспособление | holds workpiece during machining
104. deburring — удаление заусенцев | smoothing sharp edges after cutting
105. surface finish — чистота поверхности | texture quality of a surface
106. anodizing — анодирование | protective/coating process for aluminum
107. welding — сварка | joining metals by melting
108. soldering (mech sense) — пайка (механический контекст) | joining with filler metal — see electronics too
109. rivet — заклепка | permanent mechanical fastener
110. adhesive — клей | bonding agent for joining parts
111. velcro mount — крепление на липучке | quick-release fabric fastener mount
112. hot-swap — горячая замена | replace a module without full teardown
113. tool-less — без инструментов | designed for assembly without tools
114. ergonomics — эргономика | human-friendly physical design
115. center of gravity — центр тяжести | same idea as center of mass under gravity
116. tip-over stability — устойчивость к опрокидыванию | resistance to falling when accelerating
117. dynamic load — динамическая нагрузка | force from motion and impacts
118. static load — статическая нагрузка | force when the robot is still
119. resonant frequency — резонансная частота | frequency that amplifies vibration
120. NVH — шум, вибрация, жесткость | noise, vibration, harshness considerations
121. wire harness — жгут проводов | bundled cables routed through the chassis
122. power distribution board — плата распределения питания | board that splits battery power to subsystems
123. battery bay — отсек АКБ | compartment holding the battery pack
124. access panel — съёмная панель | removable cover for service
125. skid plate — защитная пластина | underside protection plate
126. fender — крыло | cover shielding a wheel
127. handle — ручка | grip for carrying the prototype
128. tie-down — швартовка / фиксация | point used to secure the robot during transport
129. leveling foot — опора для выравнивания | adjustable foot for stationary platforms
130. anti-tip bar — противоопрокидывающая планка | bar reducing tip risk
131. drive train — трансмиссия / приводная линия | path of power from motor to wheel
132. reduction ratio — передаточное число | input speed divided by output speed
133. stall torque — момент при стопоре | torque at zero speed under load
134. rated speed — номинальная скорость | intended operating speed
135. duty cycle (mech) — продолжительность включения | fraction of time under load
136. overhang — вылет | part extending beyond the wheel support
137. cantilever — консоль | beam fixed at one end only
138. shear pin — срезной штифт | sacrificial pin that breaks to protect the drive
139. clutch — сцепление (муфта) | device that can disconnect drive torque
140. brake — тормоз | device that stops or holds motion
141. parking brake — стояночный тормоз | holds robot when powered down
142. freewheel — обгонная муфта | allows rotation in one direction freely
143. universal joint — карданный шарнир | joint transmitting torque at an angle

- 144. ball joint — шаровой шарнир | spherical joint for multi-axis motion
- 145. linear rail — линейная направляющая | precision sliding guide
- 146. lead screw — ходовой винт | screw converting rotation to linear motion
- 147. rack and pinion — рейка и шестерня | converts rotation to linear travel
- 148. cam — кулачок | shaped lobe producing timed motion
- 149. spring — пружина | stores mechanical energy elastically
- 150. torsion spring — торсионная пружина | spring resisting twisting
- 151. compression spring — пружина сжатия | spring resisting squeezing
- 152. extension spring — пружина растяжения | spring resisting pulling
- 153. gas strut — газовый упор | damper/strut using compressed gas
- 154. latch — защёлка | mechanical catch holding a panel closed
- 155. hinge — петля | joint allowing a door/panel to swing
- 156. quick-release pin — палец быстрого съема | pin removable without tools
- 157. dowel pin — установочный штифт | precision pin for alignment
- 158. standoff — стойка / втулка | spacer that mounts boards above a surface
- 159. PCB standoff — стойка для платы | keeps electronics board elevated
- 160. cable tie — стяжка | nylon tie for bundling wires
- 161. heat-shrink tubing — термоусадка | tubing that shrinks with heat for insulation
- 162. ferrule — наконечник | metal tip for stranded wire ends
- 163. terminal block — клеммник | screw/clamp block for wire connections
- 164. bus bar — шина | conductive bar distributing current
- 165. ground lug — клемма массы | connection point to chassis ground
- 166. EMI shield — экран ЭМП | conductive cover reducing electromagnetic interference
- 167. ventilation slot — вентиляционная щель | opening for airflow and cooling
- 168. dust filter — пылевой фильтр | mesh reducing dust ingress
- 169. desiccant — влагопоглотитель | material that absorbs moisture
- 170. rubber foot — резиновая ножка | vibration-isolating pad
- 171. silicone pad — силиконовая прокладка | soft isolator / gripper material
- 172. neoprene — неопрен | rubber-like material used for gaskets
- 173. acrylic — акрил | clear plastic often used for covers
- 174. ABS plastic — АБС-пластик | common 3D-print / enclosure plastic
- 175. PLA — PLA | easy 3D-print polymer for prototypes
- 176. PETG — PETG | tougher 3D-print polymer
- 177. aluminum extrusion — алюминиевый профиль | T-slot structural rail
- 178. sheet metal — листовый металл | thin metal formed into panels
- 179. honeycomb panel — сотовая панель | lightweight stiff sandwich panel
- 180. carbon fiber — углепластик | light stiff composite (advanced builds)
- 181. foam core — пенопластовый сердечник | light filler in sandwich structures
- 182. edge banding — кромкование | finishing exposed panel edges
- 183. fillet — скругление | rounded inside corner for strength
- 184. chamfer — фаска | beveled edge
- 185. draft angle — уклон | taper aiding molded part release
- 186. boss — бобышка | raised feature for screws in plastic parts
- 187. rib — ребро жесткости | thin wall that stiffens a part
- 188. undercut — поднутрение | geometry hard to mold/machine
- 189. warp — коробление | unwanted bend after printing/cooling
- 190. layer adhesion — адгезия слоёв | bond strength between 3D-print layers
- 191. infill — заполнение | internal density pattern in 3D prints
- 192. wall thickness — толщина стенки | structural thickness of a shell

193. live axle — ведущая ось | axle that transmits drive torque
194. dead axle — ведомая ось | axle that only supports, does not drive
195. kingpin — шкворень | pivot pin in a steering assembly
196. steering knuckle — поворотный кулак | hub carrier in steered vehicles
197. suspension — подвеска | system isolating body from wheel motion
198. unsprung mass — неподрессоренная масса | mass not supported by suspension
199. sprung mass — поддрессоренная масса | mass supported by suspension
200. roll center — центр крена | geometric point affecting body roll
201. scrub radius — плечо обкатки | steering geometry parameter
202. Ackermann steering — рулевая схема Аккермана | geometry for differential wheel angles in turns
203. skid steering — бортовой поворот | turning by left/right speed difference
204. holonomic — голономный | can control all planar DOF independently — mecanum is holonomic
205. non-holonomic — неголономный | motion constraints prevent sideways free motion
206. kinematic model — кинематическая модель | math model of motion without forces
207. dynamic model — динамическая модель | math model including mass and forces
208. inverse kinematics — обратная кинематика | from desired motion to wheel commands
209. forward kinematics — прямая кинематика | from wheel motion to body motion
210. Jacobian — якобиан | matrix relating joint speeds to end-effector speeds
211. workspace envelope — рабочая оболочка | volume/area the system can operate in
212. payload capacity — грузоподъемность | max useful load rating
213. curb weight — снаряженная масса | mass of the robot ready to run
214. gross weight — полная масса | robot plus payload
215. weight distribution — развесовка | how mass is shared across wheels
216. contact patch — пятно контакта | area where wheel touches floor
217. rolling resistance — сопротивление качению | force opposing wheel roll
218. gradeability — преодолеваемый уклон | max slope the robot can climb
219. step climbing — преодоление порога | ability to cross a small step
220. gap crossing — преодоление щели | ability to cross a floor gap
221. soft floor — мягкий пол | carpet/soft surface that worsens odometry
222. hard floor — твердый пол | tile/concrete better for wheel odometry
223. threshold — порог | small step at doorways — airport challenge
224. ramp — пандус | inclined surface for accessibility
225. handrail clearance — зазор до поручня | space needed near rails in terminals
226. serviceability — ремонтпригодность | how easy the machine is to repair
227. mean time between failures — наработка на отказ | reliability metric (MTBF)
228. mean time to repair — среднее время ремонта | maintainability metric (MTTR)
229. spare part — запасная часть | replacement component kept in stock
230. consumable — расходник | part replaced regularly (belts, etc.)
231. field replaceable unit — блочно-сменный модуль | module swappable in the field
232. acceptance test — приемочный тест | test proving the build meets requirements
233. endurance test — ресурсный тест | long-run test for wear and overheating
234. drop test — испытание на падение | checks survival after a drop (lab)
235. ingress test — испытание на проникновение | checks dust/water sealing
236. noise test — испытание шума | measures acoustic level during motion

1.2 Electronics (243 terms)

1. microcontroller — микроконтроллер | chip that runs firmware — Arduino Mega 2560 on AeroPorter
2. Raspberry Pi — Raspberry Pi | single-board computer running Linux and ROS 2
3. Arduino Mega 2560 — Arduino Mega 2560 | MCU board with many I/O pins for motors/encoders

4. GPIO — GPIO | general-purpose input/output pins
5. PWM — ШИМ | pulse-width modulation for motor speed control
6. H-bridge — H-мост | circuit that drives a motor forward/reverse
7. motor driver — драйвер мотора | board/chip amplifying MCU signals to motors
8. encoder — энкодер | sensor measuring wheel rotation
9. quadrature encoder — квадратурный энкодер | encoder with A/B channels for direction
10. ticks per revolution — тики на оборот | encoder counts per full wheel turn (20 on AeroPorter)
11. baud rate — скорость передачи | serial bits per second (115200 to Mega)
12. UART — UART | serial communication interface
13. USB — USB | common serial/power bus for LiDAR and Mega
14. I2C — I2C | two-wire bus for sensors
15. SPI — SPI | fast serial peripheral interface
16. ADC — АЦП | analog-to-digital converter
17. DAC — ЦАП | digital-to-analog converter
18. logic level — логический уровень | voltage representing 0/1 (often 3.3 V or 5 V)
19. level shifter — преобразователь уровней | adapts 3.3 V ↔ 5 V signals
20. pull-up resistor — подтягивающий резистор | resistor holding a line high by default
21. pull-down resistor — стягивающий резистор | resistor holding a line low by default
22. debouncing — антидребезг | filtering noisy switch/encoder edges
23. noise — помехи | unwanted electrical signals
24. EMI — ЭМП | electromagnetic interference
25. ground loop — земляная петля | unwanted current path through grounds
26. common ground — общая земля | shared reference for circuits
27. decoupling capacitor — развязывающий конденсатор | local capacitor stabilizing supply voltage
28. voltage regulator — стабилизатор напряжения | keeps output voltage steady
29. buck converter — понижающий преобразователь | efficient step-down DC-DC converter
30. boost converter — повышающий преобразователь | step-up DC-DC converter
31. Li-ion battery — Li-ion аккумулятор | high energy-density rechargeable battery
32. LiPo battery — LiPo аккумулятор | lithium polymer pack often used in robots
33. battery management system — BMS | protects pack from over/under voltage and overcurrent
34. state of charge — уровень заряда | estimated remaining battery energy
35. brownout — просадка питания | voltage drop causing resets/glitches
36. inrush current — пусковой ток | surge when motors/capacitors start
37. fuse — предохранитель | sacrificial overcurrent protection
38. circuit breaker — автоматический выключатель | resettable overcurrent protection
39. current sense — измерение тока | measuring amp draw for protection/control
40. shunt resistor — шунт | low-value resistor for current measurement
41. optocoupler — оптопара | isolates signals optically
42. relay — реле | electrically controlled switch
43. MOSFET — MOSFET | transistor often used in motor drivers
44. BJT — биполярный транзистор | current-controlled transistor
45. diode — диод | allows current primarily one way
46. flyback diode — обратный диод | protects against inductive motor spikes
47. LED — светодиод | indicator or status light
48. buzzer — зуммер | simple audible alert
49. crystal oscillator — кварцевый генератор | provides stable clock for MCU
50. watchdog timer — сторожевой таймер | resets system if software hangs
51. firmware — прошивка | software on the microcontroller
52. bootloader — загрузчик | program that loads firmware

53. flash memory — флеш-память | non-volatile program storage
54. EEPROM — EEPROM | small non-volatile data storage
55. SRAM — SRAM | volatile working memory
56. pinout — распиновка | map of connector/board pins
57. Dupont wire — провод Dupont | jumper wire for prototypes
58. ribbon cable — шлейф | flat multi-wire cable
59. shielded cable — экранированный кабель | cable with EMI shield
60. twisted pair — витая пара | wires twisted to reduce noise
61. BNC — BNC | coax connector (lab gear)
62. JST connector — разъем JST | common compact power/signal connector
63. XT60 — XT60 | power connector for battery packs
64. barrel jack — цилиндрический разъем | common DC power plug
65. USB-C PD — USB-C PD | USB Power Delivery charging standard
66. PoE — PoE | Power over Ethernet
67. RTSP — RTSP | streaming protocol used by Tapo cameras
68. MIPI CSI — MIPI CSI | camera interface on some SBCs
69. HDMI — HDMI | video output for monitors
70. DSI — DSI | display serial interface
71. touch controller — контроллер касаний | chip reading the touchscreen
72. LVDS — LVDS | high-speed differential display signaling
73. oscilloscope — осциллограф | instrument viewing waveforms
74. multimeter — мультиметр | measures V/I/R
75. logic analyzer — логический анализатор | captures digital signal timing
76. soldering iron — паяльник | tool for soldering joints
77. solder joint — паяное соединение | electrical/mechanical solder connection
78. cold joint — холодная пайка | poor solder joint that fails intermittently
79. flux — флюс | helps solder wet the metal cleanly
80. PCB — печатная плата | printed circuit board
81. schematic — схема | symbolic electrical drawing
82. netlist — нетлист | list of electrical connections
83. ground plane — земляная плоскость | copper layer used as ground
84. trace — дорожка | copper path on a PCB
85. via — переходное отверстие | plated hole connecting PCB layers
86. silkscreen — шелкография | printed labels on a PCB
87. footprint (PCB) — посадочное место | copper pattern for a component
88. THT — выводной монтаж | through-hole technology
89. SMD — SMD | surface-mount device
90. reflow — рефлю | oven process soldering SMD parts
91. hot air station — термовоздушная станция | tool for SMD rework
92. ESD — ЭСП | electrostatic discharge — can kill chips
93. antistatic wrist strap — антистатический браслет | grounds the technician safely
94. isolation — гальваническая развязка | separating circuits electrically
95. opto-isolation — опторазвязка | isolation using light
96. common-mode noise — синфазная помеха | noise appearing equally on both lines
97. differential signal — дифференциальный сигнал | signal sent as voltage difference
98. impedance — импеданс | AC resistance of a circuit
99. capacitance — емкость | ability to store charge
100. inductance — индуктивность | property opposing current change — motors have it
101. RC filter — RC-фильтр | resistor-capacitor noise filter

102. LC filter — LC-фильтр | inductor-capacitor filter
103. snubber — снаббер | circuit damping inductive spikes
104. TVS diode — TVS-диод | transient voltage suppressor
105. polyfuse — самовосстанавливающийся предохранитель | PTC resettable fuse
106. thermal shutdown — тепловая защита | auto cut-off when overheating
107. derating — снижение номинала | running below max ratings for reliability
108. duty cycle (PWM) — скважность ШИМ | fraction of time PWM is high
109. dead time — мертвое время | delay preventing H-bridge shoot-through
110. shoot-through — сквозной ток | both H-bridge sides on — destructive
111. back EMF — противо-ЭДС | voltage generated by spinning motor
112. stall current — ток стопора | current when motor shaft is locked
113. no-load current — ток холостого хода | current with free-spinning motor
114. rated voltage — номинальное напряжение | intended operating voltage
115. rated current — номинальный ток | intended continuous current
116. peak current — пиковый ток | short-duration maximum current
117. continuous current — длительный ток | current sustainable over time
118. wire gauge — сечение провода | thickness of conductor (AWG/mm²)
119. voltage drop — падение напряжения | loss along wires under load
120. power budget — энергетический бюджет | planned allocation of watts to subsystems
121. idle power — потребление в покое | power used when not driving
122. active power — потребление в работе | power while navigating
123. efficiency — КПД | useful output over input energy
124. heat dissipation — теплоотвод | removing waste heat
125. thermal paste — термопаста | improves heat transfer to a sink
126. thermistor — термистор | temperature-dependent resistor
127. NTC — NTC | thermistor with negative temperature coefficient
128. RTC — часы реального времени | keeps time when powered appropriately
129. brownout detector — детектор просадки | hardware that reacts to low voltage
130. power-on reset — сброс при включении | clean MCU start when power appears
131. soft start — плавный пуск | ramps power/motor to reduce surge
132. kill switch — аварийный выключатель | manual emergency power/motion cut
133. e-stop — аварийный стоп | emergency stop circuit/button
134. interlock — блокировка | safety condition required before motion
135. status LED — светодиод статуса | shows system state to operators
136. heartbeat LED — светодиод «сердцебиения» | blinks to show firmware is alive
137. telemetry — телеметрия | reported measurements from the robot
138. firmware version — версия прошивки | identify which code is on the Mega
139. flashing — прошивка (запись) | writing firmware with avrdude/IDE
140. ISP — ISP | in-system programming interface
141. JTAG — JTAG | debug/programming interface
142. SWD — SWD | serial wire debug on many ARM MCUs
143. breakpoint — точка останова | debug pause in code
144. logic HIGH — логический «1» | asserted digital level
145. logic LOW — логический «0» | deasserted digital level
146. open-drain — открытый сток | output that can only pull low
147. push-pull — двухтактный выход | output that drives high and low
148. tri-state — третье состояние | high-impedance disconnected output
149. bus contention — конфликт шины | two drivers fighting on one line
150. clock stretching — растягивание такта | I2C slave holding SCL low

151. ACK/NACK — подтверждение/отказ | I2C acknowledge bits
152. CRC — CRC | checksum detecting transmission errors
153. parity bit — бит четности | simple error-detect bit
154. frame error — ошибка кадра | UART framing problem
155. buffer overflow — переполнение буфера | too much data for a buffer
156. DMA — DMA | direct memory access without CPU copy
157. interrupt — прерывание | CPU reaction to an event
158. ISR — обработчик прерывания | interrupt service routine
159. polling — опрос | checking a flag repeatedly
160. bit bang — бит-бэнг | software-emulated protocol timing
161. protocol — протокол | agreed rules for exchanging data
162. packet — пакет | structured unit of data
163. header — заголовок | metadata at start of a packet
164. payload (data) — полезная нагрузка данных | content bytes of a packet
165. endianness — порядок байтов | byte order in multi-byte values
166. little-endian — little-endian | least significant byte first
167. big-endian — big-endian | most significant byte first
168. signed integer — целое со знаком | integer that can be negative
169. unsigned integer — целое без знака | non-negative integer type
170. overflow — переполнение | value exceeds type range
171. underflow — антипереполнение | value goes below representable range
172. sampling rate — частота дискретизации | how often a signal is measured
173. Nyquist frequency — частота Найквиста | half the sampling rate
174. aliasing — алиасинг | false frequencies from undersampling
175. quantization — квантование | mapping continuous values to discrete levels
176. resolution — разрешение | smallest distinguishable change
177. accuracy — точность (правильность) | closeness to true value
178. precision — прецизионность | repeatability of measurements
179. drift — дрейф | slow unwanted change over time
180. offset — смещение | constant bias error
181. gain error — ошибка коэффициента | incorrect scale factor
182. linearity — линейность | how proportional output is to input
183. hysteresis — гистерезис | dependence on history of input
184. sensor fusion — слияние данных датчиков | combining multiple sensors for better estimate
185. IMU — ИМС / IMU | inertial measurement unit (accel+gyro)
186. accelerometer — акселерометр | measures proper acceleration
187. gyroscope — гироскоп | measures angular rate
188. magnetometer — магнитометр | measures magnetic field
189. barometer — барометр | measures air pressure / altitude
190. ToF sensor — датчик ToF | time-of-flight distance sensor
191. ultrasonic sensor — ультразвуковой датчик | distance via sound pulses
192. IR sensor — ИК-датчик | infrared proximity/line sensor
193. LiDAR — лидар | laser scanning distance sensor
194. laser diode — лазерный диод | light source inside many LiDARs
195. photodiode — фотодиод | light detector
196. beam divergence — расходимость луча | how a laser beam spreads
197. FOV — поле зрения | field of view of a sensor
198. angular resolution — угловое разрешение | smallest angle a scanner distinguishes
199. range — дальность | max useful sensing distance

- 200. minimum range — минимальная дальность | blind zone near the sensor
- 201. scan rate — частота сканирования | LiDAR revolutions/scans per second
- 202. point cloud — облако точек | set of 3D/2D measured points
- 203. intensity return — интенсивность отражения | strength of LiDAR echo
- 204. multipath — многолучевость | false readings from reflections
- 205. specular reflection — зеркальное отражение | mirror-like bounce that fools ranging
- 206. glass detection — обнаружение стекла | hard problem for many LiDARs
- 207. black material absorption — поглощение темными материалами | weak returns from dark surfaces
- 208. time-of-flight — время пролета | ranging principle using light travel time
- 209. triangulation — триангуляция | ranging using geometry of angles
- 210. structured light — структурированный свет | depth via projected patterns
- 211. stereo vision — стереозрение | depth from two cameras
- 212. exposure — экспозиция | camera light gathering setting
- 213. gain (camera) — усиление камеры | amplification of image signal
- 214. white balance — баланс белого | color correction for lighting
- 215. autofocus — автофокус | automatic focus adjustment
- 216. PTZ — PTZ | pan-tilt-zoom camera control (Тапо)
- 217. IR cut filter — ИК-фильтр | filter switching for day/night cameras
- 218. PoE injector — инжектор PoE | adds power onto Ethernet
- 219. USB hub — USB-хаб | expands USB ports on the Pi
- 220. powered hub — активный хаб | USB hub with external power
- 221. FTDI adapter — адаптер FTDI | USB-UART bridge chip/cable
- 222. CH340 — CH340 | common USB-UART chip
- 223. CP2102 — CP2102 | Silicon Labs USB-UART chip
- 224. avrdude — avrdude | tool used to flash AVR MCUs
- 225. Arduino IDE — Arduino IDE | environment for Mega firmware
- 226. PlatformIO — PlatformIO | alternative embedded toolchain
- 227. logic supply — питание логики | low-current rail for MCU/sensors
- 228. motor supply — питание моторов | high-current rail for actuators
- 229. star ground — звездная земля | ground wiring topology reducing loops
- 230. chassis ground — корпусная земля | metal frame used as ground reference
- 231. floating ground — плавающая земля | ground not tied to earth/chassis
- 232. earth ground — заземление | connection to protective earth
- 233. creepage — путь утечки | distance along surface between conductors
- 234. clearance (elec) — воздушный зазор | shortest air path between conductors
- 235. conformal coating — влагозащитное покрытие | protective film on PCBs
- 236. potting — заливка компаундом | encapsulating electronics in resin
- 237. hot plug — горячее подключение | connecting devices while powered
- 238. in-circuit test — внутрисхемный тест | electrical test of assembled board
- 239. continuity test — тест целостности | checks if a path conducts
- 240. short circuit — короткое замыкание | unintended low-resistance path
- 241. open circuit — обрыв цепи | broken intended path
- 242. load test — нагрузочный тест | test under realistic current draw
- 243. burn-in — термотренировка | extended run to catch early failures

1.3 Robotics (263 terms)

- 1. mobile robot — мобильный робот | robot that navigates through an environment
- 2. service robot — сервисный робот | robot assisting humans in non-industrial tasks
- 3. autonomous navigation — автономная навигация | moving to goals without continuous human steering

4. teleoperation — телеуправление | human remote control of the robot
5. semi-autonomous — полуавтономный | mix of autonomy and human oversight
6. localization — локализация | estimating where the robot is on the map
7. mapping — картографирование | building a map from sensor data
8. SLAM — SLAM | simultaneous localization and mapping
9. occupancy grid — сетка занятости | map cells marked free/occupied/unknown
10. log-odds — лог-odds | probabilistic update rule for occupancy cells
11. costmap — карта стоимости | grid encoding traversal cost for planning
12. path planning — планирование пути | computing a route from A to B
13. A* — A* | heuristic graph search used in AeroPorter
14. Dijkstra — Дейкстра | shortest-path algorithm without heuristic
15. heuristic — эвристика | estimate guiding search — Manhattan in our A*
16. pure pursuit — pure pursuit | path-following by aiming at a look-ahead point
17. look-ahead distance — дистанция упреждения | how far ahead pure pursuit targets
18. cross-track error — поперечная ошибка | lateral distance from the planned path
19. pose — поза | position plus orientation
20. odometry — одометрия | pose estimate from wheel motion
21. dead reckoning — счисление пути | integrating motion without absolute fixes
22. scan matching — сопоставление сканов | aligning LiDAR scans to refine pose
23. ICP — ICP | iterative closest point alignment
24. CSM — CSM | correlative scan matching
25. loop closure — замыкание цикла | recognizing a revisited place to fix drift
26. particle filter — частичный фильтр | probabilistic localization method (e.g. AMCL)
27. AMCL — AMCL | adaptive Monte Carlo localization — not our stack
28. Nav2 — Nav2 | ROS 2 navigation framework — we use a custom stack instead
29. ROS 2 — ROS 2 | Robot Operating System 2 middleware
30. rclpy — rclpy | Python client library for ROS 2
31. topic — топик | named ROS data stream
32. node — нода | ROS process participating in the graph
33. message — сообщение | typed ROS data structure
34. LaserScan — LaserScan | ROS message for 2D LiDAR scans
35. Odometry — Odometry | ROS message for pose and twist
36. Twist — Twist | linear and angular velocity command
37. cmd_vel — cmd_vel | standard velocity command topic
38. tf — tf | ROS transform tree between frames
39. frame_id — frame_id | coordinate frame name
40. map frame — система координат карты | fixed world frame of the occupancy map
41. base_link — base_link | robot body coordinate frame
42. footprint radius — радиус габарита | inflation radius used in planning (~0.5 m)
43. inflation — раздувание препятствий | expanding occupied cells for safety margin
44. temporary obstacle — временное препятствие | short-lived blocked cells from live hits
45. global planner — глобальный планировщик | computes overall A→B path
46. local planner — локальный планировщик | short-horizon tracking/avoidance
47. recovery behavior — поведение восстановления | actions when planning/tracking fails
48. goal tolerance — допуск цели | how close is “arrived”
49. heading — курс | yaw direction the robot faces
50. waypoint — путевая точка | intermediate target on a route
51. trajectory — траектория | time-parameterized path
52. control loop — контур управления | repeated sense→plan→act cycle

53. feedback control — управление с обратной связью | using measurements to correct commands
54. feedforward — управление по возмущению / упреждение | command based on model without waiting for error
55. PID controller — ПИД-регулятор | proportional-integral-derivative controller
56. PI controller — ПИ-регулятор | PI used on trusted wheel rates in firmware
57. setpoint — уставка | desired target value
58. steady-state error — установившаяся ошибка | remaining error after settling
59. overshoot — перерегулирование | going past the target before settling
60. settling time — время установления | time to enter and stay in a band
61. stability — устойчивость | system returns to equilibrium after disturbance
62. robustness — робастность | performance under uncertainty
63. holonomic drive — голономный привод | can translate/rotate independently — mecanum
64. differential drive — дифференциальный привод | non-holonomic two-wheel base
65. ackermann drive — рулевая схема Аккермана | car-like steering geometry
66. skid steer — бортовой поворот | turns by opposing track/wheel speeds
67. inverse kinematics (drive) — обратная кинематика привода | body velocity → wheel speeds
68. forward kinematics (drive) — прямая кинематика привода | wheel speeds → body velocity
69. velocity profile — профиль скорости | planned speed over time
70. acceleration limit — ограничение ускорения | caps how fast speed may change
71. jerk limit — ограничение рывка | limits acceleration change for smoothness
72. emergency stop — аварийная остановка | immediate halt of motion
73. fail-safe — отказобезопасный режим | defaults to a safe state on fault
74. watchdog — сторожевой механизм | stops motion if commands go stale
75. command timeout — таймаут команды | Arduino stops after ~1.2 s without commands
76. heartbeat lease — арендный heartbeat | kiosk must refresh lease or escort stops (~6 s)
77. human-robot interaction — человеко-машинное взаимодействие | how people and robots communicate
78. passenger escort — сопровождение пассажира | leading a person along a route
79. follow-me mode — режим «следуй за мной» | robot follows a person (related UX language)
80. social navigation — социальная навигация | moving politely among people
81. crowd density — плотность толпы | how crowded the environment is
82. dynamic obstacle — динамическое препятствие | moving person/object
83. static obstacle — статическое препятствие | fixed wall/furniture
84. free space — свободное пространство | traversable area on the map
85. unknown space — неизвестное пространство | unobserved map cells
86. frontier — граница исследования | edge between free and unknown
87. exploration — исследование | building map by visiting frontiers
88. remembered map — запомненная карта | saved occupancy map reused later
89. map update — обновление карты | changing cells from new scans
90. sensor model — модель датчика | how measurements relate to the world
91. motion model — модель движения | how controls change pose
92. process noise — шум процесса | uncertainty in motion prediction
93. measurement noise — шум измерений | uncertainty in sensor readings
94. covariance — ковариация | uncertainty structure of an estimate
95. Kalman filter — фильтр Калмана | optimal linear state estimator
96. EKF — EKF | extended Kalman filter for nonlinear models
97. UKF — UKF | unscented Kalman filter
98. Bayes filter — байесовский фильтр | general recursive probabilistic estimator
99. belief — belief / апостериорное | robot's probabilistic estimate of state
100. landmark — ориентир | recognizable feature for localization
101. feature extraction — извлечение признаков | detecting useful structures in sensor data

102. line extraction — выделение линий | finding walls as line features
103. corner feature — угловой признак | corner used as a landmark
104. scan deskewing — выпрямление скана | correcting scan distortion from motion
105. motion distortion — искажение из-за движения | smear in LiDAR while moving
106. ray casting — трассировка лучей | simulating rays through a grid
107. Bresenham — Брезенхем | line algorithm used in grid ray updates
108. cell resolution — разрешение ячейки | meters per occupancy cell (e.g. 5 cm)
109. map size — размер карты | extent of the occupancy grid (~40 m class)
110. origin — начало координат | map origin pose
111. quaternion — кватернион | orientation representation used in ROS
112. Euler angles — углы Эйлера | roll/pitch/yaw representation
113. rotation matrix — матрица поворота | 3×3 orientation matrix
114. homogeneous transform — однородное преобразование | 4×4 pose transform
115. SE(2) — SE(2) | 2D rigid poses: x, y, yaw
116. SE(3) — SE(3) | 3D rigid poses
117. twist covariance — ковариация скорости | uncertainty of velocity estimate
118. pose graph — граф поз | graph optimization structure for SLAM
119. bundle adjustment — уравнивание связей | joint optimization of poses/features
120. gnss — ГНСС | satellite positioning — not used indoors
121. uwb — UWB | ultra-wideband ranging option
122. beacon — маяк | fixed reference transmitter
123. docking — стыковка / докинг | precision approach to a charger/station
124. homing — возврат на базу | returning to a home pose
125. patrol — патрулирование | repeated route following
126. mission — миссия | high-level task assignment
127. behavior tree — дерево поведения | modular robot behavior architecture
128. finite state machine — конечный автомат | states and transitions for modes
129. task planner — планировщик задач | sequences high-level actions
130. middleware — промежуточное ПО | software layer connecting components — ROS 2
131. real-time — реальное время | meeting timing deadlines
132. soft real-time — мягкое реальное время | timing preferred but not hard-certified
133. latency — задержка | time from sense to act
134. jitter — джиттер | variation in loop timing
135. control frequency — частота управления | Hz of the control loop
136. sample time — период дискретизации | Δt of the discrete controller
137. deadband — мертвая зона | small errors intentionally ignored
138. saturation — насыщение | actuator hitting max command
139. anti-windup — антиwindup | prevents integrator explosion when saturated
140. gain tuning — настройка коэффициентов | choosing P/I/D or pursuit gains
141. system identification — идентификация системы | estimating model parameters from data
142. open-loop — разомкнутый контур | commands without feedback correction
143. closed-loop — замкнутый контур | commands corrected by feedback
144. reference trajectory — опорная траектория | desired path/time profile
145. tracking error — ошибка слежения | difference from reference
146. collision checking — проверка столкновений | testing if a motion intersects obstacles
147. configuration space — конфигурационное пространство | space of robot configurations
148. workspace (robot) — рабочее пространство | reachable positions of the robot
149. manipulator — манипулятор | robot arm (not AeroPorter's focus)
150. end effector — рабочий орган | tool at arm tip

- 151. gripper — захват | device for grasping objects
- 152. mobile base — мобильная платформа | driving chassis under sensors/UI
- 153. UGV — HPTK / UGV | unmanned ground vehicle
- 154. AGV — AGV | automated guided vehicle — often fixed paths
- 155. AMR — AMR | autonomous mobile robot — freer navigation
- 156. fleet management — управление флотом | coordinating multiple robots
- 157. traffic control — управление трафиком роботов | preventing multi-robot conflicts
- 158. formation control — управление строем | keeping relative geometry between robots
- 159. swarm robotics — роевая робототехника | many simple robots coordinating
- 160. bio-inspired — биоинспирированный | design ideas from biology
- 161. compliance control — управление податливостью | controlling interaction forces softly
- 162. impedance control — импедансное управление | sets dynamic relationship force↔motion
- 163. force control — силовое управление | regulating contact force
- 164. vision-based control — зрение в контуре управления | using cameras in the feedback loop
- 165. visual servoing — визуальный сервопривод | control using image features
- 166. teach pendant — пульт обучения | handheld device for industrial robots
- 167. operator console — пульт оператора | web UI at :8765 for AeroPorter
- 168. kiosk mode — режим киоска | locked passenger-facing UI
- 169. human-in-the-loop — человек в контуре | human remains part of decisions
- 170. supervisory control — супервизорное управление | human sets goals, robot executes
- 171. shared autonomy — разделенная автономия | blending human and robot control
- 172. assistive robotics — ассистивная робототехника | robots helping people with tasks
- 173. accessibility — доступность | usable by elderly / diverse passengers
- 174. HRI safety — безопасность ЧМИ | keeping people safe near robots
- 175. ISO 13482 — ISO 13482 | personal care robot safety standard (reference)
- 176. ISO 3691-4 — ISO 3691-4 | driverless industrial truck standard (reference)
- 177. risk assessment — оценка рисков | structured hazard analysis
- 178. hazard — опасность | potential source of harm
- 179. mitigation — митигация | action reducing risk
- 180. residual risk — остаточный риск | risk left after mitigations
- 181. safe speed — безопасная скорость | speed appropriate near people
- 182. protective separation — защитное разделение | distance kept from humans
- 183. speed and separation monitoring — контроль скорости и дистанции | collaborative safety mode concept
- 184. power and force limiting — ограничение силы и мощности | collaborative robot safety concept
- 185. soft cover — мягкая обшивка | padding reducing injury severity
- 186. status indication — индикация состояния | lights/sounds showing robot mode
- 187. audible warning — звуковое предупреждение | sound alert near motion
- 188. beacon light — проблесковый маяк | visual attention signal
- 189. manual override — ручной перехват | operator takes control immediately
- 190. remote stop — удаленный стоп | stop command over network
- 191. geofence — геозона | virtual boundary for allowed motion
- 192. keep-out zone — запретная зона | area marked non-traversable
- 193. preferred lane — предпочтительная полоса | preferred path corridor
- 194. semantic map — семантическая карта | map with labeled places — check-in, gates...
- 195. topological map — топологическая карта | graph of places and connections
- 196. metric map — метрическая карта | map with real-world distances
- 197. hybrid map — гибридная карта | mix of metric and topological info
- 198. calibration — калибровка | adjusting parameters to match reality
- 199. extrinsic calibration — внешняя калибровка | pose between sensors/frames

200. intrinsic calibration — внутренняя калибровка | internal sensor parameters
201. hand-eye calibration — калибровка «рука-глаз» | camera↔robot transform calibration
202. ground truth — эталон | trusted reference measurement
203. benchmark — бенчмарк | standard test for comparison
204. ablation study — абляция | testing by removing components
205. simulation — моделирование | testing in a virtual world
206. digital twin — цифровой двойник | virtual counterpart of the system
207. hardware-in-the-loop — HIL | real hardware in a simulated loop
208. field test — полевые испытания | test in realistic environment
209. pilot deployment — пилотное внедрение | limited real-world trial
210. MVP — MVP | minimum viable product
211. technology readiness level — УГТ / TRL | scale of maturity from idea to ops
212. requirement — требование | needed capability or constraint
213. specification — спецификация | detailed technical statement of needs
214. verification — верификация | checking we built it right
215. validation — валидация | checking we built the right thing
216. acceptance criteria — критерии приемки | conditions to accept a deliverable
217. regression — регрессия | old capability breaking after change
218. reproducibility — воспроизводимость | same result under same conditions
219. observability — наблюдаемость | ability to understand system state from logs
220. telemetry dashboard — панель телеметрии | UI of live robot metrics
221. black box log — черный ящик | recorded data for post-incident analysis
222. incident — инцидент | undesired event needing review
223. root cause — корневая причина | underlying reason for a failure
224. corrective action — корректирующее действие | fix that prevents recurrence
225. redundancy — резервирование | extra elements for fault tolerance
226. graceful degradation — мягкая деградация | reduced function instead of total failure
227. fallback mode — резервный режим | alternate safer behavior
228. manual teleop fallback — резервное телеуправление | operator drives if autonomy fails
229. sensor occlusion — затенение датчика | sensor blocked by object/person
230. blind spot — мертвая зона | unobserved region around the robot
231. self-filter — самофильтрация | ignoring robot body in LiDAR returns
232. frame rack filter — фильтр стоек рамы | ignore chassis posts in scans
233. yaw hold — удержание курса | using LiDAR yaw to stabilize heading
234. trim — триммирование | small correction offsets in drive calibration
235. drive_cal — drive_cal | calibration file for floor-dependent drive params
236. stale command — устаревшая команда | command older than watchdog threshold
237. velocity buffer — буфер скорости | staging of recent velocity commands
238. lease — аренда сессии | time-limited right to control escort
239. fail closed — отказ «в закрытое» | deny action if config/safety invalid
240. fail open — отказ «в открытое» | allow action when unsure — usually unsafe
241. deterministic behavior — детерминированное поведение | same inputs → same outputs
242. nondeterministic — недетерминированный | timing/races may change outcomes
243. race condition — состояние гонки | bug from unsynchronized concurrent events
244. mutex — мьютекс | lock protecting shared state
245. thread safety — потокобезопасность | correct under concurrent access
246. message queue — очередь сообщений | buffered async communication
247. publish/subscribe — публикация/подписка | ROS-style decoupled messaging
248. QoS — QoS | quality of service settings for ROS 2

- 249. DDS — DDS | middleware under ROS 2 communications
- 250. discovery — обнаружение узлов | how ROS 2 nodes find each other
- 251. lifecycle node — lifecycle-нода | ROS 2 node with managed states
- 252. parameter server — параметры ROS | runtime configuration values
- 253. launch file — launch-файл | starts a set of ROS nodes
- 254. bag file — rosbag | recorded ROS traffic for replay
- 255. replay — воспроизведение | playing recorded data into the stack
- 256. unit test — модульный тест | tests one function/module
- 257. integration test — интеграционный тест | tests combined modules
- 258. smoke test — дымовой тест | quick check that basics run
- 259. soak test — длительный тест | long run for leaks/overheat
- 260. corner case — крайевой случай | rare but important scenario
- 261. edge case — граничный случай | extreme input/condition
- 262. worst case — наихудший случай | most demanding scenario for design
- 263. nominal case — номинальный случай | expected normal operating scenario

1.4 Programming (343 terms)

- 1. algorithm — алгоритм | step-by-step procedure solving a problem
- 2. data structure — структура данных | way to organize data for efficient use
- 3. array — массив | contiguous indexed collection
- 4. list — список | ordered collection
- 5. dictionary — словарь | key→value map (Python dict)
- 6. hash map — хеш-таблица | average O(1) key lookup structure
- 7. set — множество | collection of unique elements
- 8. queue — очередь | FIFO structure
- 9. stack — стек | LIFO structure
- 10. priority queue — очередь с приоритетом | pops best-priority item first
- 11. heap — куча | tree structure for priority queues
- 12. graph — граф | nodes connected by edges — used in planning
- 13. tree — дерево | acyclic connected graph
- 14. binary search — бинарный поиск | log-time search in sorted data
- 15. complexity — сложность | resource growth vs input size
- 16. Big O — O-большое | asymptotic upper bound notation
- 17. recursion — рекурсия | function calling itself
- 18. iteration — итерация | looping through steps
- 19. API — API | interface for software interaction
- 20. endpoint — эндпоинт | HTTP URL handling a request
- 21. REST — REST | style of HTTP APIs
- 22. HTTP — HTTP | web request protocol used on :8765
- 23. JSON — JSON | text data format for configs/APIs
- 24. SQLite — SQLite | embedded SQL database for tickets
- 25. SQL query — SQL-запрос | statement reading/writing tables
- 26. schema — схема БД | structure of tables/columns
- 27. primary key — первичный ключ | unique row identifier — ticket code
- 28. foreign key — внешний ключ | reference to another table
- 29. transaction — транзакция | atomic DB operation group
- 30. CRUD — CRUD | create/read/update/delete
- 31. serialization — сериализация | object → bytes/text
- 32. deserialization — десериализация | bytes/text → object

33. parsing — разбор | interpreting structured text
34. validation — проверка данных | rejecting invalid inputs
35. sanitization — санитизация | cleaning inputs to reduce risk
36. exception — исключение | error object disrupting normal flow
37. try/except — try/except | Python error handling block
38. logging — логирование | recording events for debugging
39. debug — отладка | finding and fixing defects
40. breakpoint — точка останова | pause execution while debugging
41. unit test — модульный тест | test of a small unit
42. mock — мок | fake dependency for tests
43. fixture — фикстура | prepared test context/data
44. CI — CI | continuous integration
45. CD — CD | continuous delivery/deployment
46. version control — контроль версий | tracking code history — Git
47. commit — коммит | saved snapshot in Git
48. branch — ветка | independent line of development
49. merge — слияние | combining branches
50. pull request — pull request | proposal to merge changes
51. code review — ревью кода | peer inspection of changes
52. refactoring — рефакторинг | improving structure without changing behavior
53. technical debt — технический долг | shortcuts that cost later
54. modularity — модульность | splitting into independent parts
55. coupling — связанность | how dependent modules are
56. cohesion — связность | how focused a module is
57. abstraction — абстракция | hiding detail behind an interface
58. encapsulation — инкапсуляция | bundling data with methods
59. inheritance — наследование | class deriving from another
60. polymorphism — полиморфизм | same interface, different behavior
61. interface — интерфейс | contract of available operations
62. class — класс | blueprint for objects
63. object — объект | instance of a class
64. method — метод | function belonging to an object
65. function — функция | reusable block of code
66. parameter — параметр | input name in a function definition
67. argument — аргумент | actual value passed in
68. return value — возвращаемое значение | output of a function
69. side effect — побочный эффект | change beyond the return value
70. pure function — чистая функция | no side effects; same in→same out
71. immutable — неизменяемый | cannot be changed after creation
72. mutable — изменяемый | can be changed in place
73. scope — область видимости | where a name is accessible
74. closure — замыкание | function capturing outer variables
75. callback — колбэк | function passed to be called later
76. async — асинхронный | non-blocking concurrent style
77. await — await | wait for an async result
78. thread — поток | concurrent execution path
79. process — процесс | OS process with own memory
80. concurrency — параллелизм по времени | overlapping execution
81. parallelism — параллелизм | true simultaneous execution

- 82. race condition — состояние гонки | bug from unsafe concurrent access
- 83. deadlock — взаимная блокировка | processes waiting on each other forever
- 84. mutex — мьютекс | mutual exclusion lock
- 85. semaphore — семафор | counter-based synchronization primitive
- 86. event loop — цикл событий | core of many async servers
- 87. websocket — WebSocket | full-duplex web socket protocol
- 88. polling (net) — поллинг | periodically requesting updates
- 89. webhook — webhook | HTTP callback on events
- 90. middleware (web) — промежуточный слой | request/response processing layer
- 91. router — маршрутизатор URL | maps paths to handlers
- 92. handler — обработчик | function handling a request
- 93. request — запрос | client→server message
- 94. response — ответ | server→client message
- 95. status code — код статуса | HTTP result code — 200, 403, 404...
- 96. header (HTTP) — заголовок HTTP | metadata in HTTP messages
- 97. cookie — cookie | small stored client state
- 98. session — сессия | server-side user/interaction state
- 99. authentication — аутентификация | verifying identity
- 100. authorization — авторизация | checking permission
- 101. allowlist — белый список | explicitly permitted IPs/clients
- 102. CORS — CORS | cross-origin resource sharing rules
- 103. Origin header — заголовок Origin | browser origin for CORS checks
- 104. HTTPS — HTTPS | HTTP over TLS
- 105. TLS — TLS | transport encryption protocol
- 106. certificate — сертификат | cryptographic identity document
- 107. hash — хеш | one-way digest of data
- 108. checksum — контрольная сумма | integrity check value
- 109. encryption — шифрование | encoding secrecy of data
- 110. decryption — расшифрование | reversing encryption
- 111. secret — секрет | credential that must stay private
- 112. environment variable — переменная окружения | config injected via env
- 113. config file — конфиг | file storing settings — JSON on AeroPorter
- 114. hardcoding — хардкод | embedding values directly in code
- 115. magic number — магическое число | unexplained literal in code
- 116. constant — константа | named fixed value
- 117. enum — перечисление | named set of allowed values
- 118. typedef — псевдоним типа | alternate name for a type
- 119. static typing — статическая типизация | types checked before runtime
- 120. dynamic typing — динамическая типизация | types checked at runtime — Python
- 121. type hint — аннотация типа | optional Python type annotation
- 122. linter — линтер | static style/bug checker
- 123. formatter — форматтер | auto code style tool
- 124. package — пакет | distributable library module
- 125. dependency — зависимость | external package required
- 126. virtual environment — виртуальное окружение | isolated Python package set
- 127. pip — pip | Python package installer
- 128. requirements.txt — requirements.txt | pinned dependency list
- 129. import — импорт | bring a module into scope
- 130. namespace — пространство имен | container for names

- 131. module — модуль | single Python file as a unit
- 132. package init — `__init__.py` | marks a directory as a package
- 133. script — скрипт | program run from top to bottom
- 134. shebang — шебанг | `#!` line selecting interpreter
- 135. CLI — CLI | command-line interface
- 136. argv — argv | command-line argument vector
- 137. stdin/stdout — stdin/stdout | standard input/output streams
- 138. stderr — stderr | standard error stream
- 139. path — путь | filesystem location
- 140. absolute path — абсолютный путь | path from filesystem root
- 141. relative path — относительный путь | path from current directory
- 142. file I/O — файловый ввод-вывод | reading/writing files
- 143. binary mode — двоичный режим | file I/O of raw bytes
- 144. text mode — текстовый режим | file I/O of decoded strings
- 145. encoding — кодировка | character encoding — UTF-8
- 146. UTF-8 — UTF-8 | dominant Unicode encoding
- 147. regex — регулярное выражение | pattern matching language
- 148. state machine — конечный автомат | program logic as states/transitions
- 149. event-driven — событийно-ориентированный | reacts to events
- 150. callback hell — ад колбэков | deeply nested callbacks
- 151. promise — promise | JS future value object
- 152. DOM — DOM | browser document object model
- 153. BarcodeDetector — BarcodeDetector | browser API for barcode scanning
- 154. Canvas API — Canvas API | browser drawing API
- 155. localStorage — localStorage | browser persistent key-value store
- 156. fetch — fetch | browser HTTP request API
- 157. JSON.parse — JSON.parse | parse JSON string to object
- 158. frontend — фронтенд | client-side UI code
- 159. backend — бэкэнд | server-side logic — `lidar_map` HTTP
- 160. full stack — полный стек | frontend + backend together
- 161. monorepo — монорепозиторий | multiple projects in one repo
- 162. microservice — микросервис | small independently deployable service
- 163. monolith — монолит | single deployable application
- 164. IPC — IPC | inter-process communication
- 165. socket — сокет | endpoint for network communication
- 166. TCP — TCP | reliable stream transport
- 167. UDP — UDP | datagram transport
- 168. port — порт | numeric transport endpoint — 8765
- 169. bind — bind | attach socket to address/port
- 170. listen — listen | wait for incoming connections
- 171. client — клиент | initiator of a request
- 172. server — сервер | accepts and handles requests
- 173. daemon — демон | background service process
- 174. systemd — systemd | Linux service manager
- 175. cron — cron | time-based job scheduler
- 176. shell script — shell-скрипт | bash automation — `start_*.sh`
- 177. environment — окружение | runtime context of a program
- 178. cross-platform — кроссплатформенный | runs on multiple OSes
- 179. portability — переносимость | ease of moving across platforms

- 180. backward compatibility — обратная совместимость | new version works with old clients
- 181. breaking change — ломающее изменение | change that breaks old clients
- 182. semantic versioning — семантическое версионирование | MAJOR.MINOR.PATCH rules
- 183. changelog — changelog | human-readable change history
- 184. documentation — документация | written explanation of the system
- 185. README — README | entry-point project documentation
- 186. comment — комментарий | non-executed explanation in code
- 187. docstring — docstring | Python documentation string
- 188. TODO (avoid in delivery) — TODO | marker of unfinished work — avoid on defense as if done
- 189. dead code — мертвый код | unreachable/unused code
- 190. code smell — запах кода | symptom of deeper design problems
- 191. design pattern — шаблон проектирования | reusable design solution
- 192. singleton — одиночка | one-instance pattern
- 193. factory — фабрика | creates objects without exposing class
- 194. observer — наблюдатель | publish-subscribe design pattern
- 195. strategy pattern — стратегия | swappable algorithms behind an interface
- 196. dependency injection — внедрение зависимостей | passing dependencies in from outside
- 197. inversion of control — инверсия управления | framework calls your code
- 198. DRY — DRY | Don't Repeat Yourself
- 199. KISS — KISS | Keep It Simple, Stupid
- 200. YAGNI — YAGNI | You Aren't Gonna Need It
- 201. separation of concerns — разделение ответственности | each part has one job
- 202. single responsibility — единственная ответственность | class/module does one thing
- 203. open-closed principle — открыт-закрыт | open to extension, closed to modification
- 204. idempotent — идемпотентный | repeating has same effect as once
- 205. deterministic build — детерминированная сборка | same source → same artifact
- 206. reproducible run — воспроизводимый запуск | same inputs → same behavior
- 207. seed — сид | initial value for random generators
- 208. pseudorandom — псевдослучайный | deterministic 'random' sequence
- 209. floating point — число с плавающей точкой | approximate real-number type
- 210. integer overflow — переполнение целого | wrap/error when exceeding int range
- 211. NaN — NaN | not-a-number floating value
- 212. infinity — бесконечность | IEEE floating overflow result
- 213. epsilon — эпсилон | small tolerance for float compares
- 214. rounding error — ошибка округления | error from finite precision
- 215. numerical stability — численная устойчивость | algorithm resists precision issues
- 216. interpolation — интерполяция | estimating values between samples
- 217. extrapolation — экстраполяция | estimating beyond sampled range
- 218. filtering — фильтрация | removing noise or selecting data
- 219. moving average — скользящее среднее | simple smoothing filter
- 220. low-pass filter — ФНЧ | passes slow changes, attenuates fast
- 221. high-pass filter — ФВЧ | passes fast changes
- 222. thresholding — пороговая обработка | decisions by comparing to a threshold
- 223. hysteresis threshold — порог с гистерезисом | different on/off thresholds
- 224. state — состояние | current mode/values of the system
- 225. stateless — без состояния | handler doesn't keep client memory
- 226. cache — кэш | fast temporary storage
- 227. cache invalidation — инвалидация кэша | dropping stale cached data
- 228. memoization — мемоизация | caching function results

- 229. lazy evaluation — ленивые вычисления | compute only when needed
- 230. eager evaluation — энергичные вычисления | compute immediately
- 231. garbage collection — сборка мусора | automatic memory reclamation
- 232. memory leak — утечка памяти | memory not released when unused
- 233. buffer — буфер | temporary memory for data in transit
- 234. ring buffer — кольцевой буфер | fixed-size circular buffer
- 235. producer-consumer — производитель-потребитель | classic concurrent pattern
- 236. backpressure — обратное давление | slowing producers when consumers lag
- 237. rate limit — ограничение частоты | cap on request/event rate
- 238. retry — повтор | try again after failure
- 239. backoff — отступ по времени | increasing delay between retries
- 240. circuit breaker — автовыключатель (ПО) | stops calling a failing dependency
- 241. timeout — таймаут | max wait before aborting
- 242. deadline — дедлайн | absolute time by which work must finish
- 243. graceful shutdown — корректное завершение | clean stop closing resources
- 244. signal handler — обработчик сигнала | reacts to OS signals
- 245. SIGINT — SIGINT | interrupt signal — Ctrl+C
- 246. SIGTERM — SIGTERM | termination request signal
- 247. zombie process — зомби-процесс | finished process not yet reaped
- 248. orphan process — процесс-сирота | process whose parent exited
- 249. PID — PID процесса | process identifier
- 250. port conflict — конфликт портов | two services binding same port
- 251. health check — проверка здоровья | endpoint/process liveness check
- 252. readiness probe — probe готовности | checks if ready for traffic
- 253. liveness probe — probe жизнеспособности | checks if process should restart
- 254. metrics — метрики | numeric runtime measurements
- 255. tracing — трассировка | following a request across services
- 256. span — span | unit of work in a trace
- 257. structured logging — структурированные логи | logs as key-value/JSON fields
- 258. log level — уровень лога | DEBUG/INFO/WARN/ERROR
- 259. stack trace — стек вызовов | call chain at an exception
- 260. core dump — дамп памяти | snapshot for post-crash debug
- 261. profiler — профайлер | tool measuring time/memory hotspots
- 262. benchmark (sw) — бенчмарк ПО | performance measurement harness
- 263. optimization — оптимизация | improving speed/memory/energy
- 264. premature optimization — преждевременная оптимизация | optimizing too early — often harmful
- 265. bottleneck — узкое место | slowest limiting component
- 266. throughput — пропускная способность | work completed per unit time
- 267. latency budget — бюджет задержки | allowed end-to-end delay
- 268. bandwidth — полоса пропускания | data rate capacity
- 269. compression — сжатие | reducing data size
- 270. serialization format — формат сериализации | JSON, MessagePack, protobuf...
- 271. protobuf — protobuf | compact binary schema format
- 272. YAML — YAML | indentation-based config language
- 273. TOML — TOML | minimal config language
- 274. XML — XML | markup data format
- 275. CSV — CSV | comma-separated values
- 276. binary protocol — бинарный протокол | non-text packet layout — Mega serial
- 277. framing — кадрирование | finding packet boundaries in a stream

- 278. escape sequence — escape-последовательность | special encoded byte sequence
- 279. endian swap — смена порядка байтов | converting byte order
- 280. bitmask — битмаска | bits selecting fields/flags
- 281. bitwise operation — побитовая операция | AND/OR/XOR/shifts on bits
- 282. flag — флаг | boolean status indicator
- 283. enum class — enum class | typed enumeration
- 284. null — null/None | absent value
- 285. optional — optional | value that may be missing
- 286. Result type — тип Result | success-or-error return style
- 287. assert — assert | runtime check that must hold
- 288. invariant — инвариант | condition that must always be true
- 289. precondition — предусловие | must hold before a call
- 290. postcondition — постусловие | must hold after a call
- 291. contract — контракт | pre/post/invariant agreement
- 292. defensive programming — защитное программирование | coding assuming bad inputs/failures
- 293. offensive programming — атакующее программирование | fail fast on violated contracts
- 294. fail fast — быстрый отказ | crash/stop early on bad state
- 295. feature flag — feature flag | runtime switch enabling features
- 296. canary release — canary-релиз | gradual rollout to a small group
- 297. rollback — откат | revert to previous working version
- 298. blue-green deployment — blue-green | two environments swap for release
- 299. container — контейнер | isolated runnable filesystem+process
- 300. Docker — Docker | popular container tooling
- 301. image — образ | immutable container template
- 302. orchestrator — оркестратор | manages many containers/services
- 303. infrastructure as code — инфраструктура как код | infra defined in files
- 304. Makefile — Makefile | classic build automation file
- 305. build system — система сборки | compiles/packages software
- 306. compiler — компилятор | translates source to machine code
- 307. interpreter — интерпретатор | executes source/bytecode directly
- 308. bytecode — байткод | portable intermediate instructions
- 309. JIT — JIT | just-in-time compilation
- 310. garbage collector — сборщик мусора | runtime reclaiming unused objects
- 311. reference counting — подсчет ссылок | GC strategy via counts
- 312. weak reference — слабая ссылка | reference that doesn't keep object alive
- 313. deep copy — глубокое копирование | recursive copy of nested objects
- 314. shallow copy — поверхностное копирование | copy only top-level references
- 315. aliasing (memory) — алиасинг памяти | two names referring to same object
- 316. pointer — указатель | address of a memory location
- 317. reference — ссылка | alias to an existing object
- 318. dangling pointer — висячий указатель | pointer to freed memory
- 319. segmentation fault — segfault | invalid memory access crash
- 320. undefined behavior — неопределенное поведение | C/C++ behavior not defined by the standard
- 321. memory safety — безопасность памяти | preventing invalid accesses
- 322. bounds check — проверка границ | ensuring index is in range
- 323. off-by-one — ошибка на единицу | classic loop/index mistake
- 324. null pointer dereference — разыменование null | using a null pointer — crash
- 325. use after free — использование после освобождения | accessing freed memory
- 326. double free — двойное освобождение | freeing the same block twice

- 327. stack overflow — переполнение стека | too deep recursion / huge stack frame
- 328. heap overflow — переполнение кучи | writing past allocated heap block
- 329. buffer overrun — выход за буфер | writing outside buffer bounds
- 330. injection — инъекция | attacker-controlled data executed as code/query
- 331. SQL injection — SQL-инъекция | malicious SQL via inputs
- 332. XSS — XSS | cross-site scripting in browsers
- 333. CSRF — CSRF | cross-site request forgery
- 334. path traversal — path traversal | escaping intended directories via ../
- 335. input validation — валидация ввода | checking inputs before use
- 336. least privilege — минимальные привилегии | grant only needed permissions
- 337. defense in depth — эшелонированная защита | multiple independent security layers
- 338. threat model — модель угроз | structured view of who can attack what
- 339. attack surface — поверхность атаки | exposed ways to attack the system
- 340. sandbox — песочница | restricted execution environment
- 341. permission — право доступа | allowed action on a resource
- 342. role — роль | named set of permissions
- 343. audit log — журнал аудита | record of security-relevant actions

1.5 Computer Vision (129 terms)

- 1. computer vision — компьютерное зрение | extracting meaning from images/video
- 2. image — изображение | 2D array of pixels
- 3. frame — кадр | single image from a video stream
- 4. pixel — пиксель | smallest image element
- 5. resolution — разрешение | width×height in pixels
- 6. FPS — кадры в секунду | frames processed/displayed per second
- 7. RGB — RGB | red-green-blue color model
- 8. grayscale — оттенки серого | single-channel intensity image
- 9. HSV — HSV | hue-saturation-value color space
- 10. OpenCV — OpenCV | popular CV library used in camera/
- 11. YuNet — YuNet | face detection model used on USB camera
- 12. ONNX — ONNX | portable neural net format
- 13. Haar cascade — каскад Хаара | classical detector used in Tapo prototype
- 14. bounding box — ограничивающий прямоугольник | rectangle around a detection
- 15. confidence score — оценка уверенности | model's score for a detection
- 16. threshold — порог | cutoff for accepting detections
- 17. non-maximum suppression — NMS | suppress overlapping duplicate boxes
- 18. face detection — детекция лица | finding faces in a frame
- 19. face tracking — трекинг лица | keeping a face centered over time
- 20. digital PTZ — цифровой PTZ | crop/zoom in software without motors
- 21. optical PTZ — оптический/моторный PTZ | physical pan-tilt-zoom — Tapo
- 22. ROI — область интереса | region of interest inside the image
- 23. crop — кадрирование | cutting a sub-rectangle from an image
- 24. resize — масштабирование | changing image dimensions
- 25. letterbox — letterbox | resize preserving aspect with pads
- 26. normalization — нормализация | scaling pixel values for a model
- 27. augmentation — аугментация | synthetic training variations
- 28. annotation — разметка | labeling images for training
- 29. ground-truth label — эталонная метка | correct label for evaluation
- 30. precision (CV) — точность (precision) | $TP / (TP + FP)$

31. recall — полнота | $TP / (TP + FN)$
32. F1 score — F1-мера | harmonic mean of precision and recall
33. mAP — mAP | mean average precision for detection
34. false positive — ложное срабатывание | detected something that isn't there
35. false negative — пропуск | missed a real object
36. true positive — верное обнаружение | correct detection
37. IoU — IoU | intersection over union of boxes
38. landmark (face) — лицевые ключевые точки | eyes/nose/mouth points
39. pose estimation — оценка позы | estimating body/head orientation
40. optical flow — оптический поток | motion of pixels between frames
41. background subtraction — вычитание фона | separating moving foreground
42. edge detection — выделение границ | finding intensity edges — Canny etc.
43. Canny — Canny | classic edge detector
44. contour — контур | curve of boundary points
45. morphology — морфология | erosion/dilation operations
46. erosion — эрозия | shrinks foreground regions
47. dilation — дилатация | expands foreground regions
48. Gaussian blur — размытие по Гауссу | smoothing with Gaussian kernel
49. median filter — медианный фильтр | noise reduction preserving edges
50. histogram — гистограмма | distribution of pixel intensities
51. histogram equalization — эквализация гистограммы | contrast enhancement method
52. exposure control — управление экспозицией | camera brightness adaptation
53. motion blur — смаз движения | blur from camera/object motion
54. rolling shutter — rolling shutter | line-by-line capture distortion
55. global shutter — global shutter | whole-frame simultaneous capture
56. distortion — дисторсия | lens geometric warping
57. undistort — выпрямление | removing lens distortion
58. camera matrix — матрица камеры | intrinsic camera parameters
59. extrinsics — внешние параметры | camera pose in the world
60. intrinsics — внутренние параметры | focal length, principal point...
61. calibration pattern — калибровочный паттерн | chessboard/circles for calibration
62. reprojection error — ошибка репроекции | calibration quality metric
63. stereo pair — стереопара | two cameras for depth
64. disparity — диспаратность | pixel shift between stereo views
65. depth map — карта глубины | per-pixel distance image
66. point cloud (vision) — облако точек | 3D points from depth/vision
67. segmentation — сегментация | labeling pixels by class/instance
68. semantic segmentation — семантическая сегментация | class per pixel
69. instance segmentation — инстанс-сегментация | separate object instances
70. classification — классификация | assigning a label to an image
71. detector — детектор | model finding objects
72. tracker — трекер | model associating detections over time
73. Kalman tracker — трекер Калмана | tracking with Kalman predictions
74. SORT — SORT | simple online realtime tracker
75. DeepSORT — DeepSORT | SORT with appearance features
76. re-identification — реидентификация | recognizing the same person again
77. embedding — эмбединг | vector representation of an image/face
78. cosine similarity — косинусное сходство | similarity of embeddings
79. barcode — штрихкод | 1D machine-readable code on tickets

- 80. QR code — QR-код | 2D matrix code
- 81. BarcodeDetector API — BarcodeDetector API | browser barcode reader used in kiosk
- 82. boarding pass — посадочный талон | document/code with flight data
- 83. OCR — OCR | optical character recognition
- 84. OCR engine — OCR-движок | software converting text images to strings
- 85. template matching — шаблонное сопоставление | finding a template patch in an image
- 86. feature descriptor — дескриптор признака | vector describing a local feature
- 87. SIFT — SIFT | classic scale-invariant feature
- 88. ORB — ORB | fast binary feature detector
- 89. homography — гомография | projective transform between planes
- 90. perspective transform — перспективное преобразование | warping a quadrilateral region
- 91. image pyramid — пирамида изображений | multi-scale image representation
- 92. sliding window — скользящее окно | classical detection scan pattern
- 93. anchor box — якорный бокс | prior box shapes in detectors
- 94. backbone — backbone | feature-extracting CNN trunk
- 95. inference — инференс | running a trained model on new data
- 96. training — обучение | fitting model parameters on data
- 97. overfitting — переобучение | memorizing train set, poor generalization
- 98. underfitting — недообучение | model too weak to capture patterns
- 99. dataset — датасет | collection of labeled examples
- 100. batch size — размер батча | examples per training step
- 101. learning rate — скорость обучения | step size for optimizer
- 102. GPU — GPU | accelerator for neural nets — Jetson has one; Pi 5 is limited
- 103. CPU inference — инференс на CPU | running models on CPU — our Pi path
- 104. quantization (NN) — квантизация сети | reducing weight precision for speed
- 105. pruning — прунинг | removing unnecessary weights
- 106. ONNX Runtime — ONNX Runtime | engine to run ONNX models
- 107. preprocessing — предобработка | steps before model input
- 108. postprocessing — постобработка | steps after model output
- 109. latency (vision) — задержка зрения | time from frame grab to decision
- 110. pipeline — конвейер | chained processing stages
- 111. camera index — индекс камеры | OS index of USB camera
- 112. V4L2 — V4L2 | Linux video capture API
- 113. RTSP URL — RTSP URL | network stream address for Tapo
- 114. MJPEG — MJPEG | motion JPEG stream format
- 115. H.264 — H.264 | common compressed video codec
- 116. bitrate — битрейт | bits per second in a stream
- 117. keyframe — ключевой кадр | full frame in compressed video
- 118. drop frame — пропуск кадра | skipping frames under load
- 119. thread (capture) — поток захвата | background thread reading frames
- 120. frame buffer — буфер кадров | storage for latest frames
- 121. tearing — разрыв кадра | visual artifact from partial updates
- 122. color temperature — цветовая температура | warm/cool lighting character
- 123. glare — блик | bright reflection washing out details
- 124. low light — низкая освещенность | dim conditions hurting detection
- 125. backlight — контровой свет | subject darker than background
- 126. occlusion (vision) — окклюзия | object blocked in the camera view
- 127. privacy — конфиденциальность | protecting passenger identity/data
- 128. face anonymization — анонимизация лиц | blurring faces in logs/videos

1.6 Navigation & LiDAR (86 terms)

1. LiDAR scan — скан лидара | one revolution/set of range readings
2. range reading — отсчет дальности | single distance measurement
3. scan topic — топик /scan | ROS LaserScan stream
4. occupancy probability — вероятность занятости | belief a cell is occupied
5. free probability — вероятность свободы | belief a cell is free
6. unknown cell — неизвестная ячейка | not yet observed cell
7. hit — попадание | ray ends on an obstacle
8. miss — промах луча | ray travels through free space
9. ray tracing (map) — трассировка луча по карте | updating cells along a beam
10. map persistence — сохранение карты | saving occupancy to disk
11. autosave — автосохранение | periodic map write
12. remembered_occupancy.json — remembered_occupancy.json | saved map file path pattern
13. goal pose — целевая поза | desired end x,y,yaw
14. start pose — стартовая поза | current pose when planning starts
15. path smoothing — сглаживание пути | reducing jagged planner output
16. replanning — перепланирование | computing a new path after change
17. blocked cell — заблокированная ячейка | non-traversable grid cell
18. inflation radius — радиус инфляции | safety expansion around obstacles
19. clearance — зазор | distance kept from obstacles
20. narrow passage — узкий проход | tight free-space corridor
21. dead end — тупик | path that cannot continue
22. local minimum — локальный минимум | trap for reactive controllers
23. oscillation — осцилляция | back-and-forth unstable motion
24. recovery spin — восстановительный разворот | rotate in place to escape stuck state
25. progress check — проверка прогресса | detecting lack of advancement
26. timeout to goal — таймаут до цели | max time allowed to reach goal
27. path length — длина пути | meters along planned route
28. path cost — стоимость пути | planner's scalar cost of a route
29. Manhattan distance — манхэттенское расстояние | $|dx|+|dy|$ heuristic
30. Euclidean distance — евклидово расстояние | straight-line distance
31. 8-connected — 8-связность | diagonal moves allowed in grid
32. 4-connected — 4-связность | only orthogonal grid moves
33. lookahead point — точка упреждения | pure-pursuit target on the path
34. curvature — кривизна | how sharply the path bends
35. cte correction — коррекция СТЕ | steering correction for cross-track error
36. yaw P-control — P-регулятор по курсу | proportional heading correction
37. max linear speed — макс. линейная скорость | v_x/v_y caps
38. max angular speed — макс. угловая скорость | w_z cap
39. acceleration ramp — рампа ускорения | gradual speed changes
40. stop distance — дистанция остановки | distance needed to halt safely
41. dynamic window — динамическое окно | velocity sampling local planner idea
42. DWA — DWA | Dynamic Window Approach
43. TEB — TEB | Timed Elastic Band local planner
44. MPPI — MPPI | sampling-based local planner family
45. cost function — целевая функция | scalar score minimized/maximized
46. constraint — ограничение | hard limit the planner must respect

47. feasibility — допустимость | whether a path can be executed
48. kinematic feasibility — кинематическая допустимость | path executable under motion model
49. map frame consistency — согласованность map-frame | poses expressed in same world frame
50. destination calibration — калибровка назначения | measuring real x,y for kiosk targets
51. uncalibrated destination — неоткалиброванная точка | template coords — escort disabled
52. fail-closed escort — сопровождение fail-closed | no move if destination invalid
53. route preview — предпросмотр маршрута | showing path before motion
54. operator goal — цель оператора | goal set from operator UI
55. escort goal — цель сопровождения | goal from passenger kiosk flow
56. scan matching score — оценка сопоставления | quality of CSM/ICP alignment
57. ICP iterations — итерации ICP | refinement loops in ICP
58. correlative search — корреляционный поиск | brute-ish score over pose hypotheses
59. pose hypothesis — гипотеза позы | candidate robot pose
60. drift — накопление ошибки | growing localization error
61. relocalization — релокализация | recovering pose after getting lost
62. kidnapped robot problem — задача похищенного робота | sudden unknown pose change
63. global localization — глобальная локализация | finding pose with no prior
64. tracking localization — сопровождающая локализация | updating pose with a good prior
65. map-to-odom — map→odom | transform correcting odom drift
66. odom frame — система odom | smooth locally consistent frame
67. slip compensation — компенсация проскальзывания | reducing odometry error from slip
68. floor type — тип покрытия | tile/carpet affecting calibration
69. LiDAR frequency — частота лидара | scans per second (e.g. 8 Hz)
70. serial baud (LiDAR) — скорость UART лидара | e.g. 230400
71. driver node — нода драйвера | ROS node talking to hardware — cspc_lidar
72. sensor occlusion — затенение сенсора | people/objects blocking LiDAR
73. glass wall — стеклянная стена | often poorly seen by LiDAR
74. mirror — зеркало | can create false LiDAR geometry
75. self-hit — попадание в себя | scan returns from robot body
76. range filtering — фильтрация дальностей | dropping invalid min/max ranges
77. intensity filter — фильтр по интенсивности | using return strength
78. multi-echo — мультиэхо | multiple returns along one beam
79. scan deskew — выравнивание скана | compensate motion during a scan
80. motion compensation — компенсация движения | correct measurements for ego-motion
81. map quality — качество карты | clarity/consistency of occupancy
82. noisy map — шумная карта | speckled incorrect cells
83. map cleanup — очистка карты | removing transient clutter
84. static layer — статический слой | permanent map obstacles
85. obstacle layer — слой препятствий | live obstacle costs
86. inflation layer — слой инфляции | expanded costs around obstacles

1.7 Networks & Communication (99 terms)

1. LAN — ЛВС | local area network
2. WLAN — WLAN | wireless LAN — Wi-Fi
3. Wi-Fi — Wi-Fi | wireless network used by Pi/Тапо
4. SSID — SSID | Wi-Fi network name
5. password — пароль | network/auth secret
6. IP address — IP-адрес | numeric host address
7. IPv4 — IPv4 | 32-bit internet addresses

8. IPv6 — IPv6 | 128-bit internet addresses
9. localhost — localhost | this machine — 127.0.0.1
10. loopback — loopback | interface to self
11. subnet — подсеть | group of addresses sharing a prefix
12. subnet mask — маска подсети | defines network vs host bits
13. gateway — шлюз | router to other networks
14. DNS — DNS | resolves names to IPs
15. DHCP — DHCP | automatic IP assignment
16. static IP — статический IP | manually fixed address
17. MAC address — MAC-адрес | hardware network identifier
18. NAT — NAT | network address translation
19. firewall — межсетевой экран | filters network traffic
20. port forwarding — проброс портов | exposing an internal port externally
21. HTTP server — HTTP-сервер | serves kiosk/operator UI on :8765
22. client IP allowlist — белый список IP клиентов | who may call control APIs
23. private IP — частный IP | RFC1918 address — 192.168/10/172.16
24. public network — публичная сеть | untrusted wide access — avoid for control
25. VPN — VPN | encrypted tunnel between networks
26. latency (net) — сетевая задержка | round-trip delay
27. packet loss — потеря пакетов | dropped network packets
28. bandwidth — пропускная способность | max data rate
29. throughput — фактическая скорость | achieved data rate
30. jitter (net) — джиттер сети | variation in latency
31. TCP retransmission — повтор TCP | resending lost TCP segments
32. keepalive — keepalive | messages that keep a session alive
33. heartbeat — heartbeat | periodic “I’m alive” signal
34. WebSocket reconnect — переподключение WebSocket | restoring a dropped live channel
35. TLS certificate — TLS-сертификат | proves server identity for HTTPS
36. man-in-the-middle — MITM | attacker between client and server
37. sniffing — sniffинг | capturing network traffic
38. SSID isolation — изоляция клиентов | AP feature separating clients
39. captive portal — каптивный портал | login page on public Wi-Fi
40. airport Wi-Fi — Wi-Fi аэропорта | usually untrusted for robot control
41. RTSP stream — RTSP-поток | camera media stream
42. ONVIF — ONVIF | IP camera interoperability standard
43. USB gadget — USB gadget | device mode USB networking
44. SSH — SSH | secure remote shell to the Pi
45. SCP — SCP | secure file copy
46. SFTP — SFTP | secure file transfer
47. mDNS — mDNS | local name resolution — .local
48. hostname — имя хоста | human name of the device
49. DNS spoofing — DNS-спуфинг | forged name resolution
50. ARP spoofing — ARP-спуфинг | forged LAN address mapping
51. rate limiting — ограничение частоты запросов | protects APIs from floods
52. DoS — DoS | denial of service attack
53. DDoS — DDoS | distributed DoS
54. authentication token — токен аутентификации | credential proving identity
55. session cookie — session cookie | browser session identifier
56. SameSite — SameSite | cookie attribute reducing CSRF

57. HTTPS redirect — редирект на HTTPS | force encrypted access
58. certificate pinning — пиннинг сертификата | trust only specific certs
59. mutual TLS — взаимный TLS | both sides present certificates
60. API key — API-ключ | secret identifying an API client
61. OAuth — OAuth | delegated authorization framework
62. Zero Trust — Zero Trust | never trust network location alone
63. network segmentation — сегментация сети | isolating robot/control VLANs
64. VLAN — VLAN | virtual LAN separation
65. QoS (net) — QoS сети | prioritizing traffic classes
66. MTU — MTU | max packet payload size
67. fragmentation — фрагментация | splitting packets to fit MTU
68. ICMP — ICMP | ping/control messages
69. ping — ping | reachability/latency probe
70. traceroute — traceroute | path discovery tool
71. netstat — netstat | lists sockets/connections
72. ss — ss | modern socket statistics tool
73. iptables — iptables | Linux packet filter tool
74. nftables — nftables | newer Linux firewall framework
75. proxy — прокси | intermediary for requests
76. reverse proxy — обратный прокси | front door to internal services
77. load balancer — балансировщик нагрузки | distributes traffic across backends
78. CDN — CDN | content delivery network
79. WebRTC — WebRTC | browser realtime media stack
80. STUN/TURN — STUN/TURN | NAT traversal helpers for WebRTC
81. MQTT — MQTT | lightweight IoT pub/sub protocol
82. AMQP — AMQP | message queue protocol
83. gRPC — gRPC | RPC over HTTP/2 + protobuf
84. WebSocket ping/pong — ping/pong WebSocket | keepalive frames
85. connection pool — пул соединений | reused network connections
86. keep-alive header — Keep-Alive | HTTP persistent connections
87. chunked transfer — chunked transfer | streaming HTTP body in chunks
88. gzip — gzip | content compression
89. CORS preflight — preflight CORS | OPTIONS check before cross-origin calls
90. mixed content — смешанный контент | HTTPS page loading HTTP resources
91. secure context — secure context | browser requirements for privileged APIs
92. localhost exception — исключение localhost | some APIs allowed on local origins
93. firewall rule — правило фаервола | allow/deny entry
94. egress — исходящий трафик | traffic leaving a host/network
95. ingress — входящий трафик | traffic entering a host/network
96. exfiltration — эксфильтрация | unauthorized data leaving the system
97. air gap — воздушный зазор | physical network isolation
98. OT network — OT-сеть | operational technology network
99. IT/OT convergence — схождение IT/OT | linking enterprise and control nets

1.8 Airport & Aviation (151 terms)

1. airport — аэропорт | complex for aircraft and passengers
2. terminal — терминал | passenger building
3. departures — вылет | departure area/hall
4. arrivals — прилет | arrival area/hall

5. check-in — регистрация | desk/process for accepting passengers
6. check-in desk — стойка регистрации | counter where bags/passengers are processed
7. self-service kiosk — киоск самообслуживания | passenger machine for check-in/info
8. boarding pass — посадочный талон | document authorizing boarding
9. gate — выход на посадку | boarding gate identifier/location
10. boarding — посадка | process of entering the aircraft
11. flight number — номер рейса | airline code + number
12. airline — авиакомпания | company operating the flight
13. departure time — время вылета | scheduled/actual departure
14. arrival time — время прилета | scheduled/actual arrival
15. delay — задержка | late departure/arrival
16. cancellation — отмена | flight called off
17. gate change — смена выхода | gate reassignment — stressful for passengers
18. baggage claim — выдача багажа | area to collect checked bags
19. carousel — лента выдачи | baggage belt
20. hand luggage — ручная кладь | cabin baggage
21. checked baggage — регистрируемый багаж | bags given at check-in
22. oversize baggage — негабаритный багаж | special oversized items
23. security checkpoint — досмотр / security | screening point
24. passport control — паспортный контроль | border identity check
25. customs — таможня | goods control
26. immigration — иммиграция | entry control for persons
27. transfer — пересадка | connection between flights
28. connection time — время стыковки | time available to catch next flight
29. layover — стыковка / ожидание | time spent between flights
30. transit passenger — транзитный пассажир | passing through without entering city
31. origin airport — аэропорт вылета | where journey starts
32. destination airport — аэропорт назначения | where journey ends
33. hub — хаб | major transfer airport
34. runway — ВПП | aircraft takeoff/landing strip
35. taxiway — рулежная дорожка | aircraft ground path
36. apron — перрон | aircraft parking/handling area
37. jet bridge — телетрап | boarding bridge to aircraft
38. stand — место стоянки ВС | aircraft parking position
39. ATC — УВД | air traffic control
40. IATA code — код IATA | 3-letter airport code
41. ICAO code — код ICAO | 4-letter airport code
42. FIDS — FIDS | flight information display system
43. timetable — расписание | published flight schedule
44. status board — табло | public flight status display
45. announcement — объявление | PA message to passengers
46. wayfinding — навигация по зданию | signs/systems helping people navigate
47. signage — указатели | visual direction signs
48. information desk — справочная | staffed help point
49. meeting point — место встречи | agreed place to meet
50. lost and found — бюро находок | office for lost items
51. passenger assistance — помощь пассажирам | support for those who need help
52. PRM — PRM | passengers with reduced mobility
53. wheelchair service — услуга кресла-коляски | mobility assistance

- 54. unaccompanied minor — ребенок без сопровождения | child traveling with airline care
- 55. family lane — семейная полоса | queue prioritization for families
- 56. queue — очередь | line of waiting passengers
- 57. peak hour — час пик | busiest traffic period
- 58. dwell time — время пребывания | how long people stay in an area
- 59. throughput (рах) — пропускная способность | passengers processed per time
- 60. landside — городская зона | public area before security
- 61. airside — стерильная зона | controlled area after security
- 62. sterile area — стерильная зона | secured post-screening zone
- 63. security zone — зона безопасности | controlled access area
- 64. access control — контроль доступа | who may enter which zone
- 65. badge — пропуск | staff credential
- 66. escort (airport) — сопровождение | authorized accompanying person/robot
- 67. airside escort rules — правила сопровождения airside | strict human/process rules — robots constrained
- 68. concession — торговая точка | shop/restaurant in terminal
- 69. duty free — дьюти-фри | tax-free retail
- 70. lounge — зал ожидания повышенной комфортности | airline/priority lounge
- 71. gate holdroom — накопитель у выхода | seating area at the gate
- 72. boarding group — группа посадки | ordered boarding cohort
- 73. priority boarding — приоритетная посадка | early boarding privilege
- 74. standby — стэндбай | waitlist passenger status
- 75. overbooking — овербукинг | more seats sold than available
- 76. denied boarding — отказ в посадке | passenger not allowed to board
- 77. rebooking — перебронирование | assigning a new itinerary
- 78. itinerary — маршрутная квитанция / itinerary | full trip plan
- 79. PNR — PNR | passenger name record
- 80. e-ticket — электронный билет | ticket stored electronically
- 81. barcode boarding pass — посадочный со штрихкодом | pass readable by scanners
- 82. mobile boarding pass — мобильный посадочный | phone-displayed pass
- 83. biometric boarding — биометрическая посадка | face/fingerprint boarding flows
- 84. API (airport data) — API данных аэропорта | programmatic flight/status feeds
- 85. AODB — AODB | airport operational database
- 86. BHS — BHS | baggage handling system
- 87. CUTE/CUPPS — CUTE/CUPPS | shared check-in platform standards
- 88. SITA — SITA | major aviation IT provider
- 89. GDPR — GDPR | EU personal data regulation
- 90. personal data — персональные данные | identifying passenger information
- 91. consent — согласие | permission to process data
- 92. data minimization — минимизация данных | collect only what is needed
- 93. CCTV — CCTV | security camera system
- 94. crowd management — управление потоками | controlling dense passenger flows
- 95. evacuation route — путь эвакуации | emergency exit path
- 96. muster point — пункт сбора | emergency assembly location
- 97. fire door — противопожарная дверь | rated door on escape routes
- 98. accessibility ramp — пандус доступности | wheelchair-accessible incline
- 99. tactile paving — тактильная плитка | texture guiding visually impaired
- 100. multilingual — многоязычный | supporting many languages — RU/EN/ZH/TT
- 101. Tatar language — татарский язык | TT locale in AeroPorter kiosk
- 102. wayfinding robot — робот-навигатор | robot helping people find places

103. customer experience — клиентский опыт | quality of passenger journey
104. SLA — SLA | service level agreement
105. uptime — аптайм | time the service is available
106. incident response — реакция на инцидент | process after operational problems
107. SOP — SOP | standard operating procedure
108. airside vehicle permit — допуск транспорта airside | permission for vehicles airside
109. foreign object debris — FOD | debris hazardous to aircraft — airside concern
110. ramp safety — безопасность перрона | rules for people/vehicles near aircraft
111. jet blast — струя двигателей | dangerous engine exhaust airflow
112. pushback — pushback | aircraft pushed from stand
113. turnaround — оборот ВС | aircraft ground service between flights
114. slot — слот | allocated time for departure/arrival
115. curfew — комендантский час | night flight restrictions
116. noise abatement — снижение шума | procedures reducing noise impact
117. meteorological delay — метеозадержка | weather-caused delay
118. de-icing — противогололедная обработка | removing ice from aircraft
119. NOTAM — NOTAM | notice to airmen — operational notices
120. airport authority — администрация аэропорта | organization running the airport
121. ground handler — агент наземного обслуживания | company serving aircraft/passengers on ground
122. handling agent — агент обслуживания | similar ground service role
123. airline representative — представитель авиакомпании | airline staff in terminal
124. passenger load factor — кресло-километры / LF | share of seats filled
125. peak passenger flow — пиковый пассажиропоток | maximum simultaneous crowd
126. dwell analytics — аналитика пребывания | measuring where people spend time
127. indoor navigation app — приложение indoor-навигации | phone app for terminal maps
128. digital twin (airport) — цифровой двойник аэропорта | virtual model of terminal ops
129. robot deployment zone — зона эксплуатации робота | approved area for robot operation
130. human supervisor — человек-супервизор | required oversight for public robots
131. liability — ответственность | legal responsibility for harm/damage
132. insurance — страхование | financial risk transfer
133. pilot zone — пилотная зона | limited area for trials
134. stakeholder — стейкхолдер | party with interest — airport, airline, pax
135. procurement — закупки | buying process for airport equipment
136. TCO — TCO | total cost of ownership
137. CAPEX — CAPEX | capital expenditure
138. OPEX — OPEX | operating expenditure
139. ROI — ROI | return on investment
140. payback period — срок окупаемости | time to recover investment
141. analog (competitor) — аналог | comparable existing product — Pepper etc.
142. Pepper — Pepper | softbank-style humanoid service robot analog
143. Sanbot — Sanbot | service robot analog
144. Promobot — Promobot | Russian service robot analog
145. fixed information kiosk — стационарный инфокиоск | non-mobile passenger terminal
146. last-mile guidance — сопровождение «последней мили» | help for the final stretch to a place
147. stress reduction — снижение стресса | UX goal for confused passengers
148. language barrier — языковой барьер | difficulty from different languages
149. elderly passenger — пожилой пассажир | key assisted user group
150. family with children — семья с детьми | key assisted user group
151. foreign passenger — иностранный пассажир | key assisted user group

1.9 Engineering Verbs (205 terms)

1. design — проектировать | create the architecture/plan
2. implement — реализовывать | turn a design into working code/hardware
3. integrate — интегрировать | connect subsystems into one system
4. calibrate — калибровать | adjust parameters to match reality
5. validate — валидировать | check the right problem is solved
6. verify — верифицировать | check the implementation meets specs
7. deploy — развертывать | put the system into an operating environment
8. prototype — прототипировать | build an early testable version
9. iterate — итерировать | improve through repeated cycles
10. debug — отлаживать | find and fix defects
11. troubleshoot — диагностировать неисправность | systematic fault finding
12. measure — измерять | obtain quantitative data
13. estimate — оценивать | approximate a value under uncertainty
14. localize — локализовать | estimate pose on a map
15. map — картографировать | build an environmental map
16. plan — планировать | compute a path or action sequence
17. navigate — навигировать | move toward goals using sensing
18. track — отслеживать | follow a target over time
19. detect — детектировать | find an object/event in sensor data
20. classify — классифицировать | assign a category
21. filter — фильтровать | reduce noise or select signals
22. fuse — сливать данные | combine multiple sensor sources
23. actuate — приводить в движение | drive motors/actuators
24. command — командовать | send control setpoints
25. feedback — обеспечивать ОС | use measurements to correct
26. stabilize — стабилизировать | keep a variable near a setpoint
27. constrain — ограничивать | enforce limits
28. saturate — насыщать | hit actuator limits
29. throttle — дросселировать | intentionally limit rate/power
30. watchdog — контролировать таймаутом | stop on stale commands
31. timeout — завершать по таймауту | abort after max wait
32. heartbeat — посылать heartbeat | signal liveness periodically
33. lease — выдавать аренду управления | grant time-limited control rights
34. revoke — отзываться | take back a permission/lease
35. allowlist — вносить в белый список | explicitly permit
36. deny — отклонять | refuse an action/request
37. authenticate — аутентифицировать | verify identity
38. authorize — авторизовать | check permission
39. encrypt — шифровать | protect confidentiality
40. log — логировать | record events
41. monitor — мониторить | observe runtime health
42. profile — профилировать | measure performance hotspots
43. benchmark — бенчмаркать | run comparative performance tests
44. optimize — оптимизировать | improve efficiency/quality
45. refactor — рефакторить | restructure without behavior change
46. modularize — модуляризовать | split into modules
47. abstract — абстрагировать | hide detail behind interfaces
48. encapsulate — инкапсулировать | bundle data with behavior

49. serialize — сериализовать | convert to storable/transmittable form
50. parse — парсить | interpret structured input
51. validate input — проверять ввод | reject bad requests
52. sanitize — санитизировать | clean unsafe input
53. retry — повторять | attempt again after failure
54. backoff — делать backoff | delay increasingly between retries
55. fallback — переключаться на резерв | use alternative path
56. degrade — деградировать | reduce functionality safely
57. recover — восстанавливаться | return to normal after fault
58. reboot — перезагружать | restart the computer/MCU
59. flash — прошивать | write firmware to MCU
60. compile — компилировать | translate source to binary
61. build — собирать | produce runnable artifacts
62. install — устанавливать | place software/deps on a system
63. configure — конфигурировать | set parameters
64. parameterize — параметризовать | expose tunable values
65. tune — настраивать | adjust gains/thresholds
66. simulate — моделировать | test in a virtual environment
67. emulate — эмулировать | imitate hardware/software behavior
68. reproduce — воспроизводить | get the same result again
69. isolate — изолировать | separate cause or component
70. instrument — инструментировать | add probes/metrics
71. trace — трассировать | follow execution/request path
72. assert — утверждать (assert) | enforce a must-be-true condition
73. enforce — принудительно обеспечивать | make a rule unavoidable
74. constrain path — ограничивать путь | force planner through corridors
75. inflate — раздувать препятствия | expand obstacles for safety
76. rasterize — растривать | convert to grid cells
77. interpolate — интерполировать | estimate between samples
78. extrapolate — экстраполировать | estimate beyond samples
79. downsample — прореживать | reduce sample rate/resolution
80. upsample — увеличивать частоту/разрешение | increase sample rate/resolution
81. threshold — пороговать | decide by cutoff
82. segment — сегментировать | split into regions/classes
83. annotate — размечать | label data
84. train — обучать | fit a model
85. infer — выполнять инференс | run model prediction
86. deploy model — развертывать модель | put ML model into production path
87. stream — стримить | send continuous media/data
88. buffer — буферизовать | temporarily store flowing data
89. queue — ставить в очередь | enqueue work/messages
90. publish — публиковать | send on a ROS topic
91. subscribe — подписываться | receive from a ROS topic
92. broadcast — широко вещать | send to many receivers
93. handshake — выполнять handshake | agree protocol parameters
94. negotiate — согласовывать | agree settings between peers
95. ping — пинговать | check reachability
96. reconnect — переподключаться | restore a dropped link
97. mount — монтировать | attach storage/device in OS

- 98. unmount — размонтировать | detach storage safely
- 99. persist — персистить | save durable state
- 100. cache — кэшировать | store for faster reuse
- 101. invalidate — инвалидировать | mark cache/state as stale
- 102. migrate — мигрировать | move data/schema/versions
- 103. roll out — выкатывать | release gradually
- 104. roll back — откатывать | revert a release
- 105. canary — канареечно выкатывать | test on a small subset first
- 106. A/B test — A/B-тестировать | compare variants empirically
- 107. survey — опрашивать пользователей | collect user feedback
- 108. interview — проводить интервью | qualitative requirement gathering
- 109. specify — специфицировать | write precise requirements
- 110. prioritize — приоритизировать | order work by importance
- 111. scope — определять рамки | define what is in/out
- 112. trade off — идти на компромисс | balance competing goals
- 113. justify — обосновывать | give reasons for a choice
- 114. compare — сравнивать | evaluate alternatives
- 115. benchmark against — сравнивать с эталоном | compare to a baseline/analog
- 116. outperform — превосходить | do better than an alternative
- 117. underperform — уступать | do worse than expected/alternative
- 118. mitigate — митигировать | reduce risk impact/likelihood
- 119. accept risk — принимать риск | consciously keep residual risk
- 120. transfer risk — передавать риск | e.g. via insurance
- 121. escalate — эскалировать | raise issue to higher authority
- 122. document — документировать | write down design/ops knowledge
- 123. demonstrate — демонстрировать | show working behavior
- 124. present — презентовать | explain to an audience/jury
- 125. defend — защищать (проект) | argue for your engineering choices
- 126. clarify — уточнять | make a statement clearer
- 127. rephrase — перефразировать | say the same idea differently
- 128. summarize — резюмировать | compress to key points
- 129. elaborate — подробно раскрывать | add detail
- 130. illustrate — иллюстрировать | explain with an example
- 131. quantify — квантифицировать | express with numbers
- 132. qualify — давать оговорки | add conditions/limits to a claim
- 133. acknowledge — признавать | admit a limitation/point
- 134. concede — уступать в споре | accept part of a criticism
- 135. counter — возражать | respond with a counter-argument
- 136. refute — опровергать | show a claim is wrong
- 137. hypothesize — выдвигать гипотезу | propose a testable explanation
- 138. experiment — экспериментировать | test under controlled conditions
- 139. observe — наблюдать | collect qualitative/quantitative observations
- 140. record — записывать | store measurements/logs
- 141. analyze — анализировать | extract meaning from data
- 142. visualize — визуализировать | show data graphically
- 143. interpret — интерпретировать | explain what results mean
- 144. conclude — делать вывод | state the outcome of reasoning
- 145. recommend — рекомендовать | propose next actions
- 146. schedule — планировать по времени | set timing of tasks

147. allocate — распределять | assign resources
148. budget — бюджетировать | plan money/time/energy
149. procure — закупать | buy components/services
150. fabricate — изготавливать | make physical parts
151. assemble — собирать (механику) | put parts together
152. wire — разводить провода | connect electrical wiring
153. solder — паять | join with solder
154. crimp — обжимать | terminate wires with a tool
155. torque (verb) — затягивать моментом | tighten fasteners to spec
156. align — выравнивать | set correct geometric relationship
157. balance — балансировать | adjust mass/forces symmetrically
158. lubricate — смазывать | apply lubricant
159. inspect — инспектировать | examine for defects
160. maintain — обслуживать | keep operational via care
161. replace — заменять | swap a failed part
162. repair — ремонтировать | restore a broken part/system
163. upgrade — апгрейдить | replace with a better version
164. retrofit — модернизировать на месте | add improvements to existing hardware
165. harden — упрочнять / ужесточать | make more robust/secure
166. stress-test — нагрузочно тестировать | push beyond normal load
167. soak-test — длительно тестировать | run long-duration tests
168. acceptance-test — проводить приемку | test against acceptance criteria
169. commission — вводить в эксплуатацию | bring into service
170. decommission — выводить из эксплуатации | retire from service
171. shutdown — выключать | power/process stop sequence
172. power-cycle — сбрасывать питанием | off/on to reset
173. brownout-protect — защищать от просадок | design against voltage dips
174. fuse-protect — защищать предохранителем | limit fault current
175. isolate fault — локализовать неисправность | contain a failing subsystem
176. hot-swap — горячо заменять | replace while system stays up
177. cold-boot — холодно загружать | boot from powered-off state
178. warm-start — тепло стартовать | restart preserving some state
179. seed map — инициализировать карту | create initial occupancy prior
180. clear map — очищать карту | reset occupancy knowledge
181. save map — сохранять карту | persist occupancy to disk
182. load map — загружать карту | restore occupancy from disk
183. set goal — задавать цель | provide A→B target
184. cancel goal — отменять цель | stop current navigation goal
185. replan — перепланировать | compute a new path
186. pursue — преследовать точку пути | follow look-ahead point
187. strafe — двигаться боком | command lateral mecanum motion
188. rotate in place — разворачиваться на месте | yaw without translation
189. dock — стыковаться | approach a station precisely
190. undock — отстыковываться | leave a docking station
191. escort — сопровождать | lead a passenger along a route
192. announce — объявлять | give status to the user
193. prompt — подсказывать | ask the user for input
194. localize ticket — находить билет | look up boarding data by code
195. scan barcode — сканировать штрихкод | read ticket code optically

196. render UI — отрисовывать интерфейс | draw kiosk screens
197. localize UI — локализовать UI | translate interface strings
198. throttle logs — ограничивать логи | reduce log volume
199. rotate logs — ротировать логи | archive old log files
200. redact — редактировать/скрывать | remove sensitive data from logs
201. anonymize — анонимизировать | remove identifying information
202. minimize data — минимизировать данные | collect only necessary fields
203. comply — соответствовать требованиям | meet legal/policy rules
204. audit — аудировать | review for compliance/quality
205. sign off — утверждать | formally accept a milestone

1.10 Engineering Adjectives (228 terms)

1. robust — робастный | works under disturbance/uncertainty
2. reliable — надежный | consistently performs as expected
3. safe — безопасный | acceptable risk near people
4. fail-safe — отказобезопасный | defaults to safe on failure
5. fail-closed — закрытый при отказе | denies action when unsure
6. deterministic — детерминированный | same inputs → same outputs
7. realtime — работающий в реальном времени | meets timing needs
8. low-latency — низколатентный | small delay
9. high-throughput — высокопроизводительный по потоку | handles many ops/data
10. scalable — масштабируемый | handles growth reasonably
11. modular — модульный | composed of replaceable parts
12. maintainable — поддерживаемый | easy to service/modify
13. extensible — расширяемый | easy to add features
14. portable — переносимый | moves across platforms
15. compatible — совместимый | works with other components
16. interoperable — интероперабельный | exchanges data across systems
17. redundant — резервированный | has backups for faults
18. resilient — устойчивый к сбоям | recovers from problems
19. fault-tolerant — отказоустойчивый | continues despite faults
20. secure — защищенный | resists unauthorized access
21. encrypted — зашифрованный | confidentiality protected
22. authenticated — аутентифицированный | identity verified
23. authorized — авторизованный | permission granted
24. isolated — изолированный | separated from other domains
25. sandboxed — в песочнице | restricted execution context
26. privileged — привилегированный | has elevated rights
27. unprivileged — непривилегированный | limited rights
28. local — локальный | on-device / on-LAN
29. remote — удаленный | over a network
30. offline — офлайн | without external connectivity
31. online — онлайн | connected/available
32. embedded — встроенный | software on constrained hardware
33. bare-metal — на «голом железе» | without full OS abstraction
34. resource-constrained — ресурсно-ограниченный | limited CPU/RAM/power — Pi class
35. power-efficient — энергоэффективный | good performance per watt
36. thermal-limited — теплоограниченный | performance capped by heat
37. battery-powered — батарейный | runs from a pack

- 38. wired — проводной | uses cables
- 39. wireless — беспроводной | no cable for that link
- 40. omnidirectional — всенаправленный | moves any planar direction
- 41. holonomic — голономный | independent planar DOFs
- 42. non-holonomic — неголономный | motion-constrained platform
- 43. differential — дифференциальный | diff-drive style
- 44. mecanum-based — на механумах | using mecanum wheels
- 45. closed-loop — замкнутый | uses feedback
- 46. open-loop — разомкнутый | no feedback correction
- 47. proportional — пропорциональный | P-term style response
- 48. integral — интегральный | I-term style response
- 49. derivative — дифференциальный (D) | D-term style response
- 50. nonlinear — нелинейный | not a straight-line relationship
- 51. linear — линейный | superposition holds
- 52. discrete — дискретный | sampled in time/value
- 53. continuous — непрерывный | not sampled / smooth
- 54. probabilistic — вероятностный | explicitly models uncertainty
- 55. heuristic — эвристический | uses practical rules of thumb
- 56. optimal — оптимальный | best under a defined cost
- 57. suboptimal — субоптимальный | good but not proven best
- 58. approximate — приближенный | not exact, but useful
- 59. exact — точный (exact) | no approximation in the method
- 60. numerical — численный | computed by numeric methods
- 61. analytical — аналитический | closed-form solution
- 62. empirical — эмпирический | based on experiment/data
- 63. theoretical — теоретический | based on models/proofs
- 64. experimental — экспериментальный | obtained via tests
- 65. simulated — смоделированный | from simulation
- 66. field-tested — проверенный в поле | tested in realistic settings
- 67. lab-validated — проверенный в лаборатории | validated in controlled lab
- 68. production-ready — готовый к эксплуатации | mature enough for ops — we do NOT claim this yet
- 69. prototype-grade — прототипного уровня | acceptable for demos/learning
- 70. industrial — промышленный | built to industrial standards
- 71. consumer-grade — бытового уровня | cheaper, less rugged parts
- 72. ruggedized — упрочненный | hardened for harsh use
- 73. IP-rated — с рейтингом IP | sealed to an IP standard
- 74. calibrated — откалиброванный | parameters matched to reality
- 75. uncalibrated — неоткалиброванный | not yet measured/adjusted
- 76. aligned — выровненный | geometrically correct
- 77. misaligned — смещенный | geometric error present
- 78. synchronized — синхронизированный | time-aligned across components
- 79. asynchronous — асинхронный | not strictly time-locked
- 80. concurrent — одновременный | overlapping in time
- 81. sequential — последовательный | one after another
- 82. atomic — атомарный | indivisible operation
- 83. idempotent — идемпотентный | safe to repeat
- 84. stateful — с состоянием | keeps memory across calls
- 85. stateless — без состояния | no retained client memory
- 86. blocking — блокирующий | waits before continuing

- 87. non-blocking — неблокирующий | returns immediately
- 88. synchronous — синхронный | caller waits for completion
- 89. event-driven — событийный | reacts to events
- 90. polling-based — на опросе | checks status repeatedly
- 91. interrupt-driven — на прерываниях | reacts via ISRs
- 92. priority-based — приоритетный | ordered by importance
- 93. best-effort — по возможности | no hard guarantee
- 94. guaranteed — гарантированный | meets a hard promise
- 95. soft-realtime — мягкого РВ | timing preferred, not certified hard
- 96. hard-realtime — жесткого РВ | missing deadline = failure
- 97. deterministic-timing — с детерминированными таймингами | low jitter schedule
- 98. jittery — с джиттером | unstable timing
- 99. noisy — шумный | corrupted by noise
- 100. clean — чистый (сигнал) | low noise
- 101. biased — смещенный | has systematic offset
- 102. unbiased — несмещенный | no systematic offset
- 103. precise — прецизионный | low random scatter
- 104. accurate — правильный / точный | close to truth
- 105. repeatable — повторяемый | same result on repeats
- 106. reproducible — воспроизводимый | others can obtain same result
- 107. stable — устойчивый | returns to equilibrium
- 108. unstable — неустойчивый | diverges or oscillates badly
- 109. oscillatory — колебательный | shows oscillation
- 110. damped — задемпфированный | oscillation reduced
- 111. overdamped — передемпфированный | slow return without overshoot
- 112. underdamped — недодемпфированный | rings before settling
- 113. saturated — насыщенный | at actuator limits
- 114. unsaturated — ненасыщенный | within actuator range
- 115. feasible — допустимый | possible under constraints
- 116. infeasible — недопустимый | violates constraints
- 117. collision-free — безстолкновительный | no predicted collisions
- 118. collision-prone — склонный к столкновениям | high collision risk
- 119. obstacle-rich — насыщенный препятствиями | many obstacles
- 120. open-space — открытое пространство | few obstacles
- 121. crowded — многолюдный | many people nearby
- 122. sparse — разреженный | few obstacles/people
- 123. dynamic — динамический | changing over time
- 124. static — статический | unchanging
- 125. indoor — внутренний | inside buildings
- 126. outdoor — уличный | outside
- 127. GPS-denied — без GPS | no satellite fix — terminals
- 128. map-based — на основе карты | uses an explicit map
- 129. mapless — без карты | reactive without global map
- 130. vision-based — на зрении | uses cameras as primary sense
- 131. lidar-based — на лидаре | uses LiDAR as primary sense
- 132. multimodal — мультимодальный | multiple sensing modalities
- 133. redundant-sensing — с дублированием сенсоров | overlapping sensors
- 134. single-point-of-failure — с единой точкой отказа | one failure stops all
- 135. mission-critical — критически важный | failure has major impact

136. safety-critical — критичный для безопасности | failure can cause harm
137. user-facing — пользовательский | seen by passengers
138. operator-facing — операторский | seen by staff
139. multilingual — многоязычный | many languages
140. accessible — доступный | usable by diverse users
141. intuitive — интуитивный | easy to understand without training
142. minimal — минималистичный | only essentials
143. verbose — подробный | lots of detail/output
144. silent — тихий | low acoustic noise
145. audible — слышимый | loud enough to notice
146. visible — видимый | easy to see status/robot
147. opaque — непрозрачный | not transparent to user/operator
148. transparent — прозрачный (процесс) | easy to inspect/understand
149. explainable — объяснимый | decisions can be justified
150. black-box — черный ящик | internals not interpretable
151. white-box — белый ящик | internals inspectable
152. open-source — открытый исходный код | source publicly available
153. proprietary — проприетарный | closed intellectual property
154. custom — кастомный | built specifically for this project
155. off-the-shelf — готовое / серийное | bought ready-made
156. commercial — коммерческий | sold as a product
157. academic — академический | research/education oriented
158. cost-effective — экономически эффективный | good value for cost
159. low-cost — недорогой | cheap relative to alternatives
160. expensive — дорогой | high cost
161. lightweight — легкий | low mass
162. compact — компактный | small footprint
163. bulky — громоздкий | large/awkward
164. maneuverable — маневренный | easy to move in tight spaces
165. agile — проворный | quick to change motion
166. sluggish — вялый | slow to respond
167. responsive — отзывчивый | reacts quickly
168. smooth — плавный | without jerks
169. jerky — рывковый | abrupt motion
170. precise-tracking — с точным слежением | follows path closely
171. drift-prone — склонный к дрейфу | odometry/localization drifts
172. well-calibrated — хорошо откалиброванный | parameters trustworthy
173. floor-dependent — зависящий от покрытия | needs recalibration on new floors
174. context-aware — контекстно-зависимый | behavior depends on situation
175. passenger-centric — ориентированный на пассажира | designed around passenger needs
176. human-friendly — дружественный к человеку | pleasant/safe near people
177. airport-specific — специфичный для аэропорта | tailored to terminal domain
178. domain-specific — предметно-ориентированный | built for one domain
179. general-purpose — универсальный | meant for many domains
180. task-specific — задачно-специфичный | optimized for one task
181. end-to-end — сквозной | covers full pipeline
182. incremental — поэтапный | delivered in steps
183. iterative — итеративный | improved in cycles
184. incremental-mapping — с инкрементальным картированием | map grows over time

185. global — глобальный | whole-map / whole-mission scope
186. local — локальный (планирование) | short-horizon scope
187. reactive — реактивный | responds to immediate sensing
188. deliberative — делиберативный | plans using models/maps
189. hybrid — гибридный | mix of approaches
190. hierarchical — иерархический | layered control/planning
191. distributed — распределенный | split across machines/nodes
192. centralized — централизованный | one coordinator decides
193. edge-compute — на граничных вычислениях | compute near the robot
194. cloud-connected — облачный | depends on cloud services
195. cloud-independent — независимый от облака | works without cloud
196. privacy-preserving — сохраняющий приватность | minimizes personal data risk
197. GDPR-aware — с учетом GDPR | designed with data rules in mind
198. audit-friendly — удобный для аудита | easy to review later
199. observable — наблюдаемый | state visible via logs/metrics
200. debuggable — удобный для отладки | easy to diagnose
201. testable — тестируемый | easy to write tests for
202. flaky — плавающий (тест) | tests fail nondeterministically
203. flaky-hardware — нестабильное железо | intermittent hardware faults — FR encoder
204. trusted — доверенный | considered reliable — trusted encoders
205. untrusted — недоверенный | known unreliable — FR encoder
206. experimental-feature — экспериментальная функция | not finalized
207. stable-feature — стабильная функция | ready enough for demos
208. deprecated — устаревший | planned for removal
209. legacy — унаследованный | old but still present
210. forward-compatible — совместимый вперед | works with future versions
211. backward-compatible — совместимый назад | works with older clients
212. breaking — ломающий совместимость | requires client changes
213. non-breaking — неломаящий | safe for existing clients
214. critical — критический | highest severity/importance
215. major — мажорный | large impact
216. minor — минорный | small impact
217. negligible — пренебрежимый | tiny impact
218. acceptable — приемлемый | within tolerance
219. unacceptable — неприемлемый | outside tolerance
220. marginal — пограничный | barely acceptable
221. nominal — номинальный | within normal design point
222. peak — пиковый | maximum short-term
223. continuous — длительный (режим) | sustainable over time
224. intermittent — прерывистый | comes and goes
225. transient — переходный | short-lived
226. steady-state — установившийся | settled long-term behavior
227. warm — протретый | after thermal/state settle
228. cold-start — с холодного старта | from empty/initial state

Vocabulary count summary

- 1.1 Mechanics: 236
- 1.2 Electronics: 243
- 1.3 Robotics: 263
- 1.4 Programming: 343

- 1.5 Computer Vision: 129
 - 1.6 Navigation & LiDAR: 86
 - 1.7 Networks & Communication: 99
 - 1.8 Airport & Aviation: 151
 - 1.9 Engineering Verbs: 205
 - 1.10 Engineering Adjectives: 228
 - TOTAL: 1983 terms
-

2. Most Useful Phrases

Presentation and defense phrases — not casual small talk.

2.1 Explaining the idea (46)

1. AeroPorter is a mobile robot assistant designed to help passengers navigate an airport terminal.
2. Our goal is to turn a boarding pass into a guided route to the right place.
3. The core user journey is: scan the ticket, show flight information, and escort the passenger A to B.
4. We focus on elderly passengers, families with children, and foreign travelers.
5. The problem is not a lack of signs — it is stress, language barriers, and changing gate information.
6. We built a prototype that combines a passenger kiosk with autonomous navigation.
7. Unlike a fixed information desk, the robot can move with the passenger.
8. The project is an engineering prototype, not a certified airport product yet.
9. We separated the system into chassis, navigation, camera, and kiosk modules.
10. Each module can be tested independently, which made development safer.
11. The robot runs on a Raspberry Pi 5 with an Arduino Mega handling low-level motor control.
12. Navigation is based on LiDAR mapping and A-star path planning.
13. The passenger interface supports Russian, English, Chinese, and Tatar.
14. Ticket lookup uses a barcode scan or manual code entry as a fallback.
15. Escort starts only to calibrated destinations — fail-closed if coordinates are invalid.
16. In short, AeroPorter is a moving guide that links ticket data to a physical route.
17. We chose mecanum wheels so the robot can strafe in crowded corridors.
18. The operator console lets staff supervise mapping and teleoperation.
19. Our thesis is: help the passenger at the point of confusion, not only at a fixed desk.
20. The value is reduced stress and fewer repetitive questions to airport staff.
21. We designed for indoor GPS-denied environments typical of terminals.
22. The system is intentionally modular so one failure does not freeze all development.
23. AeroPorter answers three questions: Where should I go? How do I get there? Can someone lead me?
24. We treat the airport as a dynamic human environment, not an empty warehouse.
25. The product idea is passenger escort; the engineering core is safe indoor navigation.
26. We started from a real pain point observed in terminals: changing gates and long walking distances.
27. The robot does not replace staff — it reduces routine navigation questions.
28. Our prototype proves the software pipeline from ticket to motion command.
29. We keep claims honest: the navigation program exists; real-terminal deployment is future work.
30. The kiosk is the human interface; LiDAR navigation is the mobility engine.
31. By combining ticket context and a map, we avoid generic 'follow me' without destination meaning.
32. The project sits at the intersection of robotics, UX, and airport operations.
33. We prioritized reliability of stop-on-fault behavior over flashy autonomy claims.
34. AeroPorter is optimized for landside / terminal assistance scenarios in a pilot zone.
35. The key innovation for us is the end-to-end stack on affordable hardware.
36. We deliberately used custom navigation instead of blindly depending on Nav2.
37. The robot's job is guidance; safety stop logic is non-negotiable.
38. Think of it as a bilingual moving wayfinding system with a ticket brain.
39. We built what students can fully understand and defend: every major block is ours to explain.
40. The idea is simple to say and hard to implement — that is exactly why it is a strong engineering project.
41. Passengers need actionable help now, not another static map on the wall.
42. Our system converts a code on a ticket into a goal pose on an occupancy map.
43. If escort is unavailable, the kiosk can still show flight information — graceful degradation.
44. We designed for supervised operation first, full autonomy later.
45. We built AeroPorter as a school engineering project with a real operational story.

46. The robot is a prototype you can audit line by line.

2.2 Describing algorithms (46)

1. We plan paths with A-star on an occupancy grid.
2. The heuristic is Manhattan distance, which fits a grid world.
3. Pure pursuit follows a look-ahead point on the planned path.
4. Cross-track error is corrected while tracking the path.
5. Occupancy cells are updated with a log-odds sensor model.
6. LiDAR rays mark hits as occupied and free space along the beam as free.
7. Scan matching uses correlative search plus ICP refinement.
8. Odometry integrates wheel encoder ticks from three trusted wheels.
9. The front-right encoder is untrusted, so we do not rely on it for pose.
10. Inverse kinematics convert body velocity into four mecanum wheel commands.
11. A PI controller aligns trusted wheel rates to the commanded mix.
12. Temporary obstacles are inflated for a short TTL during planning.
13. The robot body is filtered from scans so chassis posts are not mapped as walls.
14. Yaw hold from LiDAR helps stabilize heading against wheel slip.
15. Face detection uses YuNet; the image is digitally cropped to keep the face centered.
16. Ticket barcodes are read in the browser via the BarcodeDetector API.
17. If barcode detection fails, the user can type the code manually.
18. Destination lookup maps a ticket to a calibrated goal coordinate.
19. HTTP control uses an allowlist and origin checks before motion commands.
20. A heartbeat lease stops escort if the kiosk stops refreshing.
21. The Arduino stops motors if no serial command arrives for about 1.2 seconds.
22. The drive watchdog zeroes stale velocity commands after about 0.25 seconds.
23. Map persistence saves the occupancy grid so we can reuse a remembered map.
24. Replanning occurs when the path becomes blocked by new obstacle hits.
25. We do not claim classical full SLAM with loop-closure like Cartographer.
26. Our localization is odometry plus scan matching against the occupancy map.
27. Eight-connected A-star allows diagonal steps on the grid.
28. Goal tolerance defines when we consider the robot 'arrived'.
29. Cost inflation around obstacles creates a safety buffer for the footprint.
30. The control loop repeatedly senses, plans or tracks, and actuates.
31. CSM scores pose hypotheses; ICP polishes the best alignment.
32. Encoder ticks are converted to distance using wheel diameter and ticks per revolution.
33. Mecanum kinematics allow vx, vy, and yaw rate as independent commands.
34. The occupancy resolution is fine enough for corridor navigation, on the order of centimeters per cell.
35. We separate global planning (A-star) from local tracking (pure pursuit).
36. Fail-closed destination checks prevent navigating to nonsense coordinates.
37. Demo ticket codes exist for laboratory demonstration when the SQLite DB is empty.
38. The algorithm stack is intentionally transparent — every stage is explainable at defense.
39. We prefer a simple correct pipeline over a complex black box we cannot justify.
40. Path cost combines distance with occupancy penalties.
41. Look-ahead distance trades smoothness against corner cutting.
42. If tracking error grows, we can stop and replan rather than force a bad trajectory.
43. Scan matching reduces long-term odometry drift in structured corridors.
44. The serial protocol between Pi and Mega carries velocity commands and encoder telemetry.
45. Grid planning keeps the method teachable at defense.
46. Sensor fusion here means encoders plus LiDAR scan matching, not a mysterious black box.

2.3 Comparing solutions (37)

1. Compared with a fixed kiosk, AeroPorter can move with the passenger.
2. Compared with Pepper-like humanoids, our platform is simpler, cheaper, and focused on escort.
3. Compared with Promobot-style info robots, we emphasize ticket-to-route navigation.
4. Compared with phone indoor apps, a physical robot helps people who struggle with phones.
5. Compared with Nav2, our custom stack is smaller and fully understood by the team.
6. Nav2 is powerful, but integrating and tuning it was heavier than our project scope needed.
7. A differential drive would be simpler, but mecanum gives strafe in narrow crowds.
8. An NVIDIA Jetson would add GPU power, but Raspberry Pi 5 already covers our current CPU workloads.
9. A commercial AGV stack would be more certified, but far more expensive and less educational.
10. Vision-only navigation is fragile in glass-heavy terminals; LiDAR is more reliable for ranging.
11. Ultrasonic sensors are cheap but sparse; LiDAR gives dense 360-degree structure.
12. Cloud ticket lookup is possible later; local SQLite keeps the demo self-contained.
13. QR-only tickets are common; we also support typed entry for robustness.
14. Teleoperation alone is not enough — passengers need guided autonomy between points.
15. Full humanoid arms are unnecessary for wayfinding; mobility and UI matter more.
16. Relative to Sanbot, we trade social animation for transparent navigation engineering.
17. A mapless reactive robot can avoid people but cannot promise a gate destination.
18. Semantic destinations beat anonymous ‘go forward’ behaviors for airport tasks.
19. Our cost target is prototype-class, not airline-procurement-class.
20. We compare ourselves on task fit: ticket → info → escort, not on humanoid expressiveness.
21. Staff escorts are flexible but do not scale; robots scale for repetitive guidance.
22. Digital signage helps everyone a little; a robot helps the confused passenger a lot.
23. Wheelchair assistance services are essential; AeroPorter complements, not replaces, PRM services.
24. SLAM toolboxes are mature; we chose a custom occupancy approach we can fully defend.
25. ROS 2 gives us standard messages without forcing the entire Nav2 behavior tree.
26. Browser BarcodeDetector removes the need for a dedicated hardware scanner in demos.
27. A USB camera is enough for face centering; Tapo PTZ is an optional upgrade path.
28. Private LAN control is safer than exposing motion APIs on public airport Wi-Fi.
29. Fail-closed escort is better than a confident wrong destination.
30. Three trusted encoders beat four noisy encoders when one channel is unstable.
31. A remembered map is faster to start than mapping from scratch every demo.
32. Operator supervision is a feature for airports, not a weakness of the prototype.
33. Against pure teleop toys, we add mapping, planning, and passenger workflow.
34. Against research-only SLAM demos, we add a concrete airport user story.
35. Our advantage is vertical integration: hardware, firmware, navigation, and kiosk UX.
36. The trade-off is clear: less industrial certification, more explainable student engineering.
37. We optimize for explainability per ruble, not for marketing spectacle.

2.4 Describing advantages (36)

1. The main advantage is end-to-end passenger assistance on one mobile platform.
2. Mecanum wheels improve maneuverability in crowded corridors.
3. Multilingual UI reduces language barriers immediately.
4. Ticket context makes guidance personal, not generic.
5. Safety timeouts stop the robot if communication fails.
6. Modular architecture lets us develop camera, drive, and kiosk in parallel.
7. Custom navigation means we can explain every block at defense.
8. Local processing keeps basic demos independent of airport cloud APIs.
9. Fail-closed destinations prevent dangerous wrong goals.

10. The operator console supports supervised mapping and recovery.
11. Affordable hardware makes the prototype reproducible for education.
12. Heartbeat leasing ties motion permission to a live kiosk session.
13. Manual barcode entry keeps the flow alive when camera scanning fails.
14. Saved maps reduce setup time between demonstrations.
15. Clear separation of ROS messages keeps interfaces standard where useful.
16. The design targets real airport pain points, not a random robot demo.
17. Escort can be disabled while info features still work — graceful degradation.
18. Open, readable Python code is easier to audit than a sealed vendor stack.
19. Arduino-level timeouts provide a hardware-near safety layer.
20. Digital face centering improves interaction without complex mechanics.
21. Four-language support includes Tatar, which matters regionally.
22. The footprint is suitable for indoor pedestrian spaces.
23. Staff can intervene via teleoperation when needed.
24. The project documents limitations honestly, which builds trust.
25. Calibration gates protect against unmeasured destination coordinates.
26. Dense LiDAR structure helps in GPS-denied indoor halls.
27. The kiosk UX is touch-oriented for a 10.1-inch panel.
28. Demo tickets allow repeatable laboratory evaluation.
29. The stack runs on ROS 2 Jazzy, a current middleware generation.
30. Independent folders reduce accidental coupling during development.
31. Safety is layered: MCU timeout, drive watchdog, and kiosk lease.
32. Passengers see flight data without receiving raw map coordinates.
33. The system is designed to be shown live, not only on slides.
34. Engineering choices are justified by constraints of school project resources.
35. We can measure and improve each stage: scan, plan, track, stop.
36. Layered watchdogs are a practical advantage you can test in seconds.

2.5 Admitting limitations (41)

1. We do not claim certified operation in a live airport terminal yet.
2. Destination escort requires measured coordinates; templates are intentionally invalid.
3. The front-right encoder is unreliable, so odometry uses three wheels.
4. LiDAR performance can degrade on glass and dark absorbing surfaces.
5. Crowded dynamic scenes still need more field validation.
6. Ticket databases may fall back to demo codes in the lab.
7. BarcodeDetector depends on browser support; manual entry is the backup.
8. We are not using Nav2, AMCL, or Cartographer as a full industrial stack.
9. Protective bodywork and public-space indication are still required before exploitation.
10. The base cost estimate excludes LiDAR and industrial enclosure.
11. Floor changes may require drive recalibration.
12. Face tracking and ticket scanning are related UX features, not a full biometric boarding system.
13. Tapo PTZ is a separate prototype path, not the default USB stack.
14. Private LAN allowlisting is not the same as a full zero-trust airport network design.
15. Long-term localization drift can grow without careful mapping and scan matching.
16. We have not completed an official airport authority pilot approval process.
17. Battery endurance and thermal limits need more soak testing for full-shift operation.
18. Legal and insurance constraints for public robots are outside our current software scope.
19. The system is prototype-grade, not production-ready.
20. Semantic airport live feeds are not integrated as a claimed finished feature.

21. Obstacle handling is improved by temporary layers, but dense crowds remain hard.
22. We should not oversell SLAM maturity beyond occupancy mapping plus scan matching.
23. Audio announcements are not the core proven feature of the current kiosk path.
24. Real passenger personal data must be handled under strict privacy rules — our demo avoids that risk.
25. Wireless interference in terminals can affect cameras and Wi-Fi peripherals.
26. Map quality depends on careful mapping runs and environment structure.
27. Emergency evacuation procedures must be defined with the airport, not only by the robot team.
28. One prototype cannot cover an entire multi-terminal campus.
29. Staff training and SOPs are required for any pilot.
30. We accept residual risk and mitigate with supervised operation.
31. Some presentation claims from older versions were tightened to match the repository.
32. Industrial safety certification (relevant ISO standards) is future work.
33. The robot should not enter airside zones without explicit airport process and permits.
34. False confidence is more dangerous than an honest 'not yet'.
35. Our current tests are laboratory and controlled-space oriented.
36. If LiDAR is occluded by luggage carts, perception quality drops.
37. Software completeness is ahead of deployment completeness — and we say so.
38. Analog comparisons are about task fit, not claiming we beat commercial fleets overall.
39. The FR encoder issue is documented rather than hidden.
40. Uncalibrated destinations correctly disable escort — that is a limitation and a safety feature.
41. Saying 'not yet' is part of professional engineering communication.

2.6 Answering difficult questions (41)

1. That is a fair question. Here is how we approached it in engineering terms.
2. Let me separate what works today from what is planned.
3. The short answer is X; the deeper reason is Y.
4. We chose this option because of cost, explainability, and team skills.
5. We considered alternatives, and this was the best trade-off for our constraints.
6. I cannot claim production metrics we have not measured.
7. What we can show is a controlled demonstration of A→B navigation and kiosk flow.
8. If that subsystem fails, the safety layer stops motion.
9. We treat public deployment as a separate validation stage.
10. Our architecture decision was conscious, not accidental.
11. I would rather under-claim and demonstrate than over-claim and invent.
12. Please see the safety timeouts: they are concrete, not slogans.
13. The encoder limitation is known and mitigated in firmware.
14. We are happy to walk through the code path from kiosk button to motor PWM.
15. Compared with analogs, our edge is ticket-to-route focus on affordable hardware.
16. Legal constraints would be handled with the airport authority in a pilot agreement.
17. For crowds, we currently rely on LiDAR hits, inflation, and supervised speeds.
18. Loss of connection triggers stop behavior via watchdog and lease logic.
19. Raspberry Pi was sufficient for our current perception and planning load.
20. We did not use Nav2 because we needed a stack we could fully understand and modify.
21. If LiDAR fails, we must stop autonomous escort and fall back to safe halt/teleop policy.
22. Accuracy should be discussed with measurement method, not a single magic number.
23. Yes, glass is a LiDAR challenge; that is why field tests matter.
24. Personal data in a real deployment would require minimization and access control.
25. The cost figure is a prototype BOM estimate, not a turnkey airport contract price.
26. I can show the failure mode first, then the mitigation.

27. That criticism is valid for production; for a prototype we scoped MVP differently.
28. We document unknowns instead of inventing brand names or certifications.
29. The next experiment we would run is specifically about that risk.
30. Our confidence is high for lab demo, medium for busy terminal pilots, low for unsupervised airside claims — and that ordering is intentional.
31. If you want depth, I can explain the pure-pursuit law or the occupancy update.
32. If you want product view, I can explain the passenger journey.
33. We distinguish 'software exists' from 'airport signed off'.
34. The allowlist reduces remote misuse risk on the LAN; it is not a full security certification.
35. A strong answer includes: decision, alternative, trade-off, and residual risk.
36. I will answer in 20 seconds first, then expand if you want detail.
37. The team can split: hardware, navigation, and interface — ask any layer.
38. We stress-tested stop behavior because motion safety is the first requirement.
39. That edge case is why fail-closed defaults exist.
40. Thank you — that question helps us show the depth of the design.
41. If I need a second, I will structure the answer as decision → trade-off → mitigation.

2.7 Closing an answer (36)

1. That is the core of our approach.
2. I can go deeper into the implementation if you wish.
3. This closes the loop from requirement to mechanism.
4. In one sentence: safety first, then guidance features.
5. That is why we defend this architecture.
6. Happy to demonstrate it on the robot after the questions.
7. Those are the measured facts we are willing to stand behind.
8. Everything beyond that is roadmap, not current claim.
9. Thank you for the question — it highlights a key engineering trade-off.
10. That completes my answer on this point.
11. We can continue with hardware, software, or safety — your choice.
12. This is consistent with what the repository and tests show.
13. I will stop here to keep the answer within time.
14. The residual risks are acknowledged and listed in our limitations.
15. That is how AeroPorter behaves today.
16. Future work is clear, and it does not rewrite today's results.
17. We prefer this honest boundary over marketing language.
18. This answer connects user value to a concrete algorithm.
19. That mitigation is implemented, not only planned.
20. I hope that clarifies our design rationale.
21. To summarize: choice, reason, limitation, next step.
22. We validated the critical path: command → motion → stop.
23. Any further detail can be shown in logs or a live run.
24. That is our position as an engineering team.
25. I will pass to my teammate for the electronics specifics.
26. I will pass to my teammate for the navigation specifics.
27. I will pass to my teammate for the interface specifics.
28. Thank you — we can proceed to the next question.
29. This matches the slide and the code.
30. That is the shortest accurate answer.
31. We are ready for a follow-up on measurement method.
32. We are ready for a follow-up on failure modes.

33. We are ready for a follow-up on cost assumptions.
34. And that is why the prototype is defensible.
35. End of answer.
36. That is all for this question.

2.8 Presentation transitions (36)

1. Let me move from the problem to our solution.
2. Now that the idea is clear, here is the architecture.
3. Next, I will explain how navigation works in practice.
4. With the hardware in mind, let's look at the software stack.
5. This brings us to the passenger interface.
6. After the interface, we will discuss safety.
7. Having covered safety, let's review current results and limits.
8. Before questions, I will summarize our contribution.
9. Turning to analogs, here is how we differ.
10. Let's connect the ticket flow to the motion stack.
11. I will now show what is implemented versus planned.
12. Next slide: the internals of the prototype.
13. From mechanics we go to electronics.
14. From electronics we go to algorithms.
15. From algorithms we go to human factors.
16. This leads directly to the risk discussion.
17. Keeping time in mind, I will highlight only the critical path.
18. Please focus on this block — it is the heart of AeroPorter.
19. We will return to limitations after the demo story.
20. Now a concrete example of A to B escort.
21. Switching to economics for a minute.
22. Back to engineering details.
23. This slide answers the 'how' behind the previous 'what'.
24. Before we continue, one honesty note on deployment status.
25. Let's zoom into the LiDAR pipeline.
26. Let's zoom into the kiosk pipeline.
27. I will bridge to my teammate's part here.
28. As we close the technical section, here is the takeaway.
29. Finally, the roadmap — without rewriting present claims.
30. Thank you. We are ready for your questions.
31. This transition marks the shift from idea to implementation.
32. Here we move from implementation to evaluation.
33. Evaluation leads to future work.
34. Future work leads to our ask: a supervised pilot zone.
35. And now the conclusion.
36. Let's proceed.

Phrases total: 319

3. Jury Questions

3.1 General (30)

- 3.1-1 EN: What is AeroPorter in one sentence? RU: Что такое AeroPorter одним предложением? What they want: Clear value proposition
- 3.1-2 EN: Who is on the team and what are your roles? RU: Кто в команде и какие роли? What they want: Ownership and contribution
- 3.1-3 EN: What problem are you solving? RU: Какую проблему вы решаете? What they want: Problem framing
- 3.1-4 EN: Who is the target user? RU: Кто целевой пользователь? What they want: User focus
- 3.1-5 EN: What makes this an engineering project, not just a craft build? RU: Почему это инженерия, а не только поделка? What they want: Engineering depth
- 3.1-6 EN: What is your main contribution? RU: В чём ваш главный вклад? What they want: Novelty / work share
- 3.1-7 EN: How long did development take? RU: Сколько длилась разработка? What they want: Process realism
- 3.1-8 EN: What was the hardest part? RU: Что было самым сложным? What they want: Reflection
- 3.1-9 EN: What would you do differently? RU: Что бы вы сделали иначе? What they want: Learning ability
- 3.1-10 EN: How do you measure success? RU: Как вы измеряете успех? What they want: Metrics mindset
- 3.1-11 EN: Is this a product or a prototype? RU: Это продукт или прототип? What they want: Honesty of status
- 3.1-12 EN: Where can it be deployed first? RU: Где можно внедрить сначала? What they want: Pilot thinking
- 3.1-13 EN: What are the key risks? RU: Какие ключевые риски? What they want: Risk awareness
- 3.1-14 EN: How mature is the software? RU: Насколько зрелое ПО? What they want: TRL honesty
- 3.1-15 EN: What dependencies do you rely on? RU: От каких зависимостей вы зависите? What they want: Stack awareness
- 3.1-16 EN: How do you demo the system? RU: Как вы демонстрируете систему? What they want: Demo plan
- 3.1-17 EN: What is out of scope? RU: Что вне рамок проекта? What they want: Scoping
- 3.1-18 EN: How did you divide work? RU: Как делили работу? What they want: Team process
- 3.1-19 EN: What feedback did you get during development? RU: Какую обратную связь получали? What they want: Iteration
- 3.1-20 EN: Why airports, not shopping malls? RU: Почему аэропорты, а не ТПЦ? What they want: Domain choice
- 3.1-21 EN: What is the passenger journey? RU: Какой путь пассажира? What they want: UX story
- 3.1-22 EN: How is this related to your school curriculum? RU: Как это связано с учёбой? What they want: Educational fit
- 3.1-23 EN: What ethical issues do you see? RU: Какие этические вопросы видите? What they want: Ethics
- 3.1-24 EN: How do you handle personal data? RU: Как обрабатываете персональные данные? What they want: Privacy
- 3.1-25 EN: What happens if the robot blocks a corridor? RU: Что если робот перекроет проход? What they want: Operational sense
- 3.1-26 EN: Who supervises the robot? RU: Кто контролирует робота? What they want: Human oversight
- 3.1-27 EN: What is your elevator pitch? RU: Какой у вас elevator pitch? What they want: Communication
- 3.1-28 EN: Which slide is the most important and why? RU: Какой слайд главный и почему? What they want: Prioritization
- 3.1-29 EN: What evidence supports your claims? RU: Какие доказательства ваших утверждений? What they want: Evidence
- 3.1-30 EN: What is the next milestone after defense? RU: Какой следующий этап после защиты? What they want: Roadmap

3.2 Project idea (20)

- 3.2-1 EN: Why did you choose ticket scanning as the entry point? RU: Почему входная точка — сканирование билета? What they want: Product logic

- 3.2-2 EN: Why escort instead of only showing a map? RU: Почему сопровождение, а не только карта? What they want: Differentiation
- 3.2-3 EN: How do you know passengers need this? RU: Откуда знаете, что пассажирам это нужно? What they want: Problem validation
- 3.2-4 EN: What alternatives did you reject? RU: Какие альтернативы отвергли? What they want: Design space
- 3.2-5 EN: How is AeroPorter different from Google Maps indoors? RU: Чем отличается от indoor-карт на телефоне? What they want: Comparison
- 3.2-6 EN: Why multilingual support matters here? RU: Почему важен мультиязычный интерфейс? What they want: Inclusion
- 3.2-7 EN: How do gate changes affect your concept? RU: Как смена выхода влияет на концепцию? What they want: Dynamic ops
- 3.2-8 EN: Can the robot help with baggage claim too? RU: Может ли помогать с выдачей багажа? What they want: Scope extension
- 3.2-9 EN: Is voice interaction required? RU: Нужен ли голосовой интерфейс? What they want: Feature priority
- 3.2-10 EN: How do you avoid replacing human staff badly? RU: Как не подменять персонал во вред? What they want: Human factors
- 3.2-11 EN: What is the minimum viable journey? RU: Какой минимальный полезный сценарий? What they want: MVP
- 3.2-12 EN: How do you onboard a first-time user in 10 seconds? RU: Как новичок разберётся за 10 секунд? What they want: UX clarity
- 3.2-13 EN: Why four languages including Tatar? RU: Почему четыре языка, включая татарский? What they want: Local context
- 3.2-14 EN: What does 'fail gracefully' mean for the idea? RU: Что значит мягкая деградация для идеи? What they want: Resilience product-wise
- 3.2-15 EN: How would airlines benefit? RU: Какую выгоду получают авиакомпании? What they want: Stakeholder value
- 3.2-16 EN: How would the airport benefit? RU: Какую выгоду получает аэропорт? What they want: Stakeholder value
- 3.2-17 EN: What is the emotional goal for the passenger? RU: Какая эмоциональная цель для пассажира? What they want: UX outcome
- 3.2-18 EN: Why mobile robot not only a sticky QR poster? RU: Почему мобильный робот, а не QR-плакат? What they want: Justification
- 3.2-19 EN: How do you prevent the idea from becoming too broad? RU: Как не расползтись по scope? What they want: Focus
- 3.2-20 EN: What assumption, if wrong, kills the idea? RU: Какое ложное допущение убивает идею? What they want: Critical assumption

3.3 Hardware (35)

- 3.3-1 EN: Why Raspberry Pi 5? RU: Почему Raspberry Pi 5? What they want: Compute choice
- 3.3-2 EN: Why Arduino Mega 2560? RU: Почему Arduino Mega 2560? What they want: MCU choice
- 3.3-3 EN: Why not control motors directly from the Pi? RU: Почему моторы не с Pi напрямую? What they want: Architecture
- 3.3-4 EN: Explain your motor driver approach. RU: Объясните подход к драйверам моторов. What they want: Electronics
- 3.3-5 EN: Why mecanum wheels? RU: Почему механум-колёса? What they want: Drive choice
- 3.3-6 EN: What is your wheel diameter and why it matters? RU: Какой диаметр колёс и почему важен? What they want: Odometry params
- 3.3-7 EN: How many encoders and which are trusted? RU: Сколько энкодеров и какие доверенные? What they want: Honest limitation
- 3.3-8 EN: What is the robot footprint? RU: Какие габариты робота? What they want: Physical fit
- 3.3-9 EN: What LiDAR driver do you use? RU: Какой драйвер лидара? What they want: No fake brand
- 3.3-10 EN: How is the LiDAR connected? RU: Как подключён лидар? What they want: Interfaces

3.3-11 EN: What camera do you use by default? RU: Какая камера по умолчанию? What they want: Vision HW

3.3-12 EN: What is the Tapo C200 role? RU: Какова роль Tapo C200? What they want: Optional path

3.3-13 EN: What screen size and resolution? RU: Какой экран и разрешение? What they want: UI HW

3.3-14 EN: How do you power the system? RU: Как питаете систему? What they want: Power

3.3-15 EN: How do you protect against brownouts? RU: Как защищаетесь от просадок? What they want: Power integrity

3.3-16 EN: What serial baud rates are used? RU: Какие скорости UART? What they want: Comms

3.3-17 EN: How are USB devices assigned on the Pi? RU: Как назначаются USB-устройства на Pi? What they want: Ops reality

3.3-18 EN: What thermal issues did you see? RU: Какие тепловые проблемы замечали? What they want: Reliability

3.3-19 EN: How do you mount the LiDAR to avoid self-hits? RU: Как крепите лидар против самопопаданий? What they want: Integration

3.3-20 EN: What fasteners and frame materials did you use? RU: Какие крепёж и материалы рамы? What they want: Mechanics

3.3-21 EN: How do you manage cables? RU: Как укладываете кабели? What they want: Practice

3.3-22 EN: What is the payload capacity conceptually? RU: Какова грузоподъёмность концептуально? What they want: Limits

3.3-23 EN: Can the robot climb thresholds? RU: Берёт ли пороги? What they want: Terrain

3.3-24 EN: How loud are the motors? RU: Насколько шумные моторы? What they want: Human factors

3.3-25 EN: What emergency stop hardware exists? RU: Какой аппаратный e-stop есть? What they want: Safety HW

3.3-26 EN: Why 20 ticks per revolution matters? RU: Почему важны 20 тиков на оборот? What they want: Resolution

3.3-27 EN: How do you flash the Mega? RU: Как прошиваете Mega? What they want: Toolchain

3.3-28 EN: What happens if a motor driver fails? RU: Что при отказе драйвера? What they want: Failure mode

3.3-29 EN: How do you detect a disconnected encoder? RU: Как детектируете отвал энкодера? What they want: Diagnostics

3.3-30 EN: Why H-bridge style control? RU: Почему управление в стиле H-моста? What they want: Electronics

3.3-31 EN: What connectors are single points of failure? RU: Какие разъёмы — точки отказа? What they want: Reliability

3.3-32 EN: How do you ground the electronics? RU: Как делаете заземление/общую землю? What they want: EMI/ground

3.3-33 EN: Is the enclosure passenger-safe yet? RU: Корпус уже безопасен для пассажиров? What they want: Honesty

3.3-34 EN: What spare parts do you keep? RU: Какие запчасти держите? What they want: Maintainability

3.3-35 EN: How long to replace a wheel module? RU: Сколько менять колёсный модуль? What they want: Serviceability

3.4 Programming (35)

3.4-1 EN: What languages are used where? RU: Какие языки где используются? What they want: Stack map

3.4-2 EN: Why ROS 2 Jazzy? RU: Почему ROS 2 Jazzy? What they want: Middleware

3.4-3 EN: Which ROS topics are critical? RU: Какие ROS-топики критичны? What they want: Interfaces

3.4-4 EN: Why not Nav2? RU: Почему не Nav2? What they want: Architecture choice

3.4-5 EN: How does the HTTP API fit the robot? RU: Как HTTP API связан с роботом? What they want: Integration

3.4-6 EN: What does the process on port 8765 do? RU: Что делает процесс на порту 8765? What they want: Runtime

3.4-7 EN: How is the occupancy grid stored? RU: Как хранится occupancy grid? What they want: Data

3.4-8 EN: Explain A-star in your stack. RU: Объясните A* в вашем стеке. What they want: Algorithms

3.4-9 EN: Explain pure pursuit in your stack. RU: Объясните pure pursuit. What they want: Algorithms

3.4-10 EN: What is CSM and ICP used for? RU: Для чего CSM и ICP? What they want: Localization

3.4-11 EN: How do you prevent stale commands? RU: Как боретесь с устаревшими командами? What they want: Safety SW

3.4-12 EN: What is the kiosk heartbeat lease? RU: Что такое heartbeat lease киоска? What they want: Safety SW

3.4-13 EN: How does ticket lookup work? RU: Как работает поиск билета? What they want: Backend

3.4-14 EN: Where is the ticket database? RU: Где база билетов? What they want: Data path

3.4-15 EN: What statuses are accepted for tickets? RU: Какие статусы билетов принимаются? What they want: Business rules

3.4-16 EN: How do you keep map coordinates off the browser? RU: Как не отдаёте координаты карты в браузер? What they want: Security/privacy

3.4-17 EN: What is fail-closed destination logic? RU: Что такое fail-closed для точек? What they want: Safety logic

3.4-18 EN: How is the operator UI different from the kiosk? RU: Чем UI оператора отличается от киоска? What they want: Roles

3.4-19 EN: How do you log failures? RU: Как логируете сбои? What they want: Observability

3.4-20 EN: What tests do you run? RU: Какие тесты запускаете? What they want: QA

3.4-21 EN: How do you version configuration? RU: Как версионите конфиги? What they want: Ops

3.4-22 EN: What happens on Python exceptions in the bridge? RU: Что при исключениях Python в bridge? What they want: Robustness

3.4-23 EN: How do launch scripts start the stack? RU: Как launch-скрипты поднимают стек? What they want: Ops

3.4-24 EN: Why SQLite instead of a remote DB for demos? RU: Почему SQLite для демо? What they want: Trade-off

3.4-25 EN: How is BarcodeDetector integrated? RU: Как интегрирован BarcodeDetector? What they want: Frontend

3.4-26 EN: What fallback exists if scanning fails? RU: Какой fallback если скан не сработал? What they want: UX resilience

3.4-27 EN: How do you handle concurrent operator and kiosk commands? RU: Как решаете конфликт команд оператора и киоска? What they want: Concurrency

3.4-28 EN: What is in /tmp/robot_cmd.json? RU: Что в /tmp/robot_cmd.json? What they want: IPC

3.4-29 EN: How does drive_encoders.py talk to Arduino? RU: Как drive_encoders.py общается с Arduino? What they want: Bridge

3.4-30 EN: What message type is cmd_vel? RU: Какой тип сообщения cmd_vel? What they want: ROS literacy

3.4-31 EN: How do you update the map while moving? RU: Как обновляете карту на ходу? What they want: Mapping

3.4-32 EN: What is temporary obstacle TTL? RU: Что такое TTL временных препятствий? What they want: Local cost

3.4-33 EN: How do you unit-test planners? RU: Как модульно тестируете планировщик? What they want: Testing

3.4-34 EN: What is the biggest software risk? RU: Какой главный софтверный риск? What they want: Risk

3.4-35 EN: How do you roll back a bad config on the robot? RU: Как откатываете плохой конфиг на роботе? What they want: Ops

3.5 Robotics (30)

3.5-1 EN: Define localization versus mapping. RU: Чем локализация отличается от картирования? What they want: Concepts

3.5-2 EN: Is your system SLAM? RU: Это SLAM? What they want: Honest terminology

3.5-3 EN: How do you recover from localization loss? RU: Как восстанавливаетесь при потере локализации? What they want: Recovery

3.5-4 EN: What is cross-track error? RU: Что такое cross-track error? What they want: Control

3.5-5 EN: How do you choose look-ahead distance? RU: Как выбираете look-ahead? What they want: Tuning

3.5-6 EN: What frames do you use? RU: Какие системы координат? What they want: TF literacy

3.5-7 EN: How do you inflate obstacles? RU: Как раздуваете препятствия? What they want: Safety margins

3.5-8 EN: What is the planning resolution? RU: Какое разрешение планирования? What they want: Map params

3.5-9 EN: How do mecanum inverse kinematics work at a high level? RU: Как на высоком уровне работают кинематика механумов? What they want: Drive theory

3.5-10 EN: What is holonomic motion and why it matters? RU: Что такое голономность и зачем? What they want: Theory

3.5-11 EN: How do you stop for a person stepping in front? RU: Как останавливаетесь перед человеком? What they want: Dynamic obstacles

3.5-12 EN: What is yaw hold? RU: Что такое yaw hold? What they want: Stabilization

3.5-13 EN: How do you evaluate path quality? RU: Как оцениваете качество пути? What they want: Metrics

3.5-14 EN: What is dead reckoning error growth? RU: Как растёт ошибка счисления? What they want: Odometry

3.5-15 EN: When do you replan? RU: Когда перепланируете? What they want: Behavior

3.5-16 EN: How do you detect progress failure? RU: Как детектируете отсутствие прогресса? What they want: Watchdogs

3.5-17 EN: What is the difference between global and local planning here? RU: Чем глобальное и локальное планирование у вас? What they want: Architecture

3.5-18 EN: How does teleoperation override autonomy? RU: Как телеоп перехватывает автономию? What they want: Modes

3.5-19 EN: What social navigation rules do you approximate? RU: Какие правила social navigation приближаете? What they want: HRI

3.5-20 EN: How do you set max speeds near people? RU: Как задаёте макс. скорости рядом с людьми? What they want: Safety ops

3.5-21 EN: What map size do you target? RU: Какой размер карты целите? What they want: Scale

3.5-22 EN: How do you save and reload maps? RU: Как сохраняете и загружаете карты? What they want: Persistence

3.5-23 EN: What breaks navigation on carpet? RU: Что ломает навигацию на ковре? What they want: Floor dependence

3.5-24 EN: How do you handle narrow doors? RU: Как работаете с узкими дверями? What they want: Geometry

3.5-25 EN: What is footprint radius used for? RU: Для чего радиус габарита? What they want: Collision

3.5-26 EN: How do you filter robot self-returns? RU: Как фильтруете самопопадания? What they want: Perception

3.5-27 EN: What is an occupancy probability update? RU: Что такое обновление вероятности занятости? What they want: Mapping math

3.5-28 EN: Why 8-connected search? RU: Почему 8-связность? What they want: Planner detail

3.5-29 EN: How do you define goal tolerance? RU: Как задаёте допуск цели? What they want: Arrival

3.5-30 EN: What robotics KPI would you publish first? RU: Какой KPI робототехники опубликовали бы первым? What they want: Evaluation

3.6 Electronics (20)

3.6-1 EN: What is PWM doing in your drive? RU: Что делает ШИМ в приводе? What they want: Basics

3.6-2 EN: Why do motors need flyback protection conceptually? RU: Зачем защита от противо-ЭДС? What they want: Electronics literacy

3.6-3 EN: How do quadrature encoders give direction? RU: Как квадратурный энкодер даёт направление? What they want: Sensors

3.6-4 EN: What is a logic-level risk between Pi and Mega? RU: Какой риск уровней Pi и Mega? What they want: Integration

3.6-5 EN: How can EMI affect encoder counts? RU: Как ЭМП влияет на тики энкодера? What they want: Noise

3.6-6 EN: What is a brownout reset? RU: Что такое сброс от просадки? What they want: Power

3.6-7 EN: Why separate motor supply and logic supply conceptually? RU: Зачем разделять питание моторов и логики? What they want: Design

3.6-8 EN: What does a watchdog timer mean on MCU side? RU: Что значит watchdog на МК? What they want: Firmware safety

3.6-9 EN: How do you diagnose a bad USB-UART link? RU: Как диагностируете плохой USB-UART? What they want: Debug

3.6-10 EN: What current spikes occur at motor start? RU: Какие броски тока на старте моторов? What they want: Power

3.6-11 EN: How does baud rate mismatch fail? RU: Как проявляется неверный baudrate? What they want: Serial

- 3.6-12 EN: What is debounce and do you need it for encoders? RU: Что такое антидребезг и нужен ли энкодерам? What they want: Signals
- 3.6-13 EN: How do you protect against short circuits at a prototype level? RU: Как защищаетесь от КЗ на уровне прототипа? What they want: Safety
- 3.6-14 EN: What happens if LiDAR USB power is insufficient? RU: Что если питания USB лидару мало? What they want: Power budget
- 3.6-15 EN: Why heat sinks / airflow may matter on Pi? RU: Почему важен теплоотвод Pi? What they want: Thermal
- 3.6-16 EN: What is common ground and why it matters? RU: Что такое общая земля и зачем? What they want: Fundamentals
- 3.6-17 EN: How do you verify PWM polarity is correct? RU: Как проверяете полярность ШИМ/направления? What they want: Bring-up
- 3.6-18 EN: What electrical measurement would you show the jury? RU: Какое электрическое измерение показали бы жюри? What they want: Evidence
- 3.6-19 EN: How do connector failures present in software? RU: Как отказ разъёма выглядит в софте? What they want: Debug
- 3.6-20 EN: What is your power budget thinking? RU: Как думаете об энергобюджете? What they want: System design

3.7 Economics (20)

- 3.7-1 EN: What is the approximate prototype cost? RU: Какова примерная стоимость прототипа? What they want: BOM
- 3.7-2 EN: What does the cost exclude? RU: Что не входит в оценку? What they want: Honesty
- 3.7-3 EN: What is CAPEX vs OPEX for a pilot? RU: Что CAPEX и OPEX для пилота? What they want: Business literacy
- 3.7-4 EN: How would a pilot be priced for an airport? RU: Как оценить пилот для аэропорта? What they want: Pricing
- 3.7-5 EN: What is TCO over three years? RU: Какой TCO за 3 года? What they want: Ownership
- 3.7-6 EN: Where can costs drop in a next revision? RU: Где снизить стоимость в следующей версии? What they want: Optimization
- 3.7-7 EN: What is the most expensive subsystem? RU: Какая подсистема самая дорогая? What they want: BOM insight
- 3.7-8 EN: Is LiDAR cost justified? RU: Оправдан ли дорогой лидар? What they want: Trade-off
- 3.7-9 EN: How many robots per terminal zone? RU: Сколько роботов на зону терминала? What they want: Scale economics
- 3.7-10 EN: What staffing cost does it reduce? RU: Какие затраты на персонал снижает? What they want: Value
- 3.7-11 EN: What insurance costs might appear? RU: Какие страховые расходы возможны? What they want: Risk money
- 3.7-12 EN: How do you compare cost to Pepper-class robots? RU: Как сравниваете стоимость с классом Pepper? What they want: Market
- 3.7-13 EN: What is the payback narrative? RU: Какая история окупаемости? What they want: ROI story
- 3.7-14 EN: Who pays: airport, airline, or concession? RU: Кто платит: аэропорт, авиакомпания, концессия? What they want: Buyer
- 3.7-15 EN: What maintenance budget is needed? RU: Какой бюджет обслуживания нужен? What they want: OPEX
- 3.7-16 EN: Can open-source reduce cost? RU: Может ли open-source снизить стоимость? What they want: Strategy
- 3.7-17 EN: What is the cost of downtime? RU: Какова цена простоя? What they want: Ops economics
- 3.7-18 EN: How would you fund the next stage? RU: Как финансировать следующий этап? What they want: Fundraising literacy
- 3.7-19 EN: What cheap failure is still expensive operationally? RU: Какой дешёвый отказ дорог операционно? What they want: Risk
- 3.7-20 EN: Why is 'cheap prototype' not 'cheap airport product'? RU: Почему дешёвый прототип ≠ дешёвый продукт? What they want: Maturity

3.8 Safety (20)

- 3.8-1 EN: What stops the robot if the Pi freezes? RU: Что остановит робота если Pi завис? What they want: Layered safety
- 3.8-2 EN: Explain the Arduino command timeout. RU: Объясните таймаут команд Arduino. What they want: MCU safety
- 3.8-3 EN: Explain the drive watchdog. RU: Объясните drive watchdog. What they want: SW safety
- 3.8-4 EN: Explain the kiosk heartbeat lease. RU: Объясните heartbeat lease. What they want: Session safety
- 3.8-5 EN: What is fail-closed escort? RU: Что такое fail-closed escort? What they want: Config safety
- 3.8-6 EN: How do you prevent unauthorized remote control? RU: Как мешаете чужому удалённому управлению? What they want: Access control
- 3.8-7 EN: What should passengers see when the robot is active? RU: Что пассажиры должны видеть при активности? What they want: Indication
- 3.8-8 EN: How fast should it move near people? RU: С какой скоростью ехать рядом с людьми? What they want: Speed policy
- 3.8-9 EN: What is your emergency procedure? RU: Какова аварийная процедура? What they want: SOP
- 3.8-10 EN: How do you avoid toe injuries? RU: Как избегаете травм ног? What they want: Mechanical safety
- 3.8-11 EN: What about fire load / batteries? RU: Что с пожарной нагрузкой / АКБ? What they want: Battery safety
- 3.8-12 EN: How do you handle a crowded boarding queue? RU: Как вести себя в толпе у посадки? What they want: Ops safety
- 3.8-13 EN: What logs help after an incident? RU: Какие логи нужны после инцидента? What they want: Forensics
- 3.8-14 EN: Who can press stop? RU: Кто может нажать стоп? What they want: Authority
- 3.8-15 EN: How do you keep evacuation routes clear? RU: Как не блокировать эвакуационные пути? What they want: Airport rules
- 3.8-16 EN: What residual risks remain? RU: Какие остаточные риски остаются? What they want: Honesty
- 3.8-17 EN: How do you test stop latency? RU: Как измеряете задержку останова? What they want: Verification
- 3.8-18 EN: Is ISO certification claimed? RU: Заявляете ли ISO-сертификацию? What they want: Do not overclaim
- 3.8-19 EN: What soft coverings are still needed? RU: Какие мягкие покрытия ещё нужны? What they want: Mechanical mitigation
- 3.8-20 EN: How does UI disconnect stop motion? RU: Как отключение UI останавливает движение? What they want: UI safety

3.9 Real-world application (20)

- 3.9-1 EN: Where exactly would a pilot run? RU: Где именно идёт пилот? What they want: Deployment locus
- 3.9-2 EN: Landside or airside? RU: Landside или airside? What they want: Zone rules
- 3.9-3 EN: How do you integrate with airport Wi-Fi policy? RU: Как стыкуетесь с политикой Wi-Fi? What they want: IT policy
- 3.9-4 EN: How do staff learn to supervise it? RU: Как персонал учится супервизии? What they want: Training
- 3.9-5 EN: What KPI would the airport track? RU: Какой KPI смотрит аэропорт? What they want: Ops metrics
- 3.9-6 EN: How do you avoid conflicting with cleaning robots / carts? RU: Как не конфликтовать с тележками и клинингом? What they want: Shared space
- 3.9-7 EN: Can it work during peak waves? RU: Работает ли в пиковые волны? What they want: Capacity
- 3.9-8 EN: How do you update destinations when gates rebuild seasonally? RU: Как обновлять точки при сезонных перестройках? What they want: Change mgmt
- 3.9-9 EN: What languages matter in your local airport? RU: Какие языки важны в вашем аэропорту? What they want: Localization
- 3.9-10 EN: How do you support PRM passengers without false claims? RU: Как помогать PRM без ложных заявлений? What they want: Ethics/scope
- 3.9-11 EN: What is the handoff to a human agent? RU: Как передаёте пассажира человеку? What they want: Process

3.9-12 EN: How do announcements interact with robot guidance? RU: Как объявления стыкуются с наведением? What they want: Ops UX

3.9-13 EN: What data integration is required for live flights? RU: Какая интеграция данных нужна для живых рейсов? What they want: AODB/FIDS

3.9-14 EN: How do you handle a gate change mid-escort? RU: Что при смене выхода во время escort? What they want: Dynamic replan

3.9-15 EN: What physical branding is needed so people trust it? RU: Какой брендинг нужен для доверия? What they want: Adoption

3.9-16 EN: How do you measure passenger satisfaction? RU: Как мерить удовлетворённость? What they want: CX

3.9-17 EN: What time of day is best for trials? RU: Какое время лучше для испытаний? What they want: Pilot design

3.9-18 EN: How many supervisors per N robots? RU: Сколько супервизоров на N роботов? What they want: Ops model

3.9-19 EN: What facility constraints matter (elevators, glass)? RU: Какие ограничения здания важны? What they want: Environment

3.9-20 EN: What does success look like after 30 pilot days? RU: Как выглядит успех после 30 дней пилота? What they want: Definition of done

3.10 Future of the project (20)

3.10-1 EN: What is the next hardware revision? RU: Какая следующая аппаратная ревизия? What they want: Roadmap

3.10-2 EN: Will you migrate to Nav2 later? RU: Перейдёте ли на Nav2 позже? What they want: Stack evolution

3.10-3 EN: Do you plan Jetson? RU: Планируете ли Jetson? What they want: Compute roadmap

3.10-4 EN: How will live airport APIs plug in? RU: Как подключите живые API аэропорта? What they want: Integration

3.10-5 EN: What about voice guidance? RU: Что с голосовыми подсказками? What they want: UX future

3.10-6 EN: What about fleet mode? RU: Что с флотом? What they want: Scale

3.10-7 EN: How will you improve glass handling? RU: Как улучшите работу со стеклом? What they want: Perception

3.10-8 EN: What ML models might you add? RU: Какие ML-модели можете добавить? What they want: CV/ML

3.10-9 EN: How will you harden security? RU: Как ужесточите безопасность? What they want: Security roadmap

3.10-10 EN: What enclosure redesign is planned? RU: Какой редизайн корпуса? What they want: Industrial design

3.10-11 EN: How will you formalize safety cases? RU: Как формализуете safety case? What they want: Process maturity

3.10-12 EN: What tests are next? RU: Какие тесты следующие? What they want: V&V

3.10-13 EN: How could airlines white-label it? RU: Как авиакомпании сделают white-label? What they want: Business

3.10-14 EN: What open-sourcing plans exist? RU: Какие планы open-source? What they want: Community

3.10-15 EN: How will you track TRL growth? RU: Как отслеживать рост УГТ/TRL? What they want: Maturity

3.10-16 EN: What partnership do you need first? RU: Какое партнёрство нужно первым? What they want: Go-to-market

3.10-17 EN: How will maps stay updated automatically? RU: Как карты будут обновляться автоматически? What they want: Ops automation

3.10-18 EN: What battery tech upgrade matters? RU: Какой апгрейд АКБ важен? What they want: Endurance

3.10-19 EN: How do you plan accessibility audits? RU: Как планируете аудиты доступности? What they want: A11y

3.10-20 EN: What is the five-year vision without hype? RU: Какое видение на 5 лет без хайпа? What they want: Vision

Jury questions total: 250

4. Ready Answers

For each jury question: Short (~20–30s), Full (~1–2 min), Alternative.

3.1-1 — What is AeroPorter in one sentence?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-2 — Who is on the team and what are your roles?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-3 — What problem are you solving?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-4 — Who is the target user?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-5 — What makes this an engineering project, not just a craft build?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified

airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-6 — What is your main contribution?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-7 — How long did development take?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-8 — What was the hardest part?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-9 — What would you do differently?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-10 — How do you measure success?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-11 — Is this a product or a prototype?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-12 — Where can it be deployed first?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-13 — What are the key risks?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-14 — How mature is the software?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-15 — What dependencies do you rely on?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-16 — How do you demo the system?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-17 — What is out of scope?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-18 — How did you divide work?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-19 — What feedback did you get during development?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-20 — Why airports, not shopping malls?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-21 — What is the passenger journey?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-22 — How is this related to your school curriculum?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-23 — What ethical issues do you see?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are

calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-24 — How do you handle personal data?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-25 — What happens if the robot blocks a corridor?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-26 — Who supervises the robot?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-27 — What is your elevator pitch?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-28 — Which slide is the most important and why?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-29 — What evidence supports your claims?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.1-30 — What is the next milestone after defense?

Short (20–30 sec): AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype that scans a ticket, shows flight info, and can escort a passenger A→B using a custom ROS 2 LiDAR stack — as a supervised prototype, not a certified airport product.

Full (1–2 min): AeroPorter is an engineering prototype of a mobile airport assistant. The passenger can scan a boarding-pass code, see flight-related information on a 10.1-inch multilingual kiosk, and — when destinations are calibrated — start an escort to a goal on the occupancy map. The mobility stack is custom ROS 2 on a Raspberry Pi 5 with Arduino Mega motor control and LiDAR-based mapping. We are explicit: software for navigation exists; claiming routine work inside a live commercial terminal would be premature.

Alternative: Alternative framing: AeroPorter is a ticket-aware indoor escort robot for GPS-denied terminals, built as a transparent student-engineered stack.

3.2-1 — Why did you choose ticket scanning as the entry point?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-2 — Why escort instead of only showing a map?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-3 — How do you know passengers need this?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-4 — What alternatives did you reject?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-5 — How is AeroPorter different from Google Maps indoors?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-6 — Why multilingual support matters here?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-7 — How do gate changes affect your concept?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-8 — Can the robot help with baggage claim too?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-9 — Is voice interaction required?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-10 — How do you avoid replacing human staff badly?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-11 — What is the minimum viable journey?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-12 — How do you onboard a first-time user in 10 seconds?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-13 — Why four languages including Tatar?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-14 — What does 'fail gracefully' mean for the idea?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-15 — How would airlines benefit?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-16 — How would the airport benefit?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-17 — What is the emotional goal for the passenger?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-18 — Why mobile robot not only a sticky QR poster?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-19 — How do you prevent the idea from becoming too broad?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.2-20 — What assumption, if wrong, kills the idea?

Short (20–30 sec): We start from the ticket because it gives a concrete destination; escort beats a static map for stressed or elderly passengers; phone apps don't help everyone.

Full (1–2 min): Airports already have signs, but passengers still get lost when gates change, languages differ, or stress is high. A fixed desk cannot walk with you. A phone app assumes digital confidence. AeroPorter's idea is a moving assistant that binds ticket context to a physical route. We keep the story narrow: information plus escort, not a humanoid entertainer.

Alternative: Alternative: we sell fewer lost-passenger minutes and fewer repetitive staff questions, not 'AI magic'.

3.3-1 — Why Raspberry Pi 5?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-2 — Why Arduino Mega 2560?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-3 — Why not control motors directly from the Pi?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-4 — Explain your motor driver approach.

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-5 — Why mecanum wheels?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-6 — What is your wheel diameter and why it matters?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum

wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-7 — How many encoders and which are trusted?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path. The front-right encoder is untrusted; odometry uses FL/RL/RR.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-8 — What is the robot footprint?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-9 — What LiDAR driver do you use?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-10 — How is the LiDAR connected?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-11 — What camera do you use by default?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-12 — What is the Tapo C200 role?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-13 — What screen size and resolution?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-14 — How do you power the system?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-15 — How do you protect against brownouts?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-16 — What serial baud rates are used?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-17 — How are USB devices assigned on the Pi?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-18 — What thermal issues did you see?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-19 — How do you mount the LiDAR to avoid self-hits?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-20 — What fasteners and frame materials did you use?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-21 — How do you manage cables?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-22 — What is the payload capacity conceptually?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum

wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-23 — Can the robot climb thresholds?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-24 — How loud are the motors?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-25 — What emergency stop hardware exists?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-26 — Why 20 ticks per revolution matters?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-27 — How do you flash the Mega?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-28 — What happens if a motor driver fails?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-29 — How do you detect a disconnected encoder?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path. The front-right encoder is untrusted; odometry uses FL/RL/RR.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-30 — Why H-bridge style control?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-31 — What connectors are single points of failure?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-32 — How do you ground the electronics?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-33 — Is the enclosure passenger-safe yet?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-34 — What spare parts do you keep?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.3-35 — How long to replace a wheel module?

Short (20–30 sec): Pi 5 runs Linux/ROS 2 and the kiosk stack; Mega handles PWM, encoders, and a 1.2 s command timeout; mecanum enables strafe; LiDAR uses the `cspc_lidar` driver — we don't invent a commercial model name.

Full (1–2 min): The Pi 5 is the high-level computer: ROS 2, HTTP UI, planning. The Mega is the real-time-ish actuator computer: mecanum mixing, encoder integration, PWM, and fail-safe timeout if serial commands stop. Four mecanum wheels give omnidirectional motion useful in crowds. Encoders are mounted on all four corners, but the front-right channel is untrusted, so pose uses three wheels. LiDAR arrives via the `cspc_lidar` ROS node on USB serial. Default vision is a USB camera; Tapo C200 RTSP/PTZ is a separate prototype path.

Alternative: Alternative one-liner: SBC for brains, MCU for motors, LiDAR for structure, touch UI for humans.

3.4-1 — What languages are used where?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): `lidar_map` is the runtime brain: it consumes `/scan` and `/odom`, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as `/cmd_vel` and/or `/tmp/robot_cmd.json` into `drive_encoders.py`, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — `arduino`, `lidar_map`, `camera`, `monitor` — composing one demo when launched together.

3.4-2 — Why ROS 2 Jazzy?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): `lidar_map` is the runtime brain: it consumes `/scan` and `/odom`, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as `/cmd_vel` and/or `/tmp/robot_cmd.json` into `drive_encoders.py`, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — `arduino`, `lidar_map`, `camera`, `monitor` — composing one demo when launched together.

3.4-3 — Which ROS topics are critical?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): `lidar_map` is the runtime brain: it consumes `/scan` and `/odom`, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as `/cmd_vel` and/or `/tmp/robot_cmd.json` into

drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-4 — Why not Nav2?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback. We consciously use a custom planner/tracker we can fully explain instead of Nav2.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-5 — How does the HTTP API fit the robot?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-6 — What does the process on port 8765 do?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-7 — How is the occupancy grid stored?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-8 — Explain A-star in your stack.

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-9 — Explain pure pursuit in your stack.

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-10 — What is CSM and ICP used for?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-11 — How do you prevent stale commands?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-12 — What is the kiosk heartbeat lease?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback. Without kiosk heartbeats for ~6 seconds, escort stops.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-13 — How does ticket lookup work?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-14 — Where is the ticket database?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-15 — What statuses are accepted for tickets?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-16 — How do you keep map coordinates off the browser?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-17 — What is fail-closed destination logic?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-18 — How is the operator UI different from the kiosk?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-19 — How do you log failures?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-20 — What tests do you run?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-21 — How do you version configuration?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-22 — What happens on Python exceptions in the bridge?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-23 — How do launch scripts start the stack?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-24 — Why SQLite instead of a remote DB for demos?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-25 — How is BarcodeDetector integrated?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-26 — What fallback exists if scanning fails?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-27 — How do you handle concurrent operator and kiosk commands?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-28 — What is in /tmp/robot_cmd.json?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-29 — How does drive_encoders.py talk to Arduino?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback. The front-right encoder is untrusted; odometry uses FL/RL/RR.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-30 — What message type is cmd_vel?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-31 — How do you update the map while moving?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-32 — What is temporary obstacle TTL?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-33 — How do you unit-test planners?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-34 — What is the biggest software risk?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.4-35 — How do you roll back a bad config on the robot?

Short (20–30 sec): Critical path: kiosk/operator HTTP on :8765 → velocity commands → Mega; planning is A* on an occupancy grid; tracking is pure pursuit; safety uses watchdogs and a heartbeat lease.

Full (1–2 min): lidar_map is the runtime brain: it consumes /scan and /odom, maintains occupancy, runs A* and pure pursuit, and serves operator/kiosk HTTP on port 8765. Commands go out as /cmd_vel and/or /tmp/robot_cmd.json into drive_encoders.py, which talks serial to the Mega. Tickets live in SQLite with fail-safe demo fallback. Destinations must be measured coordinates; invalid templates disable escort. Browser BarcodeDetector reads codes, with manual entry fallback.

Alternative: Alternative: think of four folders — arduino, lidar_map, camera, monitor — composing one demo when launched together.

3.5-1 — Define localization versus mapping.

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-2 — Is your system SLAM?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-3 — How do you recover from localization loss?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a

claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-4 — What is cross-track error?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-5 — How do you choose look-ahead distance?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-6 — What frames do you use?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-7 — How do you inflate obstacles?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-8 — What is the planning resolution?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-9 — How do mecanum inverse kinematics work at a high level?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a

claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-10 — What is holonomic motion and why it matters?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-11 — How do you stop for a person stepping in front?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-12 — What is yaw hold?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-13 — How do you evaluate path quality?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-14 — What is dead reckoning error growth?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-15 — When do you replan?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a

claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-16 — How do you detect progress failure?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-17 — What is the difference between global and local planning here?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-18 — How does teleoperation override autonomy?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-19 — What social navigation rules do you approximate?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-20 — How do you set max speeds near people?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-21 — What map size do you target?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a

claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-22 — How do you save and reload maps?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-23 — What breaks navigation on carpet?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-24 — How do you handle narrow doors?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-25 — What is footprint radius used for?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-26 — How do you filter robot self-returns?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-27 — What is an occupancy probability update?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a

claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-28 — Why 8-connected search?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-29 — How do you define goal tolerance?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.5-30 — What robotics KPI would you publish first?

Short (20–30 sec): We map with log-odds occupancy, localize with odometry plus CSM/ICP scan matching, plan with A*, track with pure pursuit — custom ROS 2, not Nav2.

Full (1–2 min): Mapping builds a log-odds grid; rays carve free space and mark hits. Localization combines wheel odometry with scan matching (CSM + ICP). Planning searches an 8-connected grid with A*. Tracking uses mecanum pure pursuit with cross-track correction and yaw terms. Temporary obstacle cells help with dynamic hits. This is not a claim of full industrial SLAM with perfect loop closure like a Cartographer+Nav2 deployment.

Alternative: Alternative: sense structure → update map/pose → plan → track → stop on fault.

3.6-1 — What is PWM doing in your drive?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-2 — Why do motors need flyback protection conceptually?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-3 — How do quadrature encoders give direction?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as

missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling. The front-right encoder is untrusted; odometry uses FL/RL/RR.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-4 — What is a logic-level risk between Pi and Mega?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-5 — How can EMI affect encoder counts?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling. The front-right encoder is untrusted; odometry uses FL/RL/RR.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-6 — What is a brownout reset?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-7 — Why separate motor supply and logic supply conceptually?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-8 — What does a watchdog timer mean on MCU side?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-9 — How do you diagnose a bad USB-UART link?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing `/dev/tty` devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-10 — What current spikes occur at motor start?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing `/dev/tty` devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-11 — How does baud rate mismatch fail?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing `/dev/tty` devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-12 — What is debounce and do you need it for encoders?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing `/dev/tty` devices. Firmware can be flashed with avrdude-class tooling. The front-right encoder is untrusted; odometry uses FL/RL/RR.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-13 — How do you protect against short circuits at a prototype level?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing `/dev/tty` devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-14 — What happens if LiDAR USB power is insufficient?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing `/dev/tty` devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-15 — Why heat sinks / airflow may matter on Pi?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-16 — What is common ground and why it matters?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-17 — How do you verify PWM polarity is correct?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-18 — What electrical measurement would you show the jury?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-19 — How do connector failures present in software?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.6-20 — What is your power budget thinking?

Short (20–30 sec): PWM + H-bridge style drivers move wheels; quadrature encoders feed odometry; shared grounding and clean power matter; brownouts and EMI are real failure modes we design around.

Full (1–2 min): Motor channels use PWM and direction pins into H-bridge-style drivers. Encoders provide A/B edges; tick rate control uses PI on trusted wheels. Serial links must share a coherent ground reference. Motor current spikes can brown out weak supplies, so power architecture matters as much as code. USB power/enumeration issues show up as missing /dev/tty devices. Firmware can be flashed with avrdude-class tooling.

Alternative: Alternative: if you can't explain the current path from battery to PWM, you don't understand the robot yet.

3.7-1 — What is the approximate prototype cost?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-2 — What does the cost exclude?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-3 — What is CAPEX vs OPEX for a pilot?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-4 — How would a pilot be priced for an airport?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-5 — What is TCO over three years?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-6 — Where can costs drop in a next revision?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-7 — What is the most expensive subsystem?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-8 — Is LiDAR cost justified?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-9 — How many robots per terminal zone?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-10 — What staffing cost does it reduce?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-11 — What insurance costs might appear?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-12 — How do you compare cost to Pepper-class robots?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-13 — What is the payback narrative?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-14 — Who pays: airport, airline, or concession?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-15 — What maintenance budget is needed?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-16 — Can open-source reduce cost?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-17 — What is the cost of downtime?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one. Treat ~18,000 □ as prototype BOM without LiDAR/enclosure.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-18 — How would you fund the next stage?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-19 — What cheap failure is still expensive operationally?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.7-20 — Why is ‘cheap prototype’ not ‘cheap airport product’?

Short (20–30 sec): Prototype base cost is on the order of ~18,000 □ excluding LiDAR and industrial enclosure; airport product cost is a different category with certification, support, and insurance.

Full (1–2 min): Our published prototype estimate is roughly eighteen thousand rubles for a base build and explicitly excludes LiDAR and a hardened enclosure. That number is useful for education and iteration, not as a turnkey airport procurement quote. A real pilot adds supervision labor, insurance, spare parts, networking, and compliance work — OPEX dominates after month one.

Alternative: Alternative: optimize learning per ruble now; optimize certification per ruble later.

3.8-1 — What stops the robot if the Pi freezes?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-2 — Explain the Arduino command timeout.

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules. Mega stops after ~1.2 s without serial commands.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-3 — Explain the drive watchdog.

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-4 — Explain the kiosk heartbeat lease.

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules. Without kiosk heartbeats for ~6 seconds, escort stops.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-5 — What is fail-closed escort?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-6 — How do you prevent unauthorized remote control?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-7 — What should passengers see when the robot is active?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-8 — How fast should it move near people?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-9 — What is your emergency procedure?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-10 — How do you avoid toe injuries?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-11 — What about fire load / batteries?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-12 — How do you handle a crowded boarding queue?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-13 — What logs help after an incident?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-14 — Who can press stop?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-15 — How do you keep evacuation routes clear?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-16 — What residual risks remain?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-17 — How do you test stop latency?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-18 — Is ISO certification claimed?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-19 — What soft coverings are still needed?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.8-20 — How does UI disconnect stop motion?

Short (20–30 sec): Layered stops: Mega timeout ~1.2 s, drive watchdog ~0.25 s, kiosk heartbeat ~6 s; escort is fail-closed without calibrated destinations; indication and soft covers are still needed for public use.

Full (1–2 min): Safety is layered on purpose. If high-level software dies, the Mega times out and stops. If commands go stale on the Pi side, the drive watchdog zeroes motion. If the kiosk page stops heartbeating during escort, the lease ends and the robot stops. Escort refuses uncalibrated goals. Access to motion APIs is gated. Still missing for public rollout: industrial indication, soft covers, formal SOPs, and authority-approved operating rules.

Alternative: Alternative: motion is a privilege granted by live leases and fresh commands, not a default.

3.9-1 — Where exactly would a pilot run?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-2 — Landside or airside?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-3 — How do you integrate with airport Wi-Fi policy?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-4 — How do staff learn to supervise it?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-5 — What KPI would the airport track?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-6 — How do you avoid conflicting with cleaning robots / carts?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-7 — Can it work during peak waves?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-8 — How do you update destinations when gates rebuild seasonally?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-9 — What languages matter in your local airport?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-10 — How do you support PRM passengers without false claims?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-11 — What is the handoff to a human agent?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-12 — How do announcements interact with robot guidance?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-13 — What data integration is required for live flights?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-14 — How do you handle a gate change mid-escort?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-15 — What physical branding is needed so people trust it?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-16 — How do you measure passenger satisfaction?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-17 — What time of day is best for trials?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-18 — How many supervisors per N robots?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-19 — What facility constraints matter (elevators, glass)?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages. Glass and dark surfaces are known LiDAR weaknesses; we admit them and plan field checks.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.9-20 — What does success look like after 30 pilot days?

Short (20–30 sec): First pilot should be a supervised landside/terminal zone with trained staff, calibrated destinations, and clear stop authority — not unsupervised airside operation.

Full (1–2 min): A credible first deployment is a supervised pilot in a defined terminal zone: calibrated check-in/gate/info points, staff with stop authority, offline-capable demo data or a controlled ticket source, and no airside claims. Success metrics could include escort completion rate, stop latency, passenger task time, and intervention count — not vanity autonomy percentages.

Alternative: Alternative: if the airport won't assign a supervisor and a zone, it is not a pilot — it is a wish.

3.10-1 — What is the next hardware revision?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-2 — Will you migrate to Nav2 later?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early. We consciously use a custom planner/tracker we can fully explain instead of Nav2.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-3 — Do you plan Jetson?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early. Pi 5 is enough for our current CPU workloads; Jetson is a future option if neural load grows.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-4 — How will live airport APIs plug in?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-5 — What about voice guidance?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-6 — What about fleet mode?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-7 — How will you improve glass handling?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early. Glass and dark surfaces are known LiDAR weaknesses; we admit them and plan field checks.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-8 — What ML models might you add?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-9 — How will you harden security?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-10 — What enclosure redesign is planned?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-11 — How will you formalize safety cases?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-12 — What tests are next?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-13 — How could airlines white-label it?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-14 — What open-sourcing plans exist?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-15 — How will you track TRL growth?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-16 — What partnership do you need first?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-17 — How will maps stay updated automatically?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-18 — What battery tech upgrade matters?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-19 — How do you plan accessibility audits?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2 are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

3.10-20 — What is the five-year vision without hype?

Short (20–30 sec): Next: industrial enclosure, richer field tests, possible live flight data integration, security hardening, and only then any discussion of fleet scale.

Full (1–2 min): The roadmap is enclosure and human-factors first, then denser field validation in crowd-like conditions, then selective stack upgrades (more perception redundancy, better security, optional live flight feeds). Jetson or Nav2

are options if workloads or maintainability demand them — not trophies to claim early.

Alternative: Alternative: maturity is earned by tests and SOPs, not by adding buzzwords to slides.

Ready answer sets: 250 (×3 variants each)

5. Tricky Questions (depth checks)

T1

EN: Why this architecture instead of a single SBC controlling everything? RU: Почему такая архитектура, а не один SBC на всё? Strong answer: Pi for high-level autonomy/UI, Mega for motor-level fail-safe timeouts and encoder loops. Splitting responsibilities adds a safety layer: if Linux hangs, Mega still stops after 1.2 s.

T2

EN: Why not Nav2? RU: Почему не Nav2? Strong answer: Nav2 is excellent, but heavy to tune and easy to treat as a black box. We needed a stack the whole team can explain: occupancy, A*, pure pursuit, CSM/ICP. That is a pedagogical and technical choice, not ignorance of Nav2.

T3

EN: Why Raspberry Pi rather than NVIDIA Jetson? RU: Почему Raspberry Pi, а не Jetson? Strong answer: Our current workloads — planning, HTTP, YuNet-class face crop — fit Pi 5. Jetson helps heavy neural pipelines. Buying unused GPU budget would raise cost without matching today's claims.

T4

EN: What if LiDAR fails mid-route? RU: Что если лидар откажет на маршруте? Strong answer: Autonomous escort must stop. Without trustworthy ranging, continuing is unsafe. Fall back to halt and operator teleop policy; do not 'guess' through a terminal crowd.

T5

EN: What if Wi-Fi or kiosk connection drops? RU: Что если пропал Wi-Fi или связь с киоском? Strong answer: Heartbeat lease expires (~6 s) and escort stops; drive watchdog and Mega timeout also cut motion if commands freeze. Loss of link fails safe to stop.

T6

EN: Why is your solution better than Pepper / Sanbot / Promobot? RU: Почему лучше Pepper/Sanbot/Promobot? Strong answer: Not 'better at everything'. Better fit for our goal: ticket→info→escort on affordable, explainable hardware. Humanoids win social presence; we win transparent navigation engineering and lower prototype cost.

T7

EN: What happens in a dense passenger wave? RU: Что будет при плотном потоке людей? Strong answer: Dynamic hits inflate temporary obstacles; speeds must stay conservative; supervisors intervene. We do not claim proven unsupervised peak-hour autonomy.

T8

EN: What legal limits appear in a real airport? RU: Какие юридические ограничения? Strong answer: Public operation needs airport permission, safety rules, insurance, data-protection constraints, and possibly zone restrictions (especially airside). Software readiness ≠ legal readiness.

T9

EN: Why should we trust a student-built navigator? RU: Почему доверять студенческой навигации? Strong answer: Trust the layered stops, calibrated fail-closed goals, supervised pilot design, and honest limits — not hype. Every critical block is inspectable in our repo.

T10

EN: Isn't this just teleoperation with a fancy UI? RU: Разве это не телеоп с красивым UI? Strong answer: Teleop exists for supervision, but mapping, A*, pursuit, and ticket-to-goal flows are autonomous building blocks. Teleop is a safety net, not the whole product.

T11

EN: You use ROS 2 — so why reinvent planners? RU: Раз есть ROS 2 — зачем свои планировщики? Strong answer: ROS 2 gives messages and tooling. Planners are still a choice. Owning A*/pursuit keeps behavior transparent for defense and customization for mecatronics.

T12

EN: Why mecanum if warehouses use differential drives? RU: Почему механум, если на складах часто дифференциал? Strong answer: Terminal corridors need sideways adjustment without wide turning arcs. Mecanum costs calibration complexity; we accept that trade for maneuverability.

T13

EN: How accurate is localization — really? RU: Насколько точна локализация — по-честному? Strong answer: Accurate enough for controlled demos when the map is good and scan matching holds; not a certified centimeter SLA. We report method and conditions, not a fake universal number.

T14

EN: What about glass walls in terminals? RU: А стеклянные стены терминала? Strong answer: Known LiDAR weakness. Mitigations are mapping with caution, speed limits, sensor diversity later, and human supervision in pilots.

T15

EN: Does face tracking mean biometric identification? RU: Трекинг лица = биометрия? Strong answer: No. We center a detected face for interaction UX. We do not claim biometric boarding identity matching.

T16

EN: Why publish ~18,000 □ if LiDAR is excluded? RU: Зачем ~18000□ если лидар не включён? Strong answer: To be transparent about BOM scope. Hiding exclusions would be worse. Jury should see what is included and what is not.

T17

EN: What if someone hacks the web UI on LAN? RU: Что если взломают web UI в LAN? Strong answer: Risk is real on flat networks. We gate control, prefer private networks, and still need stronger auth for production. We do not claim bulletproof security.

T18

EN: Why is the FR encoder untrusted — did you fail? RU: Почему FR энкодер недоверенный — вы провалились? Strong answer: We diagnosed instability and designed around it with three-wheel odometry. Documenting a hardware fault and mitigation is engineering maturity, not failure.

T19

EN: Can you operate airside? RU: Можете ли работать airside? Strong answer: Not as a current claim. Airside has strict vehicle/person rules. Our concept is terminal assistance under airport process — start landside/sterile-zone pilots only with authority approval.

T20

EN: What if the map is outdated after renovation? RU: Что если карта устарела после ремонта? Strong answer: Remap and recalibrate destinations. Outdated maps are an operational hazard; fail-closed goals and supervised updates are mandatory.

T21

EN: How do you avoid blocking fire exits? RU: Как не блокировать пожарные выходы? Strong answer: Keep-out zones in the map, speed policies, staff SOPs, and never stationing the robot on evacuation paths. This is process + map design, not only code.

T22

EN: Is BarcodeDetector reliable enough? RU: Достаточно ли надёжен BarcodeDetector? Strong answer: Good for demos on supported Chromium; not sacred. Manual entry and future hardware scanners are fallbacks.

T23

EN: Why SQLite and demo tickets — is that fake? RU: Почему SQLite и демо-билеты — это фейк? Strong answer: It is a deliberate lab stand-in. Real airports need controlled data sources. We separate demo scaffolding from production integration work.

T24

EN: What is your single biggest technical risk? RU: Какой один главный техриск? Strong answer: Unsafe motion near people if perception/control fails. That is why layered timeouts dominate our safety story.

T25

EN: How do you prove the watchdog works? RU: Как доказать, что watchdog работает? Strong answer: Pull the network cable / freeze the UI / stop publishing commands and measure time-to-zero velocity. Show the test, not only the slide.

T26

EN: Why shouldn't we use a phone robot stand instead? RU: Почему не стойка с телефоном? Strong answer: A phone on a stick doesn't escort, doesn't own motor safety layers, and doesn't teach mecanum+LiDAR integration. Different project class.

T27

EN: Do you overfit the demo to one hallway? RU: Не переобучены ли вы под один коридор? Strong answer: Risk exists. Mitigation: remap tests, different floors, obstacle placements, and honest reporting of where it fails.

T28

EN: What happens on soft carpet? RU: Что на мягком ковре? Strong answer: Slip rises, odometry worsens, calibration drifts. We may need retune drive_cal and slower speeds — floor dependence is admitted.

T29

EN: Why trust browser code for tickets? RU: Почему доверять браузеру билеты? Strong answer: For kiosk UX convenience. Sensitive control stays server-side with gates; boarding coordinates are not sent to the browser.

T30

EN: Is your system deterministic? RU: Детерминирована ли система? Strong answer: Planner on a fixed map is largely deterministic; timing, USB, and people are not. We design safety for the nondeterministic world.

T31

EN: Could a cheaper ultrasonic bumper replace LiDAR? RU: Может дешёвый УЗ-бампер заменить лидар? Strong answer: As bumper layer maybe; as mapping/navigation primary sensor — no, resolution and coverage are insufficient for our approach.

T32

EN: How do you respond to 'this already exists'? RU: Как отвечать на «это уже существует»? Strong answer: Agree that service robots exist. Then state our specific vertical slice, cost/explainability trade-offs, and local languages — contribution is integration depth, not inventing airports.

T33

EN: What if two passengers try to use it at once? RU: Что если двое пассажиров сразу? Strong answer: Session/lease model should serialize escort control; UX must show busy state. Fleet and queueing are future ops features.

T34

EN: Why ROS message types if you also use JSON files? RU: Зачем ROS-сообщения, если есть JSON? Strong answer: ROS connects sensing/control graph; JSON file bridge is a pragmatic IPC to the serial process. Hybrid, but intentional.

T35

EN: What is stopping you from claiming full autonomy? RU: Что мешает заявить полную автономию? Strong answer: Ethics and evidence. Autonomy theater endangers people and credibility. We claim supervised prototype capability.

T36

EN: How do you handle luggage carts as dynamic obstacles? RU: Как быть с тележками для багажа? Strong answer: They appear as LiDAR hits / temporary obstacles; narrow spaces may need wait/replan or supervisor help. Not a solved social-nav paper claim.

T37

EN: Why four languages if models can translate later? RU: Почему 4 языка, если перевод можно потом? Strong answer: Because the first confused second is when language fails. Shipping TT/EN/ZH/RU strings is practical inclusion now.

T38

EN: What is your response to a hostile 'show code' request? RU: Что на враждебное «покажите код»? Strong answer: Open the critical path: timeout constants, A* entry, pursuit, heartbeat lease. Offer a guided tour, not a 10k-line dump.

T39

EN: Can the robot scare children or pets? RU: Может ли пугать детей/животных? Strong answer: Yes — speed, indication, and approach behavior matter. Soft appearance and clear lights/sounds are part of future human-factors work.

T40

EN: Why should the jury fund a pilot? RU: Почему жюри/партнёры финансировать пилот? Strong answer: Because the risky software path already exists, remaining spend buys enclosure, SOPs, and measured passenger outcomes — efficient next TRL step.

T41

EN: What if scan matching locks onto the wrong corridor? RU: Что если scan match «залипнет» не на тот коридор? Strong answer: Detect inconsistency via poor scores / sudden jumps, stop, and require operator relocalization. Never silently trust a bad pose.

T42

EN: Is pure pursuit enough vs MPPI/DWA? RU: Хватает ли pure pursuit против MPPI/DWA? Strong answer: Enough for our current structured demos. Richer local planners are future options if dynamic crowds demand them.

T43

EN: How do you avoid overfitting answers to memorized scripts? RU: Как не отвечать заученными скриптами? Strong answer: Memorize structure: claim → evidence → limit → next test. Adapt facts to the exact question.

T44

EN: What is your unsafe answer you refuse to give? RU: Какой небезопасный ответ вы не дадите? Strong answer: We refuse 'it always works in any terminal without supervision.' That sentence is not engineering.

T45

EN: Why is fail-closed better than best-effort escort? RU: Почему fail-closed лучше best-effort escort? Strong answer: A wrong destination is worse than no escort. Airports punish confident mistakes.

T46

EN: How do batteries change the safety case? RU: Как батареи меняют safety case? Strong answer: Energy depletion can cause brownouts mid-motion; monitoring SoC and forcing safe stop is part of operational safety.

T47

EN: What is the difference between demo success and operational success? RU: Чем успех демо отличается от операционного? Strong answer: Demo: controlled path works. Operational: metrics under real crowds, staff workflows, weeks of uptime, and incident handling.

T48

EN: Why not buy an industrial AMR and slap a kiosk on it? RU: Почему не купить промышленный AMR и повесить киоск? Strong answer: Cost, closed stacks, and weak educational ownership. Our goal includes understanding the full chain.

T49

EN: How do you answer 'show statistical significance'? RU: Как отвечать на «где статистическая значимость»? Strong answer: Present sample size, conditions, and variance honestly. If N is small, say so and describe the next measurement plan.

T50

EN: What is the first experiment after defense? RU: Какой первый эксперимент после защиты? Strong answer: Stop-latency and lease-loss tests recorded on video, then crowded-obstacle runs in a supervised hall — evidence before features.

T51

EN: How do you keep answers short under time pressure? RU: Как коротко отвечать под таймером? Strong answer: Lead with the decision sentence, then one mechanism, then one limitation.

T52

EN: What misconception about ROS should students avoid? RU: Какое заблуждение про ROS избегать? Strong answer: ROS is not autopilot. It is middleware. Autonomy still needs your models, tunings, and safety case.

T53

EN: Why publish limitations on a slide? RU: Зачем публиковать ограничения на слайде? Strong answer: Because jury trust rises when claims match reality. Hidden limits become hostile questions.

T54

EN: What does 'cspc_lidar' imply for branding answers? RU: Что значит cspc_lidar для ответов про бренд? Strong answer: We name the driver we run. We do not invent 'RPLIDAR A3' if the repo doesn't say it.

T55

EN: How do you handle contradictory older presentation text? RU: Как быть со старыми противоречивыми текстами презы? Strong answer: Prefer repository truth. Say versions evolved; today's claim set is the tightened one.

T56

EN: What is a strong closing sentence under pressure? RU: Какая сильная финальная фраза под давлением? Strong answer: 'We can demonstrate the stop behavior and the A→B pipeline; deployment claims wait for a supervised pilot.'

T57

EN: Why heartbeat 6 seconds — magic number? RU: Почему heartbeat 6 секунд — магическое число? Strong answer: It balances Wi-Fi blips against unbounded motion after a closed tab. Tunable, but must be finite and tested.

T58

EN: What if someone removes the allowlist? RU: Что если кто-то убереёт allowlist? Strong answer: That would be a security regression. Production must enforce auth and network segmentation; code review and config management matter.

T59

EN: How do you talk about 'AI' without cringe? RU: Как говорить про AI без кринжа? Strong answer: Name concrete models/algorithms: YuNet, A*, ICP. Avoid 'our AI thinks'. Precision beats buzzwords.

T60

EN: What personal quality should the team show? RU: Какое личное качество показать команде? Strong answer: Calm honesty under questions — the same trait you want near a moving robot.

T61

EN: How do you share credit without chaos? RU: Как делить вклад без хаоса? Strong answer: Pre-agree who answers hardware, nav, UI. Hand off explicitly: 'I'll pass to ... for electronics.'

T62

EN: What is the clever-but-wrong answer to avoid? RU: Какой умный, но неверный ответ избегать? Strong answer: 'We use Nav2/AMCL' if you don't. Clever falsehoods collapse instantly.

T63

EN: How do you respond to a question you truly don't know? RU: Как отвечать на вопрос, который реально не знаете? Strong answer: 'I don't know yet; here is how we would measure it.' Then outline the experiment.

T64

EN: Why is supervised autonomy still impressive? RU: Почему supervised autonomy всё ещё впечатляет? Strong answer: Because the hard parts — mapping, planning, tracking, UI integration, layered stops — still had to work.

T65

EN: What is your anti-hype mantra? RU: Какая анти-хайп мантра? Strong answer: Evidence over adjectives. Timeouts over slogans. Limits over legends.

T66

EN: How do you justify custom occupancy vs slam_toolbox? RU: Как обосновать свой occupancy против slam_toolbox? Strong answer: Ownership and clarity for our mecanum platform. We can revisit packages when maintenance needs outweigh transparency.

T67

EN: What operational metric would falsify your readiness? RU: Какая операционная метрика опровергнет готовность? Strong answer: If stop latency under fault injection exceeds the safety budget, we are not ready — full stop.

T68

EN: How do you explain three-wheel odometry briefly? RU: Как кратко объяснить одометрию на трёх колёсах? Strong answer: Each trusted wheel contributes motion constraints; we integrate those and ignore the noisy FR channel rather than poison the pose.

T69

EN: What is a good answer about multilingual UX? RU: Какой хороший ответ про мультиязычность? Strong answer: Language is part of accessibility in airports; shipping four locales is product engineering, not decoration.

T70

EN: How do you discuss privacy of ticket names? RU: Как говорить о приватности имён на билетах? Strong answer: Demo data is controlled; real deployments need minimization, access control, retention limits, and legal basis — we don't casually collect PII.

T71

EN: Why is 'we'll fix it later' a bad defense line? RU: Почему «потом поправим» — плохая линия защиты? Strong answer: Because safety-critical gaps need present mitigations or explicit non-claims, not vibes.

T72

EN: What is your best one-minute architecture answer? RU: Лучший минутный ответ по архитектуре? Strong answer: Kiosk/operator → HTTP:8765 → plan/track in lidar_map → cmd to Mega → mecanum motion; LiDAR /scan updates map/pose; timeouts revoke motion privilege.

T73

EN: How do you answer cost comparisons to industrial AMRs? RU: Как отвечать на сравнение стоимости с industrial AMR? Strong answer: Different TRL and certification class. We are an explainable prototype; industrial AMRs price includes safety cases and support.

T74

EN: What if the jury asks for live airport video proof? RU: Что если просят видео из живого аэропорта? Strong answer: Show lab/controlled-space evidence and state that airport video requires permissions we do not fake.

T75

EN: How do you treat 'innovation' questions? RU: Как отвечать на вопросы про инновацию? Strong answer: Innovation = integrated ticket-aware escort stack on a transparent custom navigator, not a new law of physics.

T76

EN: What is the correct humility level? RU: Какой правильный уровень скромности? Strong answer: Proud of working pipeline; humble about deployment. Both at once.

T77

EN: How do you prevent teammates contradicting each other? RU: Как не противоречить друг другу? Strong answer: Shared truth table: Pi+Mega, no Nav2, cspc_lidar, FR untrusted, ~18k without LiDAR, no real-terminal claim.

T78

EN: What question do you hope they ask? RU: Какой вопрос вы хотите получить? Strong answer: 'Show the stop on disconnect.' It turns safety from words into evidence.

T79

EN: What question do you fear — and how reframe it? RU: Какого вопроса боитесь — и как перефразировать? Strong answer: Fear: 'Does it work in Kazan airport today?' Reframe: 'Here is what works in controlled conditions and what a pilot requires.'

T80

EN: Why is documentation part of safety? RU: Почему документация — часть безопасности? Strong answer: Operators cannot respect limits they cannot know. SOPs and honest READMEs prevent misuse.

T81

EN: How do you explain temporary obstacles simply? RU: Как просто объяснить temporary obstacles? Strong answer: Fresh LiDAR hits briefly mark 'don't go here' so the planner respects a person or cart that wasn't in the static map.

T82

EN: What is a crisp answer on UI vs autonomy? RU: Чёткий ответ про UI vs автономию? Strong answer: UI creates intent; autonomy executes motion; safety can cancel both.

T83

EN: How do you talk about school constraints positively? RU: Как говорить о школьных ограничениях позитивно? Strong answer: Constraints forced modularity, cost control, and explainability — which improved the engineering story.

T84

EN: What is your final tricky-question posture? RU: Какая финальная позиция на каверзных? Strong answer: Calm, specific, falsifiable. Invite measurement.

Tricky questions total: 100

T85

EN: Why publish driver name instead of a famous LiDAR brand? RU: Почему имя драйвера, а не известный бренд лидара? Strong answer: Because honesty beats prestige. Naming cspc_lidar matches the repo; inventing RPLIDAR would be a credibility landmine.

T86

EN: What if the occupancy map has phantom walls? RU: Что если на карте фантомные стены? Strong answer: Remap, clear transient clutter, verify with teleop, and do not escort through nonsense geometry.

T87

EN: Can you rely on odometry alone in a long corridor? RU: Можно ли только на одометрии в длинном коридоре? Strong answer: Not for long. Drift grows; scan matching against structure is what keeps pose usable.

T88

EN: How do you answer a professor who loves Nav2? RU: Как ответить профессору, который любит Nav2? Strong answer: Respect Nav2, show your stack's clarity, and discuss migration criteria — don't wage religion wars.

T89

EN: What is the ethical line for filming demos? RU: Какая этическая граница съёмки демо? Strong answer: Avoid identifiable strangers; prefer controlled spaces; blur faces in public logs.

T90

EN: Why is 'AI robot' a weak title for AeroPorter? RU: Почему «AI-робот» — слабое название? Strong answer: It hides mechanisms. Better: ticket-aware LiDAR escort prototype on Pi+Mega.

T91

EN: What if Pure Pursuit cuts corners near people? RU: Что если Pure Pursuit срезает углы у людей? Strong answer: Reduce look-ahead, inflate obstacles, lower speed, or stop-and-replan — tuning is safety.

T92

EN: How do you justify HTTP for robot control? RU: Как оправдать HTTP для управления? Strong answer: Pragmatic for browser UIs on LAN; production needs stronger auth and segmentation.

T93

EN: What is your response to 'add GPS'? RU: Что на совет «добавьте GPS»? Strong answer: Indoor terminals are GPS-denied; LiDAR maps are the right class of solution.

T94

EN: Why not claim face recognition attendance? RU: Почему не заявлять face recognition? Strong answer: We don't build identity. Claiming it would be false and privacy-hostile.

T95

EN: How do you handle jury interrupting mid-sentence? RU: Как быть, если жюри перебивает? Strong answer: Pause, answer the interrupt, then ask if they want the previous point finished.

T96

EN: What is a dangerous half-truth on cost? RU: Какая опасная полуправда про стоимость? Strong answer: Saying the robot is 'cheap' without listing excluded LiDAR/enclosure/certification.

T97

EN: How do you show modularity in 15 seconds? RU: Как показать модульность за 15 секунд? Strong answer: Point to four folders and say which can die without killing the others' development.

T98

EN: What if Mega firmware and Pi units disagree on velocity scale? RU: Что если шкалы скорости Mega и Pi расходятся? Strong answer: Recalibrate drive parameters; mismatched units cause unstable pursuit.

T99

EN: Why heartbeat rather than only watchdog? RU: Почему heartbeat, а не только watchdog? Strong answer: Watchdog catches stale motion commands; heartbeat catches abandoned passenger sessions even if some command stream looks alive.

T100

EN: What is your stance on cloud dependency? RU: Ваша позиция по зависимости от облака? Strong answer: Demos should work locally; cloud can enrich flight data later without becoming a single point of failure for stopping safely.

Tricky questions total (updated): 100

6. Follow-up Question Chains

Each chain: main question → deeper probes (how juries actually dig).

F1

How accurate is your system? ↓ How did you measure accuracy? ↓ Can you show the test setup? ↓ What is the error margin under what floor type?

F2

How much does it cost? ↓ What exactly is included in the BOM? ↓ Why is LiDAR excluded? ↓ What would a pilot budget add?

F3

Is it safe near people? ↓ What is the measured stop latency? ↓ Which layer stops it if the Pi freezes? ↓ What residual risks remain?

F4

Why not Nav2? ↓ What parts of Nav2 would help later? ↓ What did you implement instead? ↓ How do you validate your planner?

F5

Why Raspberry Pi 5? ↓ What CPU load do you see while navigating? ↓ When would you move to Jetson? ↓ What is thermally limiting?

F6

How does localization work? ↓ What happens when scan matching score is poor? ↓ How do you detect kidnapped-robot cases? ↓ Do you claim loop closure?

F7

How do tickets work? ↓ Is the database real or demo? ↓ What statuses are accepted? ↓ Why doesn't the browser get coordinates?

F8

What if connection drops? ↓ Which timeout fires first? ↓ Can you demo cable-pull stop? ↓ How is the lease renewed?

F9

Why mecanum? ↓ How do you calibrate wheel slip? ↓ What fails on carpet? ↓ How does inverse kinematics enter firmware?

F10

How do you compare to Pepper? ↓ On which metric do they win? ↓ On which metric do you win? ↓ Is the comparison fair on cost class?

F11

What about glass? ↓ Any mitigation beyond 'be careful'? ↓ Would RGB cameras help? ↓ Is this a go/no-go for some halls?

F12

How do you plan paths? ↓ Why A-star not RRT? ↓ How do diagonals affect path length? ↓ When do you replan?

F13

Explain pure pursuit. ↓ How is look-ahead chosen? ↓ What is CTE correction? ↓ Failure mode in sharp corners?

F14

What is your map resolution? ↓ Why that cell size? ↓ How big is the map in memory? ↓ How do you persist it?

F15

Who is the user? ↓ How did you validate the need? ↓ Which user is out of scope? ↓ How does UX change for elderly users?

F16

What languages are supported? ↓ Why Tatar? ↓ Who translates strings? ↓ How do you test RTL-unrelated layout issues?

F17

What is the FR encoder issue? ↓ How did you discover it? ↓ How is odometry formulated without it? ↓ Any plan to replace hardware?

F18

How do you power motors? ↓ Any brownout events logged? ↓ Fuse strategy? ↓ Idle vs drive current?

F19

What is the pilot plan? ↓ Landside or airside? ↓ Who supervises? ↓ What KPI after 30 days?

F20

How do you handle crowds? ↓ Max speed policy? ↓ Temporary obstacles TTL? ↓ When does teleop take over?

F21

Is this SLAM? ↓ What SLAM features do you lack? ↓ Why is occupancy+scan match enough for demos? ↓ What would falsify that choice?

F22

How do you secure the API? ↓ Allowlist details? ↓ Auth plans for production? ↓ Risk on public Wi-Fi?

F23

What is heartbeat lease? ↓ Why ~6 seconds? ↓ What if browser backgrounds the tab? ↓ How do you test it?

F24

Show the architecture. ↓ Where can it fail silently? ↓ Which interface is most fragile? ↓ How do folders isolate risk?

F25

What did each teammate build? ↓ Where do interfaces meet? ↓ How do you resolve conflicts? ↓ Who answers safety questions?

F26

What tests exist? ↓ Unit vs field tests? ↓ What is not tested yet? ↓ How are results recorded?

F27

What is the roadmap? ↓ What is intentionally not next? ↓ What dependency blocks deployment? ↓ How do you avoid scope creep?

F28

How do destinations get calibrated? ↓ Who measures x,y? ↓ What if calibration drifts? ↓ Why fail-closed?

F29

What about privacy? ↓ What PII appears on screen? ↓ Any face recognition claim? ↓ Retention policy idea?

F30

How loud/fast is it? ↓ Any acoustic measurements? ↓ Speed near queues? ↓ Human-factors next steps?

F31

What if a child grabs the screen? ↓ UI timeout behavior? ↓ Physical stability? ↓ Supervisor response?

F32

How do you update software on the robot? ↓ Rollback plan? ↓ Config vs code updates? ↓ Who has SSH access?

F33

Explain CSM vs ICP. ↓ Which runs first? ↓ What if both disagree with odom? ↓ CPU cost?

F34

How do you filter self-hits? ↓ What geometry is assumed? ↓ Failure if mount shifts? ↓ Calibration of filter?

F35

What is /cmd_vel? ↓ Who publishes it? ↓ Who subscribes? ↓ What if two publishers fight?

F36

How does the Mega timeout interact with Pi watchdog? ↓ Which is backup for which? ↓ Can they both fail? ↓ Hardware e-stop plans?

F37

What analogs exist in Russia? ↓ Promobot differences? ↓ Fixed kiosk differences? ↓ Your unfair advantage?

F38

How do you teach staff? ↓ One-page SOP contents? ↓ Stop authority? ↓ Escalation path?

F39

What is the battery endurance? ↓ How measured? ↓ Behavior at low SoC? ↓ Swap procedure?

F40

How do you handle gate changes? ↓ Need live FIDS? ↓ Replan mid-escort? ↓ Who confirms new gate?

F41

What is the biggest unknown? ↓ How will you reduce it? ↓ What would you measure this week? ↓ What would change the design?

F42

Why HTTP not only ROS topics for UI? ↓ Browser constraints? ↓ LAN risks? ↓ Future protocol changes?

F43

How do you validate A→B success? ↓ Start/goal definitions? ↓ Success tolerance? ↓ Failure taxonomy?

F44

What mechanical wear do you expect? ↓ Which part fails first? ↓ Spare strategy? ↓ Inspection checklist?

F45

How do you avoid marketing language? ↓ Which words are banned? ↓ How do slides stay honest? ↓ Who reviews claims?

F46

What is the educational outcome? ↓ What skills did you gain? ↓ What would you teach juniors? ↓ What remains hard?

F47

Can this generalize to hospitals? ↓ What domain assumptions break? ↓ What can reuse? ↓ Why stay focused on airports now?

F48

How do you record demos? ↓ Privacy of bystanders? ↓ Log redaction? ↓ Evidence packaging for jury?

F49

What is the role of teleop? ↓ Is teleop cheating? ↓ When is teleop mandatory? ↓ How indicated to public?

F50

How do you argue TRL? ↓ What TRL now? ↓ What TRL after pilot? ↓ What evidence moves TRL?

F51

What environmental limits? ↓ Temperature? ↓ Lighting for barcodes? ↓ Floor types?

F52

How do you set inflation radius? ↓ Too big vs too small? ↓ Relation to footprint? ↓ Empirically tuned how?

F53

What is yaw hold? ↓ When does it help? ↓ When does it hurt? ↓ Interaction with pursuit?

F54

How do you manage USB device order? ↓ udev rules? ↓ Failure symptom? ↓ Operational checklist?

F55

What is the kiosk lease busy state? ↓ Second passenger UX? ↓ Queue idea? ↓ Fleet later?

F56

How do you answer without numbers you lack? ↓ Qualitative vs quantitative? ↓ How to promise a measurement? ↓ How not to invent?

F57

What is the worst demos failure you had? ↓ Root cause? ↓ Fix? ↓ What monitoring caught it?

F58

How do you keep code modular? ↓ Folder contracts? ↓ What not to import across? ↓ CI dreams?

F59

What security baseline for pilot? ↓ Network segmentation? ↓ Auth? ↓ Physical access to robot?

F60

How do you present limitations without losing the room? ↓ Sandwich with evidence? ↓ Timeboxing? ↓ Invite pilot as next step?

F61

What question should end the defense? ↓ Offer live stop test? ↓ Offer map view? ↓ Thank jury how?

F62

How accurate is ticket scanning? ↓ Lighting conditions tested? ↓ Manual fallback rate? ↓ Scanner hardware plan?

F63

What is the operator's job during escort? ↓ Eyes on what? ↓ When to intervene? ↓ After-stop checklist?

F64

How do you estimate compute headroom? ↓ Tools? ↓ What process dominates CPU? ↓ What optimization next?

F65

Why log-odds occupancy? ↓ Binary maps alternative? ↓ Dynamic objects issue? ↓ Save format?

F66

How do you treat unknown space? ↓ Traverse unknown? ↓ Danger? ↓ Exploration claims?

F67

What is the look of a good path? ↓ Smoothness? ↓ Clearance? ↓ Human comfort?

F68

How do you coordinate camera with navigation? ↓ Same process? ↓ Failure isolation? ↓ Priority under CPU load?

F69

What is the serial protocol roughly? ↓ Command rate? ↓ Telemetry? ↓ Framing errors?

F70

How do you prove encoder trust decisions? ↓ Plots? ↓ Dropouts? ↓ Before/after odom error?

F71

What facility map constraints? ↓ Max corridor width? ↓ Glass atriums? ↓ Elevators in scope?

F72

How do you version maps? ↓ Filename discipline? ↓ Who may overwrite? ↓ Rollback of bad map?

F73

What is a calibrated destination record? ↓ Fields? ↓ Who signs off? ↓ Invalid template behavior?

F74

How do languages affect layout? ↓ String length? ↓ Font issues? ↓ Test checklist?

F75

What is your incident severity scale? ↓ Near miss? ↓ Contact? ↓ Reporting template?

F76

How do you train for Q&A? ↓ Mock interviews? ↓ Time limits? ↓ Role rotation?

F77

What is the difference between verification and validation here? ↓ Examples? ↓ Which did you do more? ↓ Gap?

F78

How do you avoid scope creep from jury suggestions? ↓ Parking lot list? ↓ Criteria to accept? ↓ Polite deferral phrase?

F79

What is your data backup plan on the Pi? ↓ Maps? ↓ Configs? ↓ Secrets?

F80

How do you handle time zones / flight times display? ↓ Source of truth? ↓ Clock sync? ↓ Stale times?

F81

What accessibility features exist? ↓ Font size? ↓ Language? ↓ Future voice/PRM?

F82

How do you justify not using AMCL? ↓ What AMCL needs? ↓ Your alternative? ↓ Risk?

F83

What is the role of temporary TTL obstacles in A*? ↓ Interaction with static map? ↓ Memory growth? ↓ Tuning?

F84

How do you evaluate scan match quality live? ↓ Score thresholds? ↓ Operator visibility? ↓ Auto-stop rule?

F85

What is your position on open-sourcing? ↓ What to publish? ↓ What to keep private? ↓ License idea?

F86

How do you respond to budget cuts? ↓ What to descope first? ↓ What never to descope? ↓ Safety vs features?

F87

What weather/HVAC airflow issues indoors? ↓ Drafts on soft robots? ↓ Door overpressures? ↓ Relevance?

F88

How do you keep the story consistent with the README? ↓ Claim review? ↓ Single source of truth? ↓ Slide lint?

F89

What is the minimal equipment to reproduce the demo? ↓ Pi image? ↓ Dependencies? ↓ Known brittle steps?

F90

How do you explain mecanum slip to non-engineers? ↓ Analogy? ↓ Why calibration? ↓ Why speed limits?

F91

What is your answer to 'show novelty mathematically'? ↓ What is standard? ↓ What is integrated uniquely? ↓ Where is originality?

F92

How do you end a too-long answer? ↓ Signal phrase? ↓ Offer appendix? ↓ Check jury satisfaction?

F93

How do you handle stop latency? ↓ How do you measure map quality? ↓ What failure looks like for CPU load? ↓ What is the next test for USB flakiness?

F94

How do you handle PWM frequency? ↓ How do you measure goal tolerance? ↓ What failure looks like for inflation radius? ↓ What is the next test for look-ahead?

F95

How do you handle lease duration? ↓ How do you measure allowlist? ↓ What failure looks like for demo tickets? ↓ What is the next test for calibrated x y?

F96

How do you handle glass atrium? ↓ How do you measure carpet slip? ↓ What failure looks like for teleop takeover? ↓ What is the next test for status lights?

F97

How do you handle soft bumper? ↓ How do you measure SOP training? ↓ What failure looks like for incident log? ↓ What is the next test for SSID policy?

F98

How do you handle SSH hardening? ↓ How do you measure image backup? ↓ What failure looks like for thermal throttle? ↓ What is the next test for battery SoC?

F99

How do you handle wheel wear? ↓ How do you measure cable snag? ↓ What failure looks like for self filter? ↓ What is the next test for scan rate?

F100

How do you handle baud mismatch? ↓ How do you measure origin frame? ↓ What failure looks like for quaternion yaw? ↓ What is the next test for path smoothing?

F101

How do you handle replan rate? ↓ How do you measure progress timeout? ↓ What failure looks like for busy kiosk? ↓ What is the next test for language pack?

F102

How do you handle font scaling? ↓ How do you measure touch calibration? ↓ What failure looks like for barcode glare? ↓ What is the next test for manual code UX?

F103

How do you handle operator blur stop? ↓ How do you measure visibility change? ↓ What failure looks like for CORS origin? ↓ What is the next test for private IP ranges?

F104

How do you handle DNS dependency? ↓ How do you measure NTP clock? ↓ What failure looks like for flight status stale? ↓ What is the next test for gate rename?

F105

How do you handle zone A D? ↓ How do you measure toilet destination? ↓ What failure looks like for baggage carousel? ↓ What is the next test for info desk?

F106

How do you handle exit door? ↓ How do you measure keepout polygon? ↓ What failure looks like for speed near queue? ↓ What is the next test for child height sensors?

F107

How do you handle pet unpredictability? ↓ How do you measure luggage overhang? ↓ What failure looks like for cart convoy? ↓ What is the next test for cleaning machine?

F108

How do you handle service elevator? ↓ How do you measure threshold ramp? ↓ What failure looks like for dock charger? ↓ What is the next test for spare Mega?

F109

How do you handle spare wheel? ↓ How do you measure tool kit? ↓ What failure looks like for field checklist? ↓ What is the next test for preflight test?

F110

How do you handle postflight inspection? ↓ How do you measure video evidence? ↓ What failure looks like for metric dashboard? ↓ What is the next test for log rotation?

F111

How do you handle PII redaction? ↓ How do you measure GDPR basis? ↓ What failure looks like for consent poster? ↓ What is the next test for supervisor tablet?

F112

How do you handle radio deadzone? ↓ How do you measure channel interference? ↓ What failure looks like for USB hub power? ↓ What is the next test for PoE camera future?

F113

How do you handle model quantization? ↓ How do you measure ONNX runtime? ↓ What failure looks like for Haar fallback? ↓ What is the next test for YuNet confidence?

F114

How do you handle NMS threshold? ↓ How do you measure ROI crop? ↓ What failure looks like for PTZ limits? ↓ What is the next test for RTSP drop?

F115

How do you handle face privacy? ↓ How do you measure crowd density metric? ↓ What failure looks like for intervention count? ↓ What is the next test for escort success rate?

F116

How do you handle mean time to stop? ↓ How do you measure false obstacle rate? ↓ What failure looks like for phantom wall rate? ↓ What is the next test for relocalization time?

F117

How do you handle map save frequency? ↓ How do you measure config pinning? ↓ What failure looks like for release notes? ↓ What is the next test for claim review?

F118

What is your claim about A-star? ↓ What evidence supports A-star? ↓ What would falsify A-star? ↓ What experiment updates A-star?

F119

How do you configure pure pursuit? ↓ Who is allowed to change pure pursuit? ↓ How do you roll back pure pursuit? ↓ How do you audit pure pursuit?

F120

Why does occupancy grid matter for passengers? ↓ How do you explain occupancy grid to non-engineers? ↓ What misconception exists about occupancy grid? ↓ What demo shows occupancy grid?

F121

What is your claim about CSM? ↓ What evidence supports CSM? ↓ What would falsify CSM? ↓ What experiment updates CSM?

F122

How do you configure ICP? ↓ Who is allowed to change ICP? ↓ How do you roll back ICP? ↓ How do you audit ICP?

F123

Why does mecanum IK matter for passengers? ↓ How do you explain mecanum IK to non-engineers? ↓ What misconception exists about mecanum IK? ↓ What demo shows mecanum IK?

F124

What is your claim about PI wheel control? ↓ What evidence supports PI wheel control? ↓ What would falsify PI wheel control? ↓ What experiment updates PI wheel control?

F125

How do you configure heartbeat lease? ↓ Who is allowed to change heartbeat lease? ↓ How do you roll back heartbeat lease? ↓ How do you audit heartbeat lease?

F126

Why does Mega timeout matter for passengers? ↓ How do you explain Mega timeout to non-engineers? ↓ What misconception exists about Mega timeout? ↓ What demo shows Mega timeout?

F127

What is your claim about drive watchdog? ↓ What evidence supports drive watchdog? ↓ What would falsify drive watchdog? ↓ What experiment updates drive watchdog?

F128

How do you configure fail-closed destinations? ↓ Who is allowed to change fail-closed destinations? ↓ How do you roll back fail-closed destinations? ↓ How do you audit fail-closed destinations?

F129

Why does BarcodeDetector matter for passengers? ↓ How do you explain BarcodeDetector to non-engineers? ↓ What misconception exists about BarcodeDetector? ↓ What demo shows BarcodeDetector?

F130

What is your claim about YuNet? ↓ What evidence supports YuNet? ↓ What would falsify YuNet? ↓ What experiment updates YuNet?

F131

How do you configure kiosk languages? ↓ Who is allowed to change kiosk languages? ↓ How do you roll back kiosk languages? ↓ How do you audit kiosk languages?

F132

Why does operator console matter for passengers? ↓ How do you explain operator console to non-engineers? ↓ What misconception exists about operator console? ↓ What demo shows operator console?

F133

What is your claim about remembered map? ↓ What evidence supports remembered map? ↓ What would falsify remembered map? ↓ What experiment updates remembered map?

F134

How do you configure temporary obstacles? ↓ Who is allowed to change temporary obstacles? ↓ How do you roll back temporary obstacles? ↓ How do you audit temporary obstacles?

F135

Why does yaw hold matter for passengers? ↓ How do you explain yaw hold to non-engineers? ↓ What misconception exists about yaw hold? ↓ What demo shows yaw hold?

F136

What is your claim about drive_cal? ↓ What evidence supports drive_cal? ↓ What would falsify drive_cal? ↓ What experiment updates drive_cal?

F137

How do you configure FR encoder policy? ↓ Who is allowed to change FR encoder policy? ↓ How do you roll back FR encoder policy? ↓ How do you audit FR encoder policy?

F138

Why does port 8765 matter for passengers? ↓ How do you explain port 8765 to non-engineers? ↓ What misconception exists about port 8765? ↓ What demo shows port 8765?

F139

What is your claim about SQLite tickets? ↓ What evidence supports SQLite tickets? ↓ What would falsify SQLite tickets? ↓ What experiment updates SQLite tickets?

F140

How do you configure sample ticket codes? ↓ Who is allowed to change sample ticket codes? ↓ How do you roll back sample ticket codes? ↓ How do you audit sample ticket codes?

F141

Why does cspc_lidar matter for passengers? ↓ How do you explain cspc_lidar to non-engineers? ↓ What misconception exists about cspc_lidar? ↓ What demo shows cspc_lidar?

F142

What is your claim about USB camera? ↓ What evidence supports USB camera? ↓ What would falsify USB camera? ↓ What experiment updates USB camera?

F143

How do you configure Tapo path? ↓ Who is allowed to change Tapo path? ↓ How do you roll back Tapo path? ↓ How do you audit Tapo path?

F144

Why does soft real-time loops matter for passengers? ↓ How do you explain soft real-time loops to non-engineers? ↓ What misconception exists about soft real-time loops? ↓ What demo shows soft real-time loops?

F145

What is your claim about LAN allowlist? ↓ What evidence supports LAN allowlist? ↓ What would falsify LAN allowlist? ↓ What experiment updates LAN allowlist?

F146

How do you configure Origin checks? ↓ Who is allowed to change Origin checks? ↓ How do you roll back Origin checks? ↓ How do you audit Origin checks?

F147

Why does stop on blur matter for passengers? ↓ How do you explain stop on blur to non-engineers? ↓ What misconception exists about stop on blur? ↓ What demo shows stop on blur?

F148

What is your claim about map autosave? ↓ What evidence supports map autosave? ↓ What would falsify map autosave? ↓ What experiment updates map autosave?

F149

How do you configure goal tolerance? ↓ Who is allowed to change goal tolerance? ↓ How do you roll back goal tolerance? ↓ How do you audit goal tolerance?

F150

Why does inflation matter for passengers? ↓ How do you explain inflation to non-engineers? ↓ What misconception exists about inflation? ↓ What demo shows inflation?

F151

What is your claim about look-ahead? ↓ What evidence supports look-ahead? ↓ What would falsify look-ahead? ↓ What experiment updates look-ahead?

F152

How do you configure CTE? ↓ Who is allowed to change CTE? ↓ How do you roll back CTE? ↓ How do you audit CTE?

F153

Why does scan self-filter matter for passengers? ↓ How do you explain scan self-filter to non-engineers? ↓ What misconception exists about scan self-filter? ↓ What demo shows scan self-filter?

F154

What is your claim about three-wheel odom? ↓ What evidence supports three-wheel odom? ↓ What would falsify three-wheel odom? ↓ What experiment updates three-wheel odom?

F155

How do you configure PWM direction pins? ↓ Who is allowed to change PWM direction pins? ↓ How do you roll back PWM direction pins? ↓ How do you audit PWM direction pins?

F156

Why does serial 115200 matter for passengers? ↓ How do you explain serial 115200 to non-engineers? ↓ What misconception exists about serial 115200? ↓ What demo shows serial 115200?

F157

What is your claim about LiDAR 230400? ↓ What evidence supports LiDAR 230400? ↓ What would falsify LiDAR 230400? ↓ What experiment updates LiDAR 230400?

F158

How do you configure ROS LaserScan? ↓ Who is allowed to change ROS LaserScan? ↓ How do you roll back ROS LaserScan? ↓ How do you audit ROS LaserScan?

F159

Why does Twist cmd_vel matter for passengers? ↓ How do you explain Twist cmd_vel to non-engineers? ↓ What misconception exists about Twist cmd_vel? ↓ What demo shows Twist cmd_vel?

F160

What is your claim about footprint 0.82x0.56? ↓ What evidence supports footprint 0.82x0.56? ↓ What would falsify footprint 0.82x0.56? ↓ What experiment updates footprint 0.82x0.56?

F161

How do you configure nav radius? ↓ Who is allowed to change nav radius? ↓ How do you roll back nav radius? ↓ How do you audit nav radius?

F162

Why does log-odds updates matter for passengers? ↓ How do you explain log-odds updates to non-engineers? ↓ What misconception exists about log-odds updates? ↓ What demo shows log-odds updates?

F163

What is your claim about 8-connected search? ↓ What evidence supports 8-connected search? ↓ What would falsify 8-connected search? ↓ What experiment updates 8-connected search?

F164

How do you configure Manhattan heuristic? ↓ Who is allowed to change Manhattan heuristic? ↓ How do you roll back Manhattan heuristic? ↓ How do you audit Manhattan heuristic?

F165

Why does escort API matter for passengers? ↓ How do you explain escort API to non-engineers? ↓ What misconception exists about escort API? ↓ What demo shows escort API?

F166

What is your claim about ticket statuses? ↓ What evidence supports ticket statuses? ↓ What would falsify ticket statuses? ↓ What experiment updates ticket statuses?

F167

How do you configure privacy of coordinates? ↓ Who is allowed to change privacy of coordinates? ↓ How do you roll back privacy of coordinates? ↓ How do you audit privacy of coordinates?

Follow-up chains total: 167

7. AeroPorter Vocabulary (project-specific)

7.1 Components (30)

1. Raspberry Pi 5 — Raspberry Pi 5 | high-level computer for ROS 2, HTTP, planning
2. Arduino Mega 2560 — Arduino Mega 2560 | motor/encoder MCU with command timeout
3. mecanum wheel set — комплект механум-колёс | omnidirectional drive
4. LiDAR + cspc_lidar — лидар + cspc_lidar | scan source via ROS driver
5. USB camera — USB-камера | default vision input
6. Tapo C200 — Tapo C200 | optional RTSP/PTZ camera prototype
7. 10.1" touchscreen — сенсор 10.1" | 1280×800 passenger/operator display class
8. H-bridge drivers — драйверы H-моста | PWM+DIR motor power stage
9. quadrature encoders — квадратурные энкодеры | wheel tick sensors
10. serial USB bridge — USB-UART мост | Pi↔Mega link
11. battery pack — АКБ | mobile power source
12. robot chassis — шасси | 0.82×0.56 m class footprint
13. kiosk browser — браузер киоска | UI client for passengers
14. operator PC/browser — браузер оператора | teleop/map console
15. ticket SQLite DB — SQLite база билетов | local boarding records
16. destinations JSON — JSON назначений | calibrated goal poses
17. remembered occupancy map — запомненная карта | saved grid under robot_nav/maps
18. primary_buttons.json — primary_buttons.json | kiosk main button config
19. drive_cal.json — drive_cal.json | floor-dependent drive calibration
20. status LEDs (planned/needed) — статус-светодиоды | public indication still needed
21. soft bumper (planned) — мягкий бампер | human-contact mitigation still needed
22. USB hub — USB-хаб | expands Pi peripherals
23. wheel module — колёсный модуль | motor+wheel+encoder assembly
24. mounting plate for LiDAR — платформа лидара | mechanical mount reducing self-hits
25. passenger kiosk UI — UI киоска пассажира | /kiosk route
26. teleop pad — телеоп-пад | on-screen drive control
27. emergency stop procedure — процедура e-stop | human stop authority
28. sample ticket codes — пример кодов билетов | lab demo inputs
29. YuNet ONNX model — модель YuNet ONNX | face detection
30. BarcodeDetector — BarcodeDetector | browser barcode API

7.2 Architecture (30)

1. four-folder monorepo — монорепо из 4 папок | arduino / lidar_map / camera / monitor
2. runtime brain lidar_map — runtime-мозг lidar_map | navigation + HTTP server
3. ScanBridge — ScanBridge | connects scans/odom to map/nav
4. HTTP :8765 — HTTP :8765 | operator + kiosk API/UI port
5. cmd_vel topic — топик cmd_vel | ROS velocity commands
6. robot_cmd.json IPC — IPC через robot_cmd.json | file bridge to serial process
7. drive_encoders process — процесс drive_encoders | Mega serial + odom publish
8. airport_service — airport_service | tickets + destinations logic
9. fail-closed config — fail-closed конфиг | invalid goals disable escort
10. operator vs kiosk roles — роли оператор/киоск | supervision vs passenger
11. modular independence — модульная независимость | folders developed separately
12. ROS 2 Jazzy layer — слой ROS 2 Jazzy | middleware, not Nav2 stack
13. custom planner/tracker — свой планировщик/трекер | A* + pure pursuit
14. safety lease session — сессия safety lease | time-limited escort rights

15. local-first demo — local-first демо | works without airport cloud
16. supervised autonomy mode — режим supervised autonomy | human remains responsible
17. graceful degradation — мягкая деградация | info without escort if needed
18. single robot MVP — MVP одного робота | no fleet claims yet
19. landside pilot target — цель пилота landside | avoid airside overclaim
20. truth table claims — таблица истинности заявлений | aligned with repository
21. interface contracts — контракты интерфейсов | HTTP/ROS/serial boundaries
22. configuration as data — конфиг как данные | JSON destinations/buttons
23. map persistence path — путь персистентности карты | ~/robot_nav/maps/...
24. ticket data path — путь данных билетов | ~/robot_nav/data/...
25. destination path — путь назначений | ~/robot_nav/config/...
26. start_kiosk_stack.sh — start_kiosk_stack.sh | common bring-up script
27. start_drive_map.sh — start_drive_map.sh | drive+map launch
28. preview_server.py — preview_server.py | UI preview without full robot
29. allowlist gate — шлюз allowlist | control API IP gating
30. origin check — проверка Origin | browser origin gating

7.3 Electronics (30)

1. PWM speed command — команда скорости ШИМ | duty cycle to drivers
2. IN1/IN2 direction pins — пины направления IN1/IN2 | H-bridge direction
3. CMD_TIMEOUT_MS ~1200 — CMD_TIMEOUT_MS ~1200 | Mega stop if silent
4. 115200 serial — UART 115200 | Pi↔Mega baud
5. encoder A/B pins 46–53 — пины энкодеров 46–53 | Mega inputs
6. trusted FL/RL/RR — доверенные FL/RL/RR | odom contributors
7. untrusted FR — недоверенный FR | feedforward-only / ignored for pose
8. PI wheel rate control — ПИ по скорости колеса | aligns trusted wheels
9. motor supply rail — шина питания моторов | high current path
10. logic supply rail — шина логики | MCU/sensor clean power
11. USB tty enumeration — enumeration USB tty | which device is LiDAR/Mega
12. LiDAR serial 230400 — UART лидара 230400 | driver parameter
13. brownout risk — риск просадки | motors starve logic
14. common ground — общая земля | reference integrity
15. avrdude flash path — прошивка через avrdude | Mega programming
16. EMI on encoder lines — ЭМП на линиях энкодера | false ticks
17. cable strain relief — разгрузка кабелей | connector longevity
18. power budget thinking — мышление энергобюджета | idle vs drive watts
19. watchdog mindset on MCU — watchdog-мышление на МК | assume host can die
20. connector SPOF — разъём как TOT | single point of failure awareness
21. thermal Pi throttling — троттлинг Pi | CPU heat limits
22. USB hub power limit — лимит питания USB-хаба | peripheral brownouts
23. PWM polarity bring-up — проверка полярности ШИМ | direction sanity
24. tick resolution 20 tpr — 20 тиков/оборот | odometry granularity
25. wheel diameter 65 mm — диаметр 65 мм | meters per tick calc
26. track width 560 mm — колея 560 мм | kinematics parameter
27. solder joint reliability — надёжность пайки | intermittent faults
28. fuse / breaker practice — предохранитель/автомат | overcurrent protection
29. status heartbeat LED idea — идея LED heartbeat | alive indication
30. e-stop hardware gap — разрыв аппаратного e-stop | still to harden for public

7.4 Navigation (30)

1. occupancy grid — сетка занятости | free/occupied/unknown cells
2. log-odds update — обновление log-odds | probabilistic cell updates
3. A→B escort — сопровождение A→B | goal navigation for passenger
4. calibrated destination — откалиброванная точка | measured x,y in map frame
5. uncalibrated template — неоткалиброванный шаблон | intentionally invalid coords
6. remembered map — запомненная карта | reloaded occupancy
7. scan matching — сопоставление сканов | pose refinement
8. CSM — CSM | correlative scan matching
9. ICP — ICP | iterative closest point polish
10. pure pursuit — pure pursuit | look-ahead path tracker
11. cross-track error — поперечная ошибка | lateral path deviation
12. yaw hold — удержание курса | LiDAR-aided heading stability
13. temporary obstacle layer — слой временных препятствий | short TTL blocks
14. inflation / clearance — инфляция / зазор | safety buffer around hits
15. footprint radius ~0.5 m — радиус габарита ~0.5 м | planning radius
16. goal tolerance — допуск цели | arrival criterion
17. replan on block — переплан при блокировке | new A* when path dies
18. operator goal API — API цели оператора | staff sets goal
19. escort API — API escort | kiosk starts escort
20. map frame consistency — единство map frame | destinations match map
21. odom frame smoothness — гладкость odom | local continuity
22. self-filter / frame rack filter — самофильтр / фильтр стоек | ignore robot body returns
23. narrow passage handling — узкие проходы | may need slow/supervised
24. glass weakness — слабость на стекле | LiDAR limitation
25. carpet slip — проскальзывание на ковре | floor-dependent calib
26. progress failure — нет прогресса | stuck detection need
27. teleop takeover — перехват телеопом | supervisor control
28. stop on fault — стоп при отказе | safe default
29. route preview — предпросмотр маршрута | show before go
30. navigation honesty line — честная линия навигации | program exists; airport not claimed

7.5 Algorithms (30)

1. **A* grid search** — поиск A* по сетке | global path
2. Manhattan heuristic — манхэттенская эвристика | admits grid moves
3. 8-connected neighbors — 8-связные соседи | diagonal steps
4. pure pursuit look-ahead — look-ahead pure pursuit | target point selection
5. CTE correction term — член коррекции CTE | lateral fix
6. yaw P term — P-член по курсу | heading fix
7. mecanum inverse kinematics — обратная кинематика механумов | vx,vy,w → wheels
8. three-wheel odometry integrate — интеграция одометрии 3 колёс | pose from trusted ticks
9. PI on tick rate — ПИ по tick rate | wheel synchronization
10. log-odds hit/miss model — модель hit/miss log-odds | beam updates
11. Bresenham-style ray walk — обход луча | cells along beam
12. CSM pose hypotheses — гипотезы позы CSM | score many poses
13. ICP refinement loops — итерации ICP | local polish
14. temporary TTL decay — затухание TTL | clear stale dynamics
15. BarcodeDetector decode — декодирование BarcodeDetector | ticket codes
16. YuNet detection — детекция YuNet | face boxes
17. digital center crop — цифровой center crop | keep face in view

18. Haar fallback (Tapo path) — Haar fallback | classical detector option
19. allowlist predicate — предикат allowlist | IP permitted?
20. lease expiry timer — таймер истечения lease | stop escort
21. stale command zeroing — обнуление stale cmd | watchdog
22. fail-closed destination parse — fail-closed разбор точек | invalid → no escort
23. ticket status filter — фильтр статусов билета | valid/checked-in/boarding
24. demo ticket fallback — fallback демо-билетов | lab continuity
25. map autosave — автосохранение карты | persist occupancy
26. path cost = distance + penalties — стоимость пути | planner scoring
27. arrival check — проверка прибытия | goal reached
28. velocity saturation — насыщение скорости | respect max vx/vy/w
29. scan score gate idea — идея порога score скана | distrust bad matches
30. manual code parse — разбор ручного кода | UX fallback

7.6 Safety (30)

1. Arduino command timeout — таймаут команд Arduino | ~1.2 s stop
2. drive watchdog — drive watchdog | ~0.25 s stale zero
3. kiosk heartbeat lease — heartbeat lease киоска | ~6 s stop escort
4. UI blur / visibility stop — стоп при blur/visibility | operator UI safety
5. fail-closed escort — fail-closed escort | no bad goals
6. motion as privilege — движение как привилегия | must be continually granted
7. layered safety — эшелонированная безопасность | MCU+SW+session
8. supervisor stop authority — право стопа у супервизора | human override
9. private LAN preference — предпочтение частной LAN | not public Wi-Fi control
10. allowlist / origin gates — шлюзы allowlist/origin | reduce remote misuse
11. coordinates not to browser — координаты не в браузер | privacy/safety of map data
12. speed near people policy — политика скорости у людей | ops rule
13. keep-out / evacuation awareness — keep-out / эвакуация | map+SOP
14. residual risk acceptance — принятие остаточного риска | honest
15. indication gap — разрыв индикации | lights/sounds still needed
16. enclosure gap — разрыв корпуса | soft covers still needed
17. battery brownout hazard — опасность просадки АКБ | power monitoring needed
18. incident logging need — нужда в журнале инцидентов | ops maturity
19. no ISO claim — без заявления ISO | don't fake certification
20. teleop as safety net — телеоп как страховка | not the whole product
21. stop-latency test — тест задержки стопа | evidence ritual
22. cable-pull mental model — ментальная модель выдёргивания кабеля | fault injection
23. session abandonment — брошенная сессия | lease solves
24. unsafe overclaim ban — запрет небезопасных оверклеймов | culture
25. child/pet unpredictability — непредсказуемость детей/животных | HRI risk
26. glass false free space — лобовое свободное на стекле | perception risk
27. crowd intervention count — число вмешательств в толпе | KPI
28. SOP one-pager — одностраничный SOP | staff usability
29. pilot zone boundary — граница пилотной зоны | ops constraint
30. airside non-claim — незаявление airside | legal/ops humility

7.7 Interface / Kiosk (30)

1. /kiosk route — маршрут /kiosk | passenger UI
2. /operator route — маршрут /operator | staff UI
3. six primary buttons — шесть главных кнопок | fast destinations/actions

4. RU/EN/ZH/TT locales — локали RU/EN/ZH/TT | multilingual
5. Scan ticket dialog — диалог Scan ticket | camera + manual
6. flight info card — карточка рейса | time/gate/desk fields
7. escort CTA — CTA сопровождения | start guidance
8. busy/lease state — состояние busy/lease | concurrency UX
9. touch 1280×800 layout — раскладка 1280×800 | panel optimized
10. portrait adaptation — адаптация portrait | responsive
11. HID wedge scanner support — поддержка HID-сканера | keyboard wedge
12. manual code field — поле ручного кода | fallback
13. destination labels — подписи назначений | human-readable places
14. kind: check-in/gates/baggage... — kind назначений | semantic classes
15. operator map view — карта оператора | see occupancy/path
16. teleop controls — элементы телеопа | manual drive
17. language switcher — переключатель языка | inclusion
18. error toast / unavailable escort — ошибка / escort недоступен | fail-closed UX
19. heartbeat POSTs — POST heartbeat | keep lease alive
20. preview without ROS — preview без ROS | monitor/preview_server
21. no raw xy to passenger — без сырых ху пассажиру | safety/privacy
22. demo ticket familiarity — знакомство с демо-кодами | lab rehearsal
23. clear primary action — ясное primary action | UX
24. reduce cognitive load — снижение когнитивной нагрузки | elderly-friendly
25. large tap targets — крупные зоны нажатия | touch UX
26. status copy honesty — честные статусные тексты | don't say 'airport live' falsely
27. hand-off to human wording — формулировки передачи человеку | ops UX
28. queue/busy future copy — тексты очереди/busy | future
29. accessibility font size — размер шрифта a11y | readability
30. brand AeroPorter — бренд AeroPorter | recognition

7.8 Limitations (30)

1. not certified airport robot — не сертифицированный робот аэропорта | status
2. no real-terminal claim — без заявления о реальном терминале | honesty
3. Nav2 not used — Nav2 не используем | stack truth
4. LiDAR SKU unnamed — SKU лидара не назван | driver only
5. FR encoder untrusted — FR энкодер недоверен | hardware
6. destinations need measuring — точки нужно измерять | calibration
7. demo tickets possible — возможны демо-билеты | lab
8. BarcodeDetector dependency — зависимость от BarcodeDetector | browser
9. glass/dark surface weakness — слабость стекло/чёрное | perception
10. crowd not fully validated — толпа не полностью валидирована | field
11. enclosure incomplete — корпус не завершён | mechanical
12. indication incomplete — индикация не завершена | HRI
13. cost excludes LiDAR/enclosure — цена без лидара/корпуса | BOM
14. floor-dependent calibration — калибровка от пола | ops
15. LAN allowlist ≠ zero trust — allowlist ≠ zero trust | security
16. Tapo path separate — путь Tapo отдельный | scope
17. no biometric boarding claim — без биометрической посадки | privacy
18. no airside claim — без airside | ops/legal
19. map quality operator-dependent — качество карты от оператора | process
20. USB enumeration fragility — хрупкость USB enumeration | ops

21. soft real-time only — только soft real-time | timing
22. statistical N may be small — малая выборка метрик | evidence
23. insurance/legal outside software — страховка/право вне софта | deployment
24. single robot not campus-wide — один робот ≠ весь кампус | scale
25. live FIDS not finished claim — live FIDS не заявлен готовым | integration
26. voice not core proven — голос не core-proven | UX
27. fleet absent — флота нет | scale
28. industrial AMR class not reached — класс industrial AMR не достигнут | TRL
29. security hardening ongoing — hardening безопасности впереди | security
30. human SOP required — нужен человеческий SOP | ops

7.9 Development plans (30)

1. industrial enclosure — промышленный корпус | next hardware
2. soft bumpers + lights — мягкие бамперы + огни | HRI safety
3. hardware e-stop — аппаратный e-stop | safety
4. supervised pilot zone — supervised пилотная зона | deployment
5. measure stop latency on video — замер stop latency на видео | evidence
6. crowd-like field tests — полевые тесты «как толпа» | validation
7. live flight data adapter — адаптер live-данных рейсов | integration
8. stronger API auth — усиление auth API | security
9. network segmentation guide — гайд сегментации сети | IT
10. FR encoder hardware fix — аппаратный фикс FR | reliability
11. optional Jetson path — опциональный путь Jetson | compute
12. evaluate Nav2 migration criteria — критерии миграции на Nav2 | stack
13. richer local planner if needed — более богатый local planner | nav
14. battery SoC monitoring — мониторинг SoC | power
15. acoustic + speed human-factors — акустика + скорость HF | comfort
16. accessibility audit — аудит доступности | a11y
17. staff training one-pager — обучение персонала one-pager | ops
18. incident severity scale — шкала серьёзности инцидентов | ops
19. metric dashboard — панель метрик | observability
20. map versioning discipline — дисциплина версий карт | ops
21. destination sign-off workflow — workflow утверждения точек | process
22. privacy review for PII — privacy review PII | legal
23. spare parts kit — комплект ЗИП | maintainability
24. soak tests for heat/battery — soak-тесты тепло/АКБ | reliability
25. glass mitigation experiments — эксперименты против стекла | perception
26. queue UX for second passenger — UX очереди второго пассажира | product
27. fleet later — not now — флот позже — не сейчас | scope control
28. TRL tracking — учёт УГТ/TRL | maturity
29. partner airport conversation — разговор с аэропортом-партнёром | go-to-market
30. claim lint before every show — lint заявлений перед каждым шоу | culture

AeroPorter vocabulary total: 270

8. Fully Ready Defense Script

8.1 Opening (60–90 seconds)

EN (speak): Good afternoon. We are the AeroPorter team — Ilya Karyakin, Stanislav Paramonov, and Artur Sabirzanov. AeroPorter is a mobile robot assistant for airport terminals. Our goal is simple: scan a ticket, show flight information, and escort a passenger from point A to point B. The robot runs on a Raspberry Pi 5 with an Arduino Mega, mecanum wheels, LiDAR mapping, and a multilingual touch kiosk. Important honesty note up front: the navigation software works in our controlled setup; we do not claim certified operation in a live commercial terminal yet.

RU paraphrase: Представьтесь, назовите цель «билет → информация → сопровождение», железо кратко, и сразу честно скажите про статус прототипа.

8.2 Slide-by-slide narration

Slide 1: Title

Spoken EN: AeroPorter — mobile airport assistant. Ticket, information, escort. Team and supervisor. Key terms: AeroPorter, prototype, escort Transition: Let me start with the problem passengers face.

Slide 2: Problem

Spoken EN: Airports are stressful. Gates change, walking distances are long, and language barriers hurt elderly passengers, families, and foreign travelers. Signs exist, but help is not always beside you when you need it. Key terms: stress, gate change, accessibility Transition: So here is our solution.

Slide 3: Solution

Spoken EN: AeroPorter scans a boarding-pass code, shows flight details on a 10.1-inch screen, and can lead the passenger along a calibrated route. It is a moving guide, not only a static kiosk. Key terms: scan, kiosk, A→B Transition: How does this compare with existing robots?

Slide 4: Analogs

Spoken EN: Pepper, Sanbot, and Promobot are strong service robots. Fixed kiosks are everywhere. Our difference is a ticket-aware escort focus on affordable, fully explainable hardware — not a claim that we beat every commercial platform at everything. Key terms: analogs, task fit, cost class Transition: Now the architecture behind that idea.

Slide 5: Architecture

Spoken EN: Four modules: Arduino chassis bridge, lidar_map navigation brain with HTTP UI, camera stack, and monitor kiosk assets. Pi handles high-level logic; Mega handles motors with a timeout fail-safe. Key terms: modular, Pi, Mega Transition: Let's zoom into navigation.

Slide 6: Navigation

Spoken EN: We build an occupancy map from LiDAR, refine pose with scan matching, plan with A-star, and track with pure pursuit. This is a custom ROS 2 stack — we are not claiming Nav2. Key terms: occupancy, A*, pure pursuit Transition: Next — what the passenger actually sees.

Slide 7: Interface

Spoken EN: The kiosk supports Russian, English, Chinese, and Tatar. Passengers can scan a barcode or type a code. Escort starts only if destinations are calibrated; otherwise the system fails closed and still can show information. Key terms: multilingual, fail-closed Transition: Here are the internals of the prototype.

Slide 8: Internals

Spoken EN: You can see the compute, wiring, LiDAR placement, and drive electronics. This slide is evidence that the system is a real integrated robot, not a slide-only concept. Key terms: wiring, LiDAR, Mega Transition: How did we develop it?

Slide 9: Development

Spoken EN: We iterated module by module: drive bring-up, mapping, planner/tracker, then kiosk workflow. Independent folders reduced merge pain and made testing safer. Key terms: iterate, modular bring-up Transition: Safety is non-negotiable — next slide.

Slide 10: Safety

Spoken EN: Layered stops: Arduino timeout about 1.2 seconds, drive watchdog about 0.25 seconds, kiosk heartbeat lease about 6 seconds. Motion is a privilege that expires. Public indication and soft covers are still required before real exploitation. Key terms: timeout, lease, residual risk Transition: What works today, and what is next?

Slide 11: Results & plans

Spoken EN: Working today: teleop, mapping, A→B in controlled conditions, kiosk demo flows. Next: enclosure, human-factors indication, supervised pilot metrics, selective integrations. We will not inflate claims to sound 'finished'. Key terms: works now / next Transition: I will close with our thesis.

Slide 12: Closing

Spoken EN: AeroPorter turns a ticket into guided motion with transparent engineering and honest limits. We are ready for your questions — including requests to explain stop behavior and the navigation pipeline. Key terms: honest thesis, Q&A ready Transition: Thank you.

8.3 Closing (45–60 seconds)

EN: To conclude: AeroPorter is a Raspberry Pi 5 + Arduino Mega mecanum prototype with a custom ROS 2 LiDAR navigator and a multilingual passenger kiosk. We can demonstrate ticket-to-info flow and A→B motion under supervised conditions. We explicitly do not claim certified live-terminal deployment. Our next step is a supervised pilot with measured stop latency, enclosure, and operational SOPs. Thank you — we welcome your questions.

8.4 Typical post-presentation Q&A (15 exchanges)

1. Q: What is new here?

A: The vertical integration: ticket context + custom navigator + layered stops on affordable hardware we can fully explain.

2. Q: Does it work in the real airport?

A: Not as a certified claim. Controlled demos yes; live terminal requires a supervised pilot and authority process.

3. Q: Why not Nav2?

A: We needed transparency for defense and education; custom A* + pursuit is intentional.

4. Q: How do you stop on disconnect?

A: Heartbeat lease, drive watchdog, and Mega timeout — layered.

5. Q: Why Pi not Jetson?

A: Current CPU workloads fit Pi 5; Jetson is optional later.

6. Q: What about glass?

A: Known LiDAR limit; slow, supervise, and plan sensor diversity later.

7. Q: Cost?

A: ~18,000 € prototype base excluding LiDAR and industrial enclosure.

8. Q: Is FR encoder a failure?

A: It is a diagnosed limitation mitigated by three-wheel odometry.

9. Q: Better than Pepper?

A: Different class: we optimize ticket→escort explainability/cost, not humanoid social acting.

10. Q: Privacy?

A: No biometric boarding claim; don't send map coordinates to the browser; real PII needs legal controls.

11. Q: Who supervises?

A: Trained staff with stop authority in a defined pilot zone.

12. Q: What is the KPI?

A: Escort completion, stop latency under faults, interventions per hour.

13. Q: Show algorithm?

A: Occupancy update → A* → pure pursuit → cmd_vel/serial → PWM; scan match refines pose.

14. Q: Languages?

A: RU, EN, ZH, TT — inclusion for local and international passengers.

15. Q: Next step?

A: Enclosure + indication + measured pilot metrics before feature inflation.

9. Mock Interviews

12 full rehearsals. Practice aloud. Time answers: 20–40s default.

Mock Interview 1: General defense warm-up

Questions

1. What is AeroPorter?
2. Who are the users?
3. What is implemented vs planned?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. What is your thesis sentence?
5. What evidence will you show?
6. What is out of scope?
7. How did you split roles?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. What was hardest?
9. What would you redo?
10. How do you demo in 3 minutes?
11. What ethical issue matters most?
12. How do you keep claims honest?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. What is the elevator pitch?
14. What slide is load-bearing?
15. What risk keeps you up at night?
16. How do you define success today?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. What is the ask after defense?
18. How do you handle unknown questions?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 2: Hardware deep dive

Questions

1. Why Pi 5?
2. Why Mega?
3. Why mecanum?

- Follow-up: Can you prove it with a measurement or code path?
- Follow-up: What is the limitation behind your answer?
- 4. Encoder trust model?
- 5. LiDAR connection?
- 6. Footprint?
- 7. Power architecture?
- Follow-up: Can you prove it with a measurement or code path?
- Follow-up: What is the limitation behind your answer?
- 8. USB enumeration issues?
- 9. Thermal limits?
- 10. How to flash Mega?
- 11. Motor driver topology?
- 12. Self-hit mounting?
- Follow-up: Can you prove it with a measurement or code path?
- Follow-up: What is the limitation behind your answer?
- 13. Spare parts?
- 14. Wheel diameter effect?
- 15. Baud rates?
- 16. Brownout scenarios?
- Follow-up: Can you prove it with a measurement or code path?
- Follow-up: What is the limitation behind your answer?
- 17. Enclosure readiness?
- 18. E-stop hardware gap?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 3: Software & ROS 2

Questions

1. Port 8765 role?
2. Why not Nav2?
3. A* where in code conceptually?
- Follow-up: Can you prove it with a measurement or code path?
- Follow-up: What is the limitation behind your answer?
4. Pure pursuit inputs?
5. Heartbeat lease?
6. Watchdog?
7. SQLite path?
- Follow-up: Can you prove it with a measurement or code path?
- Follow-up: What is the limitation behind your answer?
8. Fail-closed destinations?

9. Allowlist?
10. cmd_vel vs JSON IPC?
11. Ticket statuses?
12. Barcode fallback?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Map persistence?
14. How test planners?
15. Concurrency operator/kiosk?
16. Logging?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Launch scripts?
18. Biggest software risk?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 4: Navigation grilling

Questions

1. Localization vs mapping?
2. Is it SLAM?
3. CSM vs ICP?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. CTE?
5. Look-ahead trade-off?
6. Inflation?
7. Temporary obstacles?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. Yaw hold?
9. Replan triggers?
10. Goal tolerance?
11. Glass problem?
12. Carpet problem?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Holonomic meaning?
14. Progress failure?
15. Map frame?

16. Odometry drift?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Teleop override?
18. KPI for nav?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 5: Electronics & power

Questions

1. PWM role?
2. H-bridge shoot-through concept?
3. Quadrature direction?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. EMI on encoders?
5. Common ground?
6. Logic vs motor rails?
7. Inrush?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. Fusing?
9. USB power budget?
10. Baud mismatch symptom?
11. Logic levels Pi/Mega?
12. Debounce vs encoders?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Heat on Pi?
14. Measuring current?
15. Connector failures?
16. Firmware heartbeat?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Power budget numbers approach?
18. Safe bring-up order?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.

21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 6: Economics & analogs

Questions

1. BOM ~18k meaning?
2. Exclusions?
3. CAPEX/OPEX pilot?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. Compare Pepper cost class?
5. Who is buyer?
6. TCO 3 years?
7. Maintenance budget?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. Downtime cost?
9. Insurance?
10. Descope under cuts?
11. Price a pilot?
12. Open-source cost effect?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Most expensive subsystem?
14. Payback narrative?
15. Cheap prototype vs product?
16. Funding next stage?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Staffing savings claim care?
18. Metric for value?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 7: Safety & failure modes

Questions

1. Layered stops?
2. Mega timeout proof?
3. Lease loss proof?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. Fail-closed why?
5. Remote misuse?
6. Speed near people?
7. Evacuation paths?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. Residual risks?
9. ISO claim?
10. Indication gap?
11. Battery hazard?
12. Incident logs?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Child grabs UI?
14. Stop authority?
15. Cable-pull test?
16. Unsafe sentence you refuse?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Crowd policy?
18. SOP contents?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 8: Real deployment & ops

Questions

1. Pilot zone where?
2. Landside/airside?
3. Supervisor model?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. KPI 30 days?
5. Wi-Fi policy?
6. Gate change mid-escort?

7. Staff training?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. FIDS integration need?
9. PRM honesty?
10. Cleaning robots conflict?
11. Peak wave behavior?
12. Map after renovation?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Languages locally?
14. Handoff to human?
15. Success definition?
16. IT segmentation?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Permissions needed?
18. What not to pilot yet?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 9: Computer vision & tickets

Questions

1. Why ticket entry point?
2. Why escort?
3. Why not only phone maps?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. Multilingual value?
5. Voice needed?
6. MVP journey?
7. Critical assumption?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. Avoid scope creep?
9. Airline benefit?
10. Airport benefit?
11. Emotional goal?
12. YuNet vs biometrics?
 - Follow-up: Can you prove it with a measurement or code path?

- Follow-up: What is the limitation behind your answer?
13. BarcodeDetector limits?
 14. Demo tickets ethics?
 15. Privacy of names?
 16. Second passenger?
- Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Queue UX?
 18. What kills the idea?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 10: Roadmap & TRL

Questions

1. Enclosure first why?
 2. Jetson when?
 3. Nav2 when?
- Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. Live APIs how?
 5. Fleet when not?
 6. Glass experiments?
 7. Auth hardening?
- Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. SoC monitor?
 9. TRL move evidence?
 10. Partner first?
 11. A11y audit?
 12. Metric dashboard?
- Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Map versioning?
 14. Hardware e-stop?
 15. Voice later?
 16. Avoid hype roadmap?
- Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Five-year non-hype vision?
 18. First experiment after defense?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 11: Hostile jury simulation

Questions

1. Layered stops?
2. Mega timeout proof?
3. Lease loss proof?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. Fail-closed why?
5. Remote misuse?
6. Speed near people?
7. Evacuation paths?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. Residual risks?
9. ISO claim?
10. Indication gap?
11. Battery hazard?
12. Incident logs?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Child grabs UI?
14. Stop authority?
15. Cable-pull test?
16. Unsafe sentence you refuse?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Crowd policy?
18. SOP contents?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.

- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

Mock Interview 12: Mixed technical + business

Questions

1. BOM ~18k meaning?
2. Exclusions?
3. CAPEX/OPEX pilot?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
4. Compare Pepper cost class?
5. Who is buyer?
6. TCO 3 years?
7. Maintenance budget?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
8. Downtime cost?
9. Insurance?
10. Descope under cuts?
11. Price a pilot?
12. Open-source cost effect?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
13. Most expensive subsystem?
14. Payback narrative?
15. Cheap prototype vs product?
16. Funding next stage?
 - Follow-up: Can you prove it with a measurement or code path?
 - Follow-up: What is the limitation behind your answer?
17. Staffing savings claim care?
18. Metric for value?

Unexpected curveballs

19. If your LiDAR returned free space through glass right now, what do you do in the next 5 seconds?
20. A jury member says your cost slide is misleading — respond.
21. They ask you to compare yourself to an industrial AMR in one minute — without hype.

Coaching tips

- Lead with a decision sentence, then one mechanism, then one limit.
- Hand off explicitly to teammates by subsystem.
- Offer a stop-latency or fail-closed demonstration when challenged on safety.
- If you don't know: say how you would measure it.
- Keep the shared truth table consistent (no Nav2, cspc_lidar, FR untrusted, no live-terminal claim).

10. Most Common Mistakes

10.1 Wrong engineering terms

1. ☐ 'We use Nav2' when you don't → ☐ 'Custom ROS 2 occupancy + A* + pure pursuit' | false stack claims collapse instantly
2. ☐ 'Full SLAM like Cartographer' → ☐ 'Occupancy mapping with scan matching' | don't oversell
3. ☐ 'RPLIDAR A1/A3' without evidence → ☐ 'LiDAR via cspc_lidar driver' | inventing brands destroys trust
4. ☐ 'Hard real-time Linux' → ☐ 'Soft real-time control with MCU timeout backup' | precision
5. ☐ 'AI decides the route' → ☐ 'A* plans; pure pursuit tracks' | name algorithms
6. ☐ 'Biometric boarding' → ☐ 'Face centering for UX, not identity' | ethics
7. ☐ 'Autonomous in any airport' → ☐ 'Supervised prototype in controlled conditions' | safety
8. ☐ 'Odometry from four encoders' → ☐ 'Three trusted wheels; FR untrusted' | honesty
9. ☐ 'Jetson-class performance on Pi' → ☐ 'Pi 5 fits current CPU workload' | realism
10. ☐ 'Production-ready' → ☐ 'Prototype-grade, pilot-ready only after enclosure/SOP' | TRL
11. ☐ 'Ultrasonic SLAM' → ☐ 'LiDAR occupancy mapping' | sensor class
12. ☐ 'GPS indoor navigation' → ☐ 'GPS-denied LiDAR map navigation' | domain
13. ☐ 'Machine learning path planning' → ☐ 'Classical A* on a grid' | don't invent ML
14. ☐ 'The cloud controls motors' → ☐ 'Local HTTP/ROS control with timeouts' | architecture
15. ☐ '100% accurate localization' → ☐ 'Adequate under mapped structured conditions' | metrics
16. ☐ 'Zero risk' → ☐ 'Mitigated residual risk with layered stops' | safety literacy
17. ☐ 'ISO certified' → ☐ 'Not certified; designing toward safer ops' | legal
18. ☐ 'ROS is the autopilot' → ☐ 'ROS is middleware' | conceptual
19. ☐ 'Mecanum = no calibration' → ☐ 'Mecanum needs careful calibration' | practice
20. ☐ 'Watchdog optional' → ☐ 'Watchdog/lease are core safety' | priorities
21. ☐ 'Fail-open escort' → ☐ 'Fail-closed escort' | safety philosophy
22. ☐ 'Map coordinates to phone UI' → ☐ 'Flight info only to browser' | privacy/safety
23. ☐ 'Teleop means not autonomous' → ☐ 'Teleop is supervision layer around autonomy blocks' | framing
24. ☐ 'Demo tickets = fake project' → ☐ 'Lab scaffolding pending controlled data integration' | maturity
25. ☐ 'One timeout is enough' → ☐ 'Layered timeouts MCU+SW+session' | defense-in-depth
26. ☐ 'Accuracy 1 cm' without method → ☐ 'Method + conditions + error bars' | scientific
27. ☐ 'We beat Pepper overall' → ☐ 'Better task fit on cost/explainability for escort' | comparison hygiene
28. ☐ 'Airsides ready' → ☐ 'No airside claim' | ops law
29. ☐ 'Always works on glass' → ☐ 'Glass is a known LiDAR weakness' | perception
30. ☐ 'Firmware on Pi motors directly' → ☐ 'Mega owns PWM/encoders' | architecture
31. ☐ 'Quaternion ignored / yaw only vibes' → ☐ 'SE(2) pose x,y,yaw in map/odom frames' | robotics literacy
32. ☐ 'Occupancy = photo map' → ☐ 'Probabilistic grid from rays' | mapping literacy
33. ☐ 'Pure pursuit = A*' → ☐ 'A* plans; pursuit tracks' | separation
34. ☐ 'Heartbeat = ping only' → ☐ 'Lease granting motion privilege' | security semantics
35. ☐ 'Allowlist = full security' → ☐ 'Necessary but not sufficient' | security maturity
36. ☐ 'CSM = neural net' → ☐ 'Correlative scan matching' | terms
37. ☐ 'ICP = path planner' → ☐ 'Pose alignment refinement' | terms
38. ☐ 'Baud rate doesn't matter' → ☐ 'Wrong baud = garbage serial' | electronics
39. ☐ 'Brownout is software bug' → ☐ 'Often power architecture' | root cause
40. ☐ 'Multilingual = Google Translate live' → ☐ 'Shipped locale strings RU/EN/ZH/TT' | product truth
41. ☐ 'Cost 18k all-in product' → ☐ 'Prototype BOM excluding LiDAR/enclosure' | economics
42. ☐ 'Encoder ticks = meters directly' → ☐ 'Convert via diameter & tpr' | units
43. ☐ 'Inflation = map cosmetics' → ☐ 'Safety clearance for footprint' | planning
44. ☐ 'Unknown space is free' → ☐ 'Unknown is not proven free' | safety
45. ☐ 'Stop latency unmeasured is fine' → ☐ 'Measure cable-pull / lease-loss' | V&V

10.2 False friends / переводческие ловушки

1. аккуратный → □ accurate → □ neat/careful (accuracy = точность измерения)
2. техника → □ technique only → □ also hardware/equipment depending on context
3. контроль → □ control (always) → □ inspection/monitoring sometimes; 'motor control' is OK
4. оператор → □ surgeon → □ operator (staff) for robot console
5. кабинет → □ cabinet → □ office; for electronics 'enclosure/cabinet' carefully
6. магазин → □ magazine → □ shop; 'magazine' is storage for ammo/articles
7. реализация → □ realization → □ implementation
8. разработка → □ development/exploration mix → □ development (software) / design
9. доказательство → □ only proof math → □ evidence/argument also
10. акцент → □ accent only → □ emphasis ('we emphasize safety')
11. физика → □ physical education → □ physics
12. данные → □ data are singular habit → □ data (often plural agreement in formal EN)
13. информация → □ informations → □ information (uncountable)
14. софт → □ soft → □ software
15. хард → □ hard → □ hardware
16. мышка → □ mouse animal only → □ mouse (computer) clear by context
17. камера → □ room → □ camera (vision) / chamber if room
18. драйвер → □ driver person always → □ driver (software) for LiDAR/motor
19. карта → □ card → □ map (navigation) / card (boarding) — disambiguate
20. билет → □ bill → □ ticket
21. выход → □ exit only → □ gate (airport boarding gate) often
22. стойка → □ stand ambiguous → □ check-in desk
23. лента → □ ribbon → □ baggage carousel/belt
24. сеть → □ net → □ network
25. узел → □ knot → □ node (ROS)
26. топик → □ topic essay → □ topic (ROS)
27. путь → □ path/way mix → □ path (planner) / route (human)
28. цель → □ goal/aim/target mix → □ goal pose (nav) / aim (intent)
29. точность → □ precision/accuracy mix → □ accuracy vs precision correctly
30. безопасность → □ security/safety mix → □ safety (harm) vs security (attackers)
31. защита → □ protection/defense mix → □ defense (project) / protection (safety)
32. испытание → □ test/trial/temptation → □ test/trial
33. опыт → □ experience/experiment → □ experience vs experiment
34. заказчик → □ customer/client mix → □ customer/stakeholder (airport)
35. персонал → □ personnel/staff → □ staff/personnel
36. очередь → □ queue/line/turn → □ queue
37. задержка → □ delay/latency mix → □ delay (flight) vs latency (system)
38. производительность → □ productivity/performance → □ performance (system)
39. мощность → □ power/capacity mix → □ power (watts) / performance carefully
40. сопротивление → □ resistance/resistor → □ resistor (part) / resistance (physics)
41. схема → □ scheme (sinister) → □ circuit diagram / schematic
42. плата → □ payment plate → □ PCB / board
43. провод → □ wire/conduct → □ wire/cable
44. питание → □ nutrition → □ power supply
45. заземление → □ grounding/earth → □ ground/earth
46. помехи → □ interference/noise → □ noise/EMI
47. прерывание → □ interruption social → □ interrupt (ISR)
48. поток → □ flow/thread/stream → □ disambiguate thread vs stream vs passenger flow

49. кадр → □ frame/shot → □ frame (image)
50. разрешение → □ resolution (image) / permission — don't mix

10.3 Typical answer mistakes

1. Inventing numbers → Measure or refuse the number
2. Answering a different question → Mirror the question words first
3. Too long preamble → Decision sentence in first 5 seconds
4. Contradicting a teammate → Use shared truth table
5. Defensive ego → Thank, then reframe with evidence
6. Hiding FR encoder issue → Document mitigation proudly
7. Saying 'Nav2' by accident → Correct yourself immediately
8. Overclaiming airport deployment → Restate prototype boundary
9. Ignoring safety when asked UX → Bridge to stop layers
10. Buzzword 'AI' spam → Name YuNet/A*/ICP
11. Cost without exclusions → Always list exclusions
12. 'We'll fix later' on safety → Present mitigation or non-claim
13. Reading slides verbatim → Speak thesis-level sentences
14. No handoff → 'I'll pass to ... for electronics'
15. Guessing LiDAR brand → Say driver name
16. Promising airside → Explicitly decline
17. Comparing unfairly to Pepper → State metric and class
18. Forgetting fail-closed → Mention calibrated destinations
19. No example → Give one concrete failure test
20. Panic silence → Buy time phrase, then structure
21. Interrupting jury → Let them finish
22. Speaking only Russian terms → Practice EN names of modules
23. Saying 'always' → Prefer 'in our tests / under conditions'
24. Mixing security vs safety → Split the answer
25. Skipping units → Add seconds/meters/ baud
26. Answering follow-up before main → Finish, then dig
27. Blaming tools only → Show your design response
28. No roadmap boundary → Say what is NOT next
29. Mocking analogs → Respect + differentiate
30. Ending without check → 'Does that answer your question?'
31. Showing secret keys in demo → Redact
32. Claiming GDPR compliance casually → Describe practices, not fake certification

10.4 Words/phrases better NOT to use

1. 'always' → overclaim → 'in our tested conditions'
2. 'never fails' → impossible → 'fails safe to stop'
3. 'AI magic' → empty → name algorithms
4. 'completely autonomous' → misleading → 'supervised autonomy blocks'
5. 'production-ready' → TRL false → 'prototype'
6. 'certified' → legal false → 'not certified'
7. 'like Tesla Autopilot' → bad analogy → indoor escort robot
8. 'we use Nav2' → false → custom stack
9. 'military-grade' → cringe → concrete IP/impact ratings if any
10. 'unlimited range' → false → battery/map limits
11. '100% accurate' → false → method + margin
12. 'no risks' → false → residual risks

13. 'hack-proof' → false → mitigations
 14. 'works in any airport tomorrow' → false → pilot requirements
 15. 'better than all robots' → arrogant → task-fit comparison
 16. 'just a toy' → undersell → engineering prototype
 17. 'trivial' → insults complexity → 'constrained but nontrivial'
 18. 'obviously' → condescending → explain
 19. 'you don't understand' → hostile → rephrase simpler
 20. 'secret sauce' → unscientific → mechanisms
 21. 'trust me' → weak → show test
 22. 'probably safe enough' → vague → timeouts + measurements
 23. 'cheap Chinese junk' → unprofessional → discuss component class politely
 24. 'passengers are dumb' → unethical → accessibility framing
 25. 'GDPR solved' → false → ongoing compliance work
 26. 'ISO soon' without plan → vapor → list next safety artifacts
 27. 'Jetson makes it smart' → cargo cult → workload justification
 28. 'cloud will fix it' → handwave → local safety first
 29. 'SLAM' as flex word → unclear → define your mapping/localization
 30. 'realtime' casually → ambiguous → soft vs hard
 31. 'infinite scalability' → hype → single-robot MVP now
 32. 'zero cost OPEX' → false → supervision/maintenance
 33. 'we copied Pepper' → undermines → inspired by category, different stack
 34. 'don't ask about limitations' → fatal → invite limitations
 35. 'because ChatGPT said' → credibility death → engineering reasoning
 36. 'random constants' → weak → 'tuned then tested'
 37. 'temporary forever' → joke bad → roadmap owners/dates
 38. 'airside is fine' → dangerous → non-claim
 39. 'encoders lie but OK' → careless → mitigation story
 40. 'security later' → reckless → baseline now + roadmap
-

11. Useful Add-ons

11.1 Linking words (110)

1. however — e.g., 'However, AeroPorter fails closed when destinations are uncalibrated.'
2. therefore — e.g., 'Therefore, AeroPorter fails closed when destinations are uncalibrated.'
3. consequently — e.g., 'Consequently, AeroPorter fails closed when destinations are uncalibrated.'
4. furthermore — e.g., 'Furthermore, AeroPorter fails closed when destinations are uncalibrated.'
5. moreover — e.g., 'Moreover, AeroPorter fails closed when destinations are uncalibrated.'
6. nevertheless — e.g., 'Nevertheless, AeroPorter fails closed when destinations are uncalibrated.'
7. nonetheless — e.g., 'Nonetheless, AeroPorter fails closed when destinations are uncalibrated.'
8. accordingly — e.g., 'Accordingly, AeroPorter fails closed when destinations are uncalibrated.'
9. similarly — e.g., 'Similarly, AeroPorter fails closed when destinations are uncalibrated.'
10. likewise — e.g., 'Likewise, AeroPorter fails closed when destinations are uncalibrated.'
11. in contrast — e.g., 'In contrast, AeroPorter fails closed when destinations are uncalibrated.'
12. on the other hand — e.g., 'On the other hand, AeroPorter fails closed when destinations are uncalibrated.'
13. for example — e.g., 'For example, AeroPorter fails closed when destinations are uncalibrated.'
14. for instance — e.g., 'For instance, AeroPorter fails closed when destinations are uncalibrated.'
15. in particular — e.g., 'In particular, AeroPorter fails closed when destinations are uncalibrated.'
16. specifically — e.g., 'Specifically, AeroPorter fails closed when destinations are uncalibrated.'
17. notably — e.g., 'Notably, AeroPorter fails closed when destinations are uncalibrated.'
18. importantly — e.g., 'Importantly, AeroPorter fails closed when destinations are uncalibrated.'
19. crucially — e.g., 'Crucially, AeroPorter fails closed when destinations are uncalibrated.'
20. ultimately — e.g., 'Ultimately, AeroPorter fails closed when destinations are uncalibrated.'
21. initially — e.g., 'Initially, AeroPorter fails closed when destinations are uncalibrated.'
22. subsequently — e.g., 'Subsequently, AeroPorter fails closed when destinations are uncalibrated.'
23. previously — e.g., 'Previously, AeroPorter fails closed when destinations are uncalibrated.'
24. meanwhile — e.g., 'Meanwhile, AeroPorter fails closed when destinations are uncalibrated.'
25. simultaneously — e.g., 'Simultaneously, AeroPorter fails closed when destinations are uncalibrated.'
26. alternatively — e.g., 'Alternatively, AeroPorter fails closed when destinations are uncalibrated.'
27. otherwise — e.g., 'Otherwise, AeroPorter fails closed when destinations are uncalibrated.'
28. instead — e.g., 'Instead, AeroPorter fails closed when destinations are uncalibrated.'
29. rather — e.g., 'Rather, AeroPorter fails closed when destinations are uncalibrated.'
30. indeed — e.g., 'Indeed, AeroPorter fails closed when destinations are uncalibrated.'
31. in fact — e.g., 'In fact, AeroPorter fails closed when destinations are uncalibrated.'
32. clearly — e.g., 'Clearly, AeroPorter fails closed when destinations are uncalibrated.'
33. obviously (use carefully) — e.g., 'Obviously (use carefully), AeroPorter fails closed when destinations are uncalibrated.'
34. admittedly — e.g., 'Admittedly, AeroPorter fails closed when destinations are uncalibrated.'
35. certainly — e.g., 'Certainly, AeroPorter fails closed when destinations are uncalibrated.'
36. presumably — e.g., 'Presumably, AeroPorter fails closed when destinations are uncalibrated.'
37. reportedly — e.g., 'Reportedly, AeroPorter fails closed when destinations are uncalibrated.'
38. respectively — e.g., 'Respectively, AeroPorter fails closed when destinations are uncalibrated.'
39. collectively — e.g., 'Collectively, AeroPorter fails closed when destinations are uncalibrated.'
40. overall — e.g., 'Overall, AeroPorter fails closed when destinations are uncalibrated.'
41. in summary — e.g., 'In summary, AeroPorter fails closed when destinations are uncalibrated.'
42. to summarize — e.g., 'To summarize, AeroPorter fails closed when destinations are uncalibrated.'
43. in conclusion — e.g., 'In conclusion, AeroPorter fails closed when destinations are uncalibrated.'
44. as a result — e.g., 'As a result, AeroPorter fails closed when destinations are uncalibrated.'
45. as such — e.g., 'As such, AeroPorter fails closed when destinations are uncalibrated.'

46. hence — e.g., 'Hence, AeroPorter fails closed when destinations are uncalibrated.'
47. thus — e.g., 'Thus, AeroPorter fails closed when destinations are uncalibrated.'
48. thereby — e.g., 'Thereby, AeroPorter fails closed when destinations are uncalibrated.'
49. thereof — e.g., 'Thereof, AeroPorter fails closed when destinations are uncalibrated.'
50. wherein — e.g., 'Wherein, AeroPorter fails closed when destinations are uncalibrated.'
51. whereby — e.g., 'Whereby, AeroPorter fails closed when destinations are uncalibrated.'
52. thereafter — e.g., 'Thereafter, AeroPorter fails closed when destinations are uncalibrated.'
53. hereinafter (docs) — e.g., 'Hereinafter (docs), AeroPorter fails closed when destinations are uncalibrated.'
54. aforementioned — e.g., 'Aforementioned, AeroPorter fails closed when destinations are uncalibrated.'
55. corresponding — e.g., 'Corresponding, AeroPorter fails closed when destinations are uncalibrated.'
56. comparable — e.g., 'Comparable, AeroPorter fails closed when destinations are uncalibrated.'
57. conversely — e.g., 'Conversely, AeroPorter fails closed when destinations are uncalibrated.'
58. equally — e.g., 'Equally, AeroPorter fails closed when destinations are uncalibrated.'
59. explicitly — e.g., 'Explicitly, AeroPorter fails closed when destinations are uncalibrated.'
60. implicitly — e.g., 'Implicitly, AeroPorter fails closed when destinations are uncalibrated.'
61. essentially — e.g., 'Essentially, AeroPorter fails closed when destinations are uncalibrated.'
62. effectively — e.g., 'Effectively, AeroPorter fails closed when destinations are uncalibrated.'
63. practically — e.g., 'Practically, AeroPorter fails closed when destinations are uncalibrated.'
64. theoretically — e.g., 'Theoretically, AeroPorter fails closed when destinations are uncalibrated.'
65. empirically — e.g., 'Empirically, AeroPorter fails closed when destinations are uncalibrated.'
66. quantitatively — e.g., 'Quantitatively, AeroPorter fails closed when destinations are uncalibrated.'
67. qualitatively — e.g., 'Qualitatively, AeroPorter fails closed when destinations are uncalibrated.'
68. primarily — e.g., 'Primarily, AeroPorter fails closed when destinations are uncalibrated.'
69. secondarily — e.g., 'Secondarily, AeroPorter fails closed when destinations are uncalibrated.'
70. partially — e.g., 'Partially, AeroPorter fails closed when destinations are uncalibrated.'
71. fully — e.g., 'Fully, AeroPorter fails closed when destinations are uncalibrated.'
72. approximately — e.g., 'Approximately, AeroPorter fails closed when destinations are uncalibrated.'
73. roughly — e.g., 'Roughly, AeroPorter fails closed when destinations are uncalibrated.'
74. precisely — e.g., 'Precisely, AeroPorter fails closed when destinations are uncalibrated.'
75. exactly — e.g., 'Exactly, AeroPorter fails closed when destinations are uncalibrated.'
76. nearly — e.g., 'Nearly, AeroPorter fails closed when destinations are uncalibrated.'
77. virtually — e.g., 'Virtually, AeroPorter fails closed when destinations are uncalibrated.'
78. largely — e.g., 'Largely, AeroPorter fails closed when destinations are uncalibrated.'
79. mainly — e.g., 'Mainly, AeroPorter fails closed when destinations are uncalibrated.'
80. chiefly — e.g., 'Chiefly, AeroPorter fails closed when destinations are uncalibrated.'
81. especially — e.g., 'Especially, AeroPorter fails closed when destinations are uncalibrated.'
82. particularly — e.g., 'Particularly, AeroPorter fails closed when destinations are uncalibrated.'
83. generally — e.g., 'Generally, AeroPorter fails closed when destinations are uncalibrated.'
84. typically — e.g., 'Typically, AeroPorter fails closed when destinations are uncalibrated.'
85. usually — e.g., 'Usually, AeroPorter fails closed when destinations are uncalibrated.'
86. rarely — e.g., 'Rarely, AeroPorter fails closed when destinations are uncalibrated.'
87. frequently — e.g., 'Frequently, AeroPorter fails closed when destinations are uncalibrated.'
88. occasionally — e.g., 'Occasionally, AeroPorter fails closed when destinations are uncalibrated.'
89. eventually — e.g., 'Eventually, AeroPorter fails closed when destinations are uncalibrated.'
90. gradually — e.g., 'Gradually, AeroPorter fails closed when destinations are uncalibrated.'
91. suddenly — e.g., 'Suddenly, AeroPorter fails closed when destinations are uncalibrated.'
92. immediately — e.g., 'Immediately, AeroPorter fails closed when destinations are uncalibrated.'
93. promptly — e.g., 'Promptly, AeroPorter fails closed when destinations are uncalibrated.'
94. temporarily — e.g., 'Temporarily, AeroPorter fails closed when destinations are uncalibrated.'

95. permanently — e.g., 'Permanently, AeroPorter fails closed when destinations are uncalibrated.'
96. currently — e.g., 'Currently, AeroPorter fails closed when destinations are uncalibrated.'
97. formerly — e.g., 'Formerly, AeroPorter fails closed when destinations are uncalibrated.'
98. beforehand — e.g., 'Beforehand, AeroPorter fails closed when destinations are uncalibrated.'
99. afterwards — e.g., 'Afterwards, AeroPorter fails closed when destinations are uncalibrated.'
100. herein — e.g., 'Herein, AeroPorter fails closed when destinations are uncalibrated.'
101. herewith — e.g., 'Herewith, AeroPorter fails closed when destinations are uncalibrated.'
102. hereinafter — e.g., 'Hereinafter, AeroPorter fails closed when destinations are uncalibrated.'
103. notwithstanding — e.g., 'Notwithstanding, AeroPorter fails closed when destinations are uncalibrated.'
104. albeit — e.g., 'Albeit, AeroPorter fails closed when destinations are uncalibrated.'
105. whereas — e.g., 'Whereas, AeroPorter fails closed when destinations are uncalibrated.'
106. therein — e.g., 'Therein, AeroPorter fails closed when destinations are uncalibrated.'
107. thereupon — e.g., 'Thereupon, AeroPorter fails closed when destinations are uncalibrated.'
108. henceforth — e.g., 'Henceforth, AeroPorter fails closed when destinations are uncalibrated.'
109. insofar as — e.g., 'Insofar as, AeroPorter fails closed when destinations are uncalibrated.'
110. inasmuch as — e.g., 'Inasmuch as, AeroPorter fails closed when destinations are uncalibrated.'

11.2 Professional engineering expressions (106)

1. trade-off between X and Y — e.g., used when discussing AeroPorter's trade-off aspects.
2. within project constraints — e.g., used when discussing AeroPorter's within aspects.
3. from an engineering standpoint — e.g., used when discussing AeroPorter's from aspects.
4. in controlled conditions — e.g., used when discussing AeroPorter's in aspects.
5. under supervised operation — e.g., used when discussing AeroPorter's under aspects.
6. fail-safe by design — e.g., used when discussing AeroPorter's fail-safe aspects.
7. defense in depth — e.g., used when discussing AeroPorter's defense aspects.
8. single point of failure — e.g., used when discussing AeroPorter's single aspects.
9. mean time to stop — e.g., used when discussing AeroPorter's mean aspects.
10. operational design domain — e.g., used when discussing AeroPorter's operational aspects.
11. residual risk — e.g., used when discussing AeroPorter's residual aspects.
12. safety case — e.g., used when discussing AeroPorter's safety aspects.
13. threat model — e.g., used when discussing AeroPorter's threat aspects.
14. attack surface — e.g., used when discussing AeroPorter's attack aspects.
15. least privilege — e.g., used when discussing AeroPorter's least aspects.
16. graceful degradation — e.g., used when discussing AeroPorter's graceful aspects.
17. human in the loop — e.g., used when discussing AeroPorter's human aspects.
18. supervisor in the loop — e.g., used when discussing AeroPorter's supervisor aspects.
19. end-to-end pipeline — e.g., used when discussing AeroPorter's end-to-end aspects.
20. critical path — e.g., used when discussing AeroPorter's critical aspects.
21. root cause analysis — e.g., used when discussing AeroPorter's root aspects.
22. bring-up sequence — e.g., used when discussing AeroPorter's bring-up aspects.
23. integration risk — e.g., used when discussing AeroPorter's integration aspects.
24. interface contract — e.g., used when discussing AeroPorter's interface aspects.
25. configuration drift — e.g., used when discussing AeroPorter's configuration aspects.
26. bit rot — e.g., used when discussing AeroPorter's bit aspects.
27. technical debt — e.g., used when discussing AeroPorter's technical aspects.
28. reproducible demo — e.g., used when discussing AeroPorter's reproducible aspects.
29. ground truth — e.g., used when discussing AeroPorter's ground aspects.
30. error budget — e.g., used when discussing AeroPorter's error aspects.
31. latency budget — e.g., used when discussing AeroPorter's latency aspects.

32. power budget — e.g., used when discussing AeroPorter's power aspects.
33. thermal headroom — e.g., used when discussing AeroPorter's thermal aspects.
34. compute headroom — e.g., used when discussing AeroPorter's compute aspects.
35. false positive rate — e.g., used when discussing AeroPorter's false aspects.
36. false negative rate — e.g., used when discussing AeroPorter's false aspects.
37. occupancy belief — e.g., used when discussing AeroPorter's occupancy aspects.
38. pose estimate — e.g., used when discussing AeroPorter's pose aspects.
39. scan score — e.g., used when discussing AeroPorter's scan aspects.
40. path cost — e.g., used when discussing AeroPorter's path aspects.
41. goal tolerance — e.g., used when discussing AeroPorter's goal aspects.
42. keep-out zone — e.g., used when discussing AeroPorter's keep-out aspects.
43. pilot zone — e.g., used when discussing AeroPorter's pilot aspects.
44. TRL advancement — e.g., used when discussing AeroPorter's TRL aspects.
45. BOM estimate — e.g., used when discussing AeroPorter's BOM aspects.
46. total cost of ownership — e.g., used when discussing AeroPorter's total aspects.
47. return on attention — e.g., used when discussing AeroPorter's return aspects.
48. claim hygiene — e.g., used when discussing AeroPorter's claim aspects.
49. source of truth — e.g., used when discussing AeroPorter's source aspects.
50. executable specification — e.g., used when discussing AeroPorter's executable aspects.
51. observability gap — e.g., used when discussing AeroPorter's observability aspects.
52. fault injection — e.g., used when discussing AeroPorter's fault aspects.
53. cable-pull test — e.g., used when discussing AeroPorter's cable-pull aspects.
54. lease expiry — e.g., used when discussing AeroPorter's lease aspects.
55. stale command — e.g., used when discussing AeroPorter's stale aspects.
56. watchdog trip — e.g., used when discussing AeroPorter's watchdog aspects.
57. brownout event — e.g., used when discussing AeroPorter's brownout aspects.
58. USB enumeration race — e.g., used when discussing AeroPorter's USB aspects.
59. map frame consistency — e.g., used when discussing AeroPorter's map aspects.
60. odometry drift — e.g., used when discussing AeroPorter's odometry aspects.
61. scan matching lock — e.g., used when discussing AeroPorter's scan aspects.
62. cross-track error — e.g., used when discussing AeroPorter's cross-track aspects.
63. look-ahead distance — e.g., used when discussing AeroPorter's look-ahead aspects.
64. inflation radius — e.g., used when discussing AeroPorter's inflation aspects.
65. temporary TTL — e.g., used when discussing AeroPorter's temporary aspects.
66. holonomic base — e.g., used when discussing AeroPorter's holonomic aspects.
67. inverse kinematics — e.g., used when discussing AeroPorter's inverse aspects.
68. wheel slip — e.g., used when discussing AeroPorter's wheel aspects.
69. floor dependence — e.g., used when discussing AeroPorter's floor aspects.
70. calibrated destination — e.g., used when discussing AeroPorter's calibrated aspects.
71. fail-closed gate — e.g., used when discussing AeroPorter's fail-closed aspects.
72. allowlist enforcement — e.g., used when discussing AeroPorter's allowlist aspects.
73. origin check — e.g., used when discussing AeroPorter's origin aspects.
74. session lease — e.g., used when discussing AeroPorter's session aspects.
75. escort completion rate — e.g., used when discussing AeroPorter's escort aspects.
76. intervention count — e.g., used when discussing AeroPorter's intervention aspects.
77. human factors — e.g., used when discussing AeroPorter's human aspects.
78. passenger journey — e.g., used when discussing AeroPorter's passenger aspects.
79. language barrier — e.g., used when discussing AeroPorter's language aspects.
80. accessibility requirement — e.g., used when discussing AeroPorter's accessibility aspects.

81. privacy by design — e.g., used when discussing AeroPorter's privacy aspects.
82. data minimization — e.g., used when discussing AeroPorter's data aspects.
83. PII redaction — e.g., used when discussing AeroPorter's PII aspects.
84. chain of custody (logs) — e.g., used when discussing AeroPorter's chain aspects.
85. standard operating procedure — e.g., used when discussing AeroPorter's standard aspects.
86. change management — e.g., used when discussing AeroPorter's change aspects.
87. versioned map — e.g., used when discussing AeroPorter's versioned aspects.
88. signed-off waypoint — e.g., used when discussing AeroPorter's signed-off aspects.
89. incident severity — e.g., used when discussing AeroPorter's incident aspects.
90. near miss — e.g., used when discussing AeroPorter's near aspects.
91. corrective action — e.g., used when discussing AeroPorter's corrective aspects.
92. preventive action — e.g., used when discussing AeroPorter's preventive aspects.
93. acceptance criterion — e.g., used when discussing AeroPorter's acceptance aspects.
94. verification vs validation — e.g., used when discussing AeroPorter's verification aspects.
95. smoke test — e.g., used when discussing AeroPorter's smoke aspects.
96. soak test — e.g., used when discussing AeroPorter's soak aspects.
97. regression — e.g., used when discussing AeroPorter's regression aspects.
98. canary change — e.g., used when discussing AeroPorter's canary aspects.
99. rollback plan — e.g., used when discussing AeroPorter's rollback aspects.
100. feature flag — e.g., used when discussing AeroPorter's feature aspects.
101. scope creep — e.g., used when discussing AeroPorter's scope aspects.
102. parking-lot list — e.g., used when discussing AeroPorter's parking-lot aspects.
103. non-goal — e.g., used when discussing AeroPorter's non-goal aspects.
104. explicit non-claim — e.g., used when discussing AeroPorter's explicit aspects.
105. honest boundary — e.g., used when discussing AeroPorter's honest aspects.
106. evidence over adjectives — e.g., used when discussing AeroPorter's evidence aspects.

11.3 Ways to buy thinking time (50)

1. That's a good question — let me structure the answer.
2. Let me separate what we implemented from what we plan.
3. I want to be precise, so I'll answer in two layers.
4. First the short version, then the mechanism.
5. Let me recall the exact timeout values.
6. I'll start with the safety behavior, then the feature.
7. Allow me one second to choose the most accurate claim.
8. There are two ways to interpret the question — I'll take the engineering one.
9. Let me point to the subsystem responsible.
10. I'll hand the electronics part to my teammate after a brief overview.
11. Let me avoid a marketing answer and give a technical one.
12. I can answer qualitatively now and quantitatively after a measurement.
13. Let me restate the question to confirm I understood.
14. There is a short answer and a deeper answer — which do you prefer? I'll give short first.
15. Let me anchor this in the architecture diagram.
16. I'll use the failure-mode framing.
17. Let me compare options briefly, then state our choice.
18. I want to include the limitation explicitly.
19. Let me give the decision, the trade-off, then the residual risk.
20. I'll answer with the test we would run.
21. Let me map this to our truth table of claims.

22. One moment — I will not invent a number.
23. Let me explain the interface boundaries first.
24. I'll start from the passenger journey, then dive into code.
25. Let me isolate whether you ask about safety or security.
26. Good catch — that touches a known limitation.
27. Let me show the layered stop logic.
28. I'll clarify terminology before the details.
29. Let me avoid conflating Nav2 with our custom stack.
30. I'll answer regarding controlled conditions only.
31. Let me check I'm not overclaiming deployment status.
32. I'll give an analogy, then the real mechanism.
33. Let me sequence the timeline of bring-up.
34. I'll distinguish demo scaffolding from production integration.
35. Let me phrase this without buzzwords.
36. I'll answer as three bullets.
37. Let me prioritize the safety-critical part of your question.
38. I'll defer legal details and stick to engineering controls we own.
39. Let me reframe toward measurable KPIs.
40. I'll state the assumption behind our design.
41. One second to choose the best example from our tests.
42. Let me avoid contradicting our README truth.
43. I'll answer hardware first, then software.
44. Let me highlight what would falsify our claim.
45. I'll keep this under thirty seconds unless you want depth.
46. Let me connect this to the heartbeat lease.
47. I'll start from 'motion is a privilege'.
48. Let me put numbers only where we measured them.
49. I'll invite a follow-up on methodology.
50. Thank you — this question is exactly about our honesty boundary.

11.4 Polite disagreement (50)

1. I see your point; our measurements suggest a different bottleneck.
2. That's a valid concern — here's how we mitigate it today.
3. We considered that approach; we rejected it because...
4. I agree with the risk, not with the conclusion that the design is invalid.
5. You're right that Nav2 is powerful; we still chose transparency for this project.
6. I understand the preference for Jetson; our workload doesn't justify it yet.
7. That's fair for a production AMR; our TRL is prototype-class.
8. I agree glass is hard; that is why we don't claim unsupervised halls of glass.
9. We share the concern about LAN attacks; allowlisting is a baseline, not the end state.
10. I take the cost critique seriously — exclusions are listed for that reason.
11. Respectfully, 'always works' is not a claim we make.
12. I agree staff will still be needed; the robot reduces routine load, not responsibility.
13. That's an important legal issue; engineering readiness isn't legal readiness.
14. I understand why it sounds like teleop-only; the planner/tracker are real blocks.
15. We don't disagree that industrial robots exist; we differ on task fit and explainability.
16. You're right that demo tickets aren't airport truth; they're lab scaffolding.
17. I accept that our statistical N may be small; here's the next measurement plan.
18. I agree the enclosure is incomplete; that's why public deployment isn't claimed.

19. We hear the 'use GPS' suggestion; indoors it's the wrong sensor class.
20. I agree buzzwords are tempting; we prefer algorithm names.
21. That's a reasonable alternative architecture; here's our trade-off table.
22. I understand the push for fleet; we refuse scope creep before one-robot pilot metrics.
23. You're right that HTTP isn't ideal forever; it's pragmatic for the current UI.
24. I agree Soft real-time isn't hard certification; MCU timeout backs us up.
25. We don't claim ISO; we claim layered stops we can demonstrate.
26. I understand wanting a famous LiDAR brand name; we won't invent one.
27. That's fair criticism of odometry drift; scan matching is our mitigation.
28. I agree Pepper wins social presence; we optimize escort navigation clarity.
29. Respectfully, replacing human PRM services isn't our claim.
30. I understand 'just buy an AMR'; education and ownership are part of our goals.
31. We accept the FR encoder criticism; mitigation is documented.
32. I agree public Wi-Fi control would be reckless; we advise against it.
33. That's a sharp point on privacy; we don't do biometric boarding.
34. I understand map renovations break goals; recalibration is mandatory.
35. You're right that indication lights are needed; listed as remaining work.
36. I agree voice could help later; it's not our proven core now.
37. We don't dismiss ultrasonics; they're insufficient as primary mapping.
38. I understand wanting cloud tickets; local demo must still fail safe.
39. That's a good challenge on look-ahead cutting corners; we tune and inflate.
40. I agree 'AI robot' is weak branding for defense.
41. Respectfully, airside is out of our current claim set.
42. I understand the desire for a single magic accuracy number; conditions matter.
43. We accept that allowlists aren't zero trust.
44. I agree SOP training is required for pilots.
45. That's fair — slides must match the repo; we tightened claims accordingly.
46. I understand why fail-open feels user-friendly; fail-closed is safer.
47. You're right that supervision costs money; that's real OPEX.
48. I agree we must not block evacuation routes.
49. We hear 'add more ML'; classical planning still fits our defendable MVP.
50. Thank you — we'll adopt that test if it falsifies a risk we care about.

11.5 Discussing limitations (50)

1. This is a known limitation we document explicitly.
2. We constrain the claim to controlled conditions.
3. That feature is implemented as a demo path, not a certified ops path.
4. We fail closed rather than guess.
5. Our map quality depends on careful mapping runs.
6. Glass and dark surfaces reduce LiDAR reliability.
7. FR encoder noise forced a three-wheel odometry design.
8. Destinations must be measured before escort is enabled.
9. Demo tickets may be used when the airport DB isn't connected.
10. BarcodeDetector depends on browser support.
11. We are not claiming Nav2-level industrial navigation.
12. Enclosure and soft covers are still required.
13. Public indication is incomplete.
14. Stop latency must be measured on each hardware revision.
15. Carpet changes drive calibration.

16. Crowds need more field validation.
17. Security is gated but not fully hardened for hostile networks.
18. Airside operation is out of scope for current claims.
19. Live FIDS integration is future work.
20. Voice guidance isn't a proven core feature.
21. Fleet coordination isn't implemented.
22. Statistical sample sizes may still be small.
23. USB device order can be operationally fragile.
24. Soft real-time Linux isn't a hard-real-time certificate.
25. Battery endurance needs soak testing for full shifts.
26. Legal/insurance readiness is separate from software readiness.
27. One robot doesn't cover a whole campus.
28. Scan matching can fail in degenerate geometry.
29. Temporary obstacles don't solve all social navigation.
30. We don't claim biometric identification.
31. Privacy processes for real PII are not 'done'.
32. Operator skill affects map quality.
33. Teleop may be required in edge cases.
34. We don't publish a universal centimeter SLA.
35. Cost figures exclude LiDAR and industrial enclosure.
36. Analog comparisons are task-fit, not absolute superiority.
37. Some older slide wording was tightened to match the repo.
38. Unknown space is not treated as free.
39. Wi-Fi interference can affect peripherals.
40. We accept residual risk under supervision.
41. Pilot success needs airport process, not only code.
42. Hardware e-stop buttons still need industrial implementation.
43. Thermal throttling can appear under sustained load.
44. Documentation must travel with the robot for safe ops.
45. We refuse unsafe overclaims even if they sound impressive.
46. Limitations are part of the engineering story, not an apology-only appendix.
47. If a limit is safety-critical, it becomes a non-claim or a test.
48. We prefer an honest 'not yet' to a fragile 'yes'.
49. Each limitation has a mitigation or a roadmap owner.
50. The boundary between demo and deployment is intentional.

11.6 Future improvements (50)

1. Next we will harden the enclosure and human-factors indication.
2. A hardware e-stop is high on the list.
3. We will film stop-latency fault-injection tests.
4. Crowd-like supervised trials are the next validation step.
5. Live flight data adapters can plug in without rewriting safety.
6. API authentication will be strengthened for pilot networks.
7. Network segmentation guidance will accompany IT discussions.
8. FR encoder hardware replacement is planned.
9. Battery SoC monitoring will gate motion at low charge.
10. We may evaluate Jetson if neural workload grows.
11. Nav2 migration remains a criteria-based option, not a trophy.
12. Richer local planners can be added if crowds demand them.

13. Glass mitigation experiments will combine sensors and policy.
14. Map versioning and destination sign-off will be formalized.
15. Staff SOP one-pagers will be written before any pilot.
16. An incident severity scale will structure reporting.
17. Metric dashboards will track escort KPIs.
18. Accessibility audits will review font size and flows.
19. Privacy review will precede any real PII.
20. Spare-parts kits will support maintainability.
21. Soak tests will characterize heat and endurance.
22. Queue UX for a second passenger will be designed.
23. Fleet remains explicitly later.
24. TRL will be tracked with evidence, not adjectives.
25. Partner conversations will seek a supervised zone.
26. Claim lint will run before every public show.
27. Optional Tapo PTZ may merge carefully into the main camera path.
28. Acoustic comfort tuning will follow speed policies.
29. Rollback procedures for bad maps/configs will be documented.
30. We will keep non-goals visible to prevent scope creep.
31. Open-sourcing decisions will separate public and private parts.
32. Charger docking is a possible mobility convenience later.
33. Stronger relocalization UX for operators is planned.
34. Temporary obstacle parameters will be retuned from field logs.
35. We will add clearer busy-state messaging on the kiosk.
36. Multilingual content will expand with native review.
37. Security tests will include unauthorized control attempts on LAN.
38. We will define keep-out polygons for evacuation routes.
39. Human handoff phrasing will be standardized.
40. Post-defense experiment #1 is stop-on-lease-loss on video.
41. Enclosure materials will prioritize soft contact surfaces.
42. Lighting for barcode reliability will be tested.
43. We will document USB udev rules for stable device names.
44. Calibration checklists will be made operator-proof.
45. We will separate lab demo mode from pilot mode in software flags.
46. Longer corridors will test scan-match drift behavior.
47. We will study cart and stroller obstacle classes.
48. Partnership success means shared KPI ownership with the airport.
49. Future slides will keep the same honesty boundary.
50. Improvements must not weaken layered timeouts.

11.7 Describing test results (50)

1. In our cable-pull test, motion stopped within the watchdog bound.
2. When we paused heartbeats, escort halted after lease expiry.
3. A→B runs completed in a mapped corridor under supervision.
4. Scan matching reduced noticeable heading drift compared with odometry alone.
5. On carpet, we observed higher slip and retuned drive_cal.
6. Barcode scans succeeded under good lighting; manual entry covered failures.
7. Demo ticket lookup returned expected flight fields.
8. Uncalibrated destinations correctly disabled escort.
9. Temporary obstacles caused replanning around a blocking person.

10. Teleop override immediately took control from autonomy.
11. UI blur triggered a stop in the operator console path.
12. CPU load remained acceptable on Pi 5 during planning demos.
13. Map autosave restored occupancy after restart.
14. Three-wheel odometry behaved more stably than including FR.
15. Self-filter removed chassis posts from the map.
16. Goal tolerance determined practical arrival behavior.
17. Inflation that was too large blocked narrow doors — we retuned.
18. Look-ahead too large cut corners; smaller values tracked better.
19. Stop-latency is the KPI we trust most for safety talks.
20. We log interventions per session as an ops metric.
21. Failure taxonomy includes localization loss, blockages, and UI disconnects.
22. Repeated runs showed qualitative repeatability in the same map.
23. Glass scenes produced optimistic free space — marked as risk.
24. USB reordering once broke device assignment — checklist added.
25. Soak runs surfaced thermal throttling under sustained load.
26. Power brownouts appeared when motor supply was undersized.
27. Unit tests cover planner basics; field tests cover people.
28. We treat N as small until more trials are recorded.
29. Video evidence beats slide adjectives.
30. Escort completion rate is reported with conditions attached.
31. False obstacle speckles decreased after mapping hygiene.
32. Phantom walls led to remap, not 'push through'.
33. Operator training reduced bad goal placements.
34. Language switching did not break primary CTA layout.
35. Ticket status filters rejected non-accepted states.
36. Browser without BarcodeDetector still worked via typing.
37. Lease busy state prevented dual control conflicts in tests.
38. Watchdog zeroed stale commands when publisher froze.
39. Mega timeout stopped motors when serial went silent.
40. We prefer median and worst-case stop times, not best-case only.
41. Results are environment-dependent — floors matter.
42. We distinguish smoke tests from endurance soaks.
43. Regressions are caught when an old route fails after changes.
44. A bad map version was rolled back successfully.
45. Calibration sign-off prevented a wrong desk coordinate demo.
46. Crowd-ish tests increased intervention count as expected.
47. Quiet halls are not proof for peak waves.
48. Metrics without methodology are rejected in our own reviews.
49. Negative results are kept — they guide mitigations.
50. The best test answer includes setup, metric, and limit.

11.8 Comparing with analogs (50)

1. Compared with Pepper, we trade humanoid expressiveness for transparent navigation ownership.
2. Compared with Sanbot, our focus is ticket-to-escort rather than broad social animation.
3. Compared with Promobot info robots, we emphasize calibrated A→B mobility.
4. Compared with fixed kiosks, we can move with the passenger.
5. Compared with phone indoor maps, we help users who struggle with phones.
6. Compared with industrial AMRs, we are lower TRL and lower cost, with educational transparency.

7. Compared with pure teleop toys, we add mapping, planning, and passenger workflow.
8. Compared with Nav2-centric builds, we optimize explainability for defense.
9. Compared with vision-only navigators, LiDAR ranging is more robust for structure.
10. Compared with ultrasonic bumpers alone, LiDAR provides dense geometry.
11. Compared with differential drives, mecanum improves sideways maneuvers in crowds.
12. Compared with Jetson-first builds, Pi 5 matches our current compute need.
13. Compared with cloud-dependent assistants, our demo path is local-first.
14. Compared with biometric gates, we only center faces for UX — not identity.
15. Compared with staff escorts, we scale routine guidance but still need supervision.
16. Compared with digital signage, we provide personalized ticket context.
17. Compared with generic service robots, airport semantics are first-class.
18. Compared with closed vendor stacks, our code paths are auditable by students.
19. Compared with research SLAM demos, we attach a concrete passenger journey.
20. Compared with AGV tape systems, we use free grid planning on a map.
21. On cost class, we are prototype BOM, not airline procurement.
22. On social presence, Pepper-class robots usually win.
23. On task clarity, ticket→info→escort is our edge.
24. On certification, industrial platforms win — we don't fake that.
25. On multilingual inclusion, shipping TT/EN/ZH/RU is a practical plus.
26. On safety storytelling, layered timeouts are easy to demonstrate.
27. On maintenance, simpler mechanics can be an advantage.
28. On map dependence, we require calibration discipline.
29. On crowd social nav papers, we are not claiming SOTA.
30. On privacy posture, avoiding biometric claims is deliberate.
31. On deployment readiness, fixed kiosks still win for zero motion risk.
32. On learning value, building the stack beats only integrating a black box.
33. On speed of bringing a UI, browser kiosks are productive.
34. On ruggedness, industrial AMRs win until we enclosure-harden.
35. On explainability per ruble, we aim to lead.
36. On airside legality, none of the student stacks should casually claim readiness.
37. On glass atriums, all LiDAR robots share pain — we admit it.
38. On staff trust, honest limits beat hype.
39. On passenger stress, a moving guide can beat another poster.
40. On analog slides, we compare metrics, not vibes.
41. On Promobot heritage, we respect the category and specify differences.
42. On Sanbot, similar respect — different vertical slice.
43. On Pepper, different chassis philosophy entirely.
44. On fixed kiosks, complementary, not always competitive.
45. On phone apps, complementary for digitally confident users.
46. On 'already exists', integration depth still matters.
47. On fairness, use the same environment assumptions when comparing.
48. On marketing, never say 'better at everything'.
49. On jury questions, lead with shared metrics: cost class, TRL, task fit, safety evidence.
50. On our own ego, analogs are teachers, not enemies.

Bonus

B1. Pronunciation tips

1. LiDAR — LIE-dar
2. mecanum — meh-CAN-um
3. odometry — oh-DOM-uh-tree
4. quaternion — kwa-TUR-nee-un
5. heuristic — hyu-RIS-tik
6. occupancy — OCK-yuh-pan-see
7. calibration — cal-ih-BRAY-shun
8. enclosure — en-CLOH-zhur
9. baud — bawd
10. GPIO — jee-pee-eye-oh
11. UART — you-art
12. PWM — pee-double-you-em
13. encoder — en-COH-der
14. algorithm — AL-guh-rith-um
15. autonomous — aw-TON-uh-mus
16. prototype — PRO-tuh-type
17. architecture — AR-ki-tek-chur
18. deployment — dih-PLOY-ment
19. latency — LAY-tun-see
20. throughput — THROUGH-put
21. redundant — rih-DUN-dunt
22. resilient — rih-ZIL-yunt
23. mitigation — mit-ih-GAY-shun
24. hazard — HAZ-erd
25. residual — rih-ZIJ-oo-ul
26. biometric — by-oh-MET-rik
27. privacy — PRY-vuh-see
28. queue — kyoo
29. carousel — kair-uh-SEL
30. boarding — BOR-ding
31. escort — ES-kort (noun/verb stress may vary)
32. kiosk — KEE-osk
33. multilingual — mul-tee-LING-gwul
34. Tatar — TAH-tar
35. Arduino — ar-DWEE-no
36. Raspberry — RAZ-bair-ee
37. firmware — FIRM-wair
38. middleware — MID-ul-wair
39. deterministic — dih-tur-min-ISS-tik
40. probabilistic — prob-uh-bih-LISS-tik
41. correlative — kuh-REL-uh-tiv
42. iterative — IT-uh-ruh-tiv
43. inflation (map) — in-FLAY-shun
44. clearance — CLEER-uns
45. footprint — FOOT-print
46. watchdog — WATCH-dog

47. heartbeat — HART-beet
48. allowlist — uh-LOW-list
49. fail-closed — fail-CLOZED
50. brownout — BROWN-out

B2. Elevator pitches (3 variants)

1. Product: AeroPorter helps lost passengers by scanning a ticket, showing flight info, and escorting them through the terminal on a mapped route.
2. Engineering: It's a Pi 5 + Mega mecanum robot with custom ROS 2 LiDAR mapping, A*, pure pursuit, and layered motion timeouts.
3. Honest: We've built a supervised prototype pipeline you can audit; real airport rollout needs enclosure, SOPs, and a pilot — and we say so.

B3. Emergency recovery if you blank

1. I need five seconds — I'll answer with decision, mechanism, limit.
2. Let me restart from the architecture sentence.
3. I'll hand this to my teammate while I pull the exact value.
4. I don't remember the number; I remember how we measure it.
5. Please repeat the last part of the question.
6. I'll answer the safety angle first because that's critical.
7. My previous sentence was unclear — here's the corrected claim.
8. I blanked on the brand on purpose: we name the driver, not a guess.
9. Let me draw the path: kiosk → planner → Mega → stop layers.
10. I can continue after stating our non-claim on live terminals.

B4. Confidence checklist

1. Truth table memorized (Nav2? no. Live terminal? no. FR? untrusted.)
2. Timeouts memorized (~1.2 / ~0.25 / ~6)
3. Thesis sentence ready
4. Stop demo story ready
5. Cost exclusions ready
6. Analog comparison metric ready
7. Teammate handoff words ready
8. Buy-time phrase ready
9. Limitation phrase ready
10. Closing thank-you ready

B5. Mini glossary — don't confuse

- safety ≠ security — harm to people vs attackers
- accuracy ≠ precision — truth vs repeatability
- mapping ≠ localization — build map vs find pose
- A* ≠ pure pursuit — plan vs track
- CSM ≠ ICP — search vs polish
- ROS 2 ≠ Nav2 — middleware vs nav stack
- odom frame ≠ map frame — smooth local vs world
- fail-open ≠ fail-closed — allow vs deny when unsure
- watchdog ≠ heartbeat lease — stale cmd vs session privilege
- demo ticket ≠ production ticket source — lab vs airport data

Appendix — Count verification

- Technical vocabulary entries: 1983
- Useful phrases: 319
- Jury questions: 250
- Ready answer sets: 250
- Tricky questions: 100
- Follow-up chains: 167
- AeroPorter project vocab entries: 270
- Mock interviews: 12

Generated for AeroPorter defense preparation. Prefer repository truth over older slide overclaims.